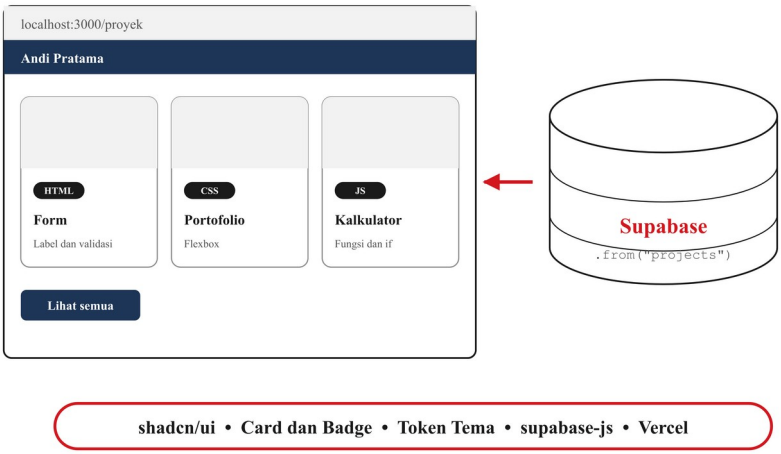
**MODUL KELAS INDUSTRI**

# INTEGRASI NEXT.JS DENGAN SUPABASE DAN SHADCN/UI

Paket 1 Full Stack Basic, Minggu 14 (P1-W14)




Kelas X Semester 1

Kompetensi Keahlian Pengembangan Perangkat Lunak dan Gim


Disusun oleh:

**Tim Kelas Industri UBIG**



**PROGRAM KELAS INDUSTRI**

**PT UNIVERSAL BIG DATA**

Tahun Ajaran 2026/2027

# KATA PENGANTAR

Selamat datang di modul keempat belas Paket 1 *Full Stack Basic* Kelas Industri PT Universal Big Data (UBIG). Di Minggu 13 kamu sudah menyiapkan database portofolio di Supabase: tabel `projects` berisi enam proyek dan tabel `skills` berisi lima keahlian. Namun website `portofolio-next` belum memakainya, karena semua data masih ditulis di dalam kode.

Minggu ini dua hal besar terjadi. Pertama, tampilan portofolio dirapikan dengan **shadcn/ui**, kumpulan komponen siap pakai yang dipakai banyak perusahaan, sehingga kartu, tombol, dan label tampil seragam di mode terang maupun gelap. Kedua, Next.js dihubungkan ke Supabase, sehingga halaman Proyek, Beranda, dan detail proyek menampilkan data langsung dari database, baik di laptop maupun di website *live* Vercel.

Semua langkah di modul ini dipraktikkan sungguhan pada project `portofolio-next` milik Andi Pratama, termasuk *error* yang muncul di tengah jalan. Tampilan CLI shadcn/ui dan dasbor Vercel bisa sedikit berubah dari waktu ke waktu; jika letak tombol berbeda, cari tombol dengan nama yang sama di layarmu.

Malang, September 2026
Tim Kelas Industri UBIG

# DAFTAR ISI

KATA PENGANTAR.....i
DAFTAR ISI.....ii
DAFTAR GAMBAR.....vi
DAFTAR TABEL.....ix
PETUNJUK PENGGUNAAN MODUL.....x
Petunjuk bagi Siswa.....x
Petunjuk bagi Guru dan Mentor.....x
BAB I PENDAHULUAN.....1
1.1 Deskripsi Modul.....1
1.2 Prasyarat.....1
1.3 Tujuan Pembelajaran.....2
1.4 Peta Konsep.....2
1.5 Alokasi Waktu.....3
BAB II KEGIATAN PEMBELAJARAN 1: MENGENAL DAN MEMASANG SHADCN/UI.....4
2.1 Tujuan Kegiatan Pembelajaran.....4
2.2 Uraian Materi.....4
2.2.1 Mengapa Memakai Pustaka Komponen.....4
2.2.2 Cara Kerja shadcn/ui.....5
2.3 Praktik: Memasang shadcn/ui.....6
2.3.1 Tahap 1: Menjalankan shadcn init.....7
2.3.2 Tahap 2: Memeriksa Berkas Baru.....9
2.3.3 Tahap 3: Memperbaiki Huruf yang Berubah.....10
2.3.4 Tahap 4: Menambah Komponen Card, Badge, dan Separator.....13
2.3.5 Tahap 5: Tombol Pertama dari shadcn/ui.....14
2.4 Rangkuman.....16
2.5 Latihan.....16
BAB III KEGIATAN PEMBELAJARAN 2: TOKEN TEMA DAN KOMPONEN CARD.....17
3.1 Tujuan Kegiatan Pembelajaran.....17

3.2 Uraian Materi..... 17
3.2.1 Token Tema..... 17
3.2.2 Komponen Card dan Badge..... 18
3.3 Praktik: Tema dan Komponen..... 19
3.3.1 Tahap 1: Mengatur Warna Utama di globals.css..... 19
3.3.2 Tahap 2: Mengganti Kelas dark: dengan Token..... 21
3.3.3 Tahap 3: ProyekCard dengan Card dan Badge..... 23
3.3.4 Tahap 4: Tombol Kategori dengan Button..... 25
3.4 Rangkuman..... 27
3.5 Latihan..... 27
BAB IV KEGIATAN PEMBELAJARAN 3: MENGAMBIL DATA PROYEK DARI SUPABASE..... 28
4.1 Tujuan Kegiatan Pembelajaran..... 28
4.2 Uraian Materi..... 28
4.2.1 Alur Data dari Supabase ke Browser..... 28
4.2.2 Klien Supabase dan Environment Variable..... 29
4.2.3 Mengambil Data dengan await dan { data, error }..... 29
4.3 Praktik: Data Proyek dari Supabase..... 30
4.3.1 Tahap 1: Memasang supabase-js..... 30
4.3.2 Tahap 2: Membuat src/lib/supabase.js..... 31
4.3.3 Tahap 3: Halaman Proyek Mengambil Data..... 32
4.3.4 Tahap 4: Menyesuaikan ProyekCard dengan Kolom gambar_url..... 35
4.3.5 Tahap 5: Menangani Error dan Data Kosong..... 36
4.3.6 Tahap 6: Proyek Terbaru di Beranda dari Supabase..... 38
4.4 Rangkuman..... 40
4.5 Latihan..... 40
BAB V KEGIATAN PEMBELAJARAN 4: HALAMAN DETAIL DARI DATABASE ..... 41
5.1 Tujuan Kegiatan Pembelajaran..... 41
5.2 Uraian Materi..... 41
5.2.1 Mengambil Satu Baris dengan eq dan single..... 41
5.2.2 Satu Fungsi untuk Dua Tempat..... 42

5.3 Praktik: Halaman Detail dari Database....42
5.3.1 Tahap 1: Mengubah Halaman Detail....42
5.3.2 Tahap 2: Mencoba Slug yang Salah....45
5.3.3 Tahap 3: Menghapus src/data/proyek.js....45
5.4 Rangkuman....47
5.5 Latihan....47
BAB VI KEGIATAN PEMBELAJARAN 5: DEPLOY KE VERCEL DENGAN ENVIRONMENT VARIABLES....48
6.1 Tujuan Kegiatan Pembelajaran....48
6.2 Uraian Materi....48
6.2.1 Mengapa Vercel Perlu Environment Variables Sendiri....48
6.2.2 Halaman Statis dan Dinamis....49
6.3 Praktik: Deploy dengan Environment Variables....50
6.3.1 Tahap 1: Memeriksa dengan lint dan build....50
6.3.2 Tahap 2: Commit dan Push....51
6.3.3 Tahap 3: Membaca Build yang Gagal....52
6.3.4 Tahap 4: Menambahkan Environment Variables....53
6.3.5 Tahap 5: Redeploy....56
6.3.6 Tahap 6: Memeriksa Versi Live....58
6.4 Memeriksa Hasil Kerja....59
6.5 Rangkuman....59
6.6 Latihan....60
6.7 Tugas dan Pengumpulan Minggu Ini....60
6.7.1 Tugas Rumah....60
6.7.2 Tantangan (Opsional)....68
6.7.3 Kriteria Selesai....70
6.7.4 Ketentuan Pengumpulan P1-W14....70
6.7.5 Urutan Mengerjakan Tugas....71
BAB VII EVALUASI....72
7.1 Soal Pilihan Ganda....72
7.2 Soal Uraian....74
7.3 Soal Praktik....74

7.4 Rubrik Penilaian Tugas.....75
BAB VIII PENUTUP.....76
GLOSARIUM.....77
DAFTAR PUSTAKA.....78

# DAFTAR GAMBAR

Gambar 1.1 Peta konsep Modul P1-W14.....3
Gambar 2.1 Halaman Proyek hasil Minggu 13 dengan kartu dan tombol buatan sendiri.5
Gambar 2.2 Pustaka komponen biasa dibandingkan shadcn/ui.....6
Gambar 2.3 npx meminta izin mengunduh paket shadcn.....7
Gambar 2.4 Pertanyaan pustaka komponen: pilih Base UI.....7
Gambar 2.5 Pertanyaan preset: pilih Nova.....8
Gambar 2.6 shadcn init selesai.....8
Gambar 2.7 Berkas yang diubah dan dibuat shadcn init.....9
Gambar 2.8 Isi components.json.....10
Gambar 2.9 Setelah shadcn init, huruf halaman berubah menjadi Times New Roman. .11
Gambar 2.10 layout.js setelah variabel huruf diganti.....12
Gambar 2.11 Huruf halaman kembali normal dengan Geist.....12
Gambar 2.12 Hasil npx shadcn add.....13
Gambar 2.13 Daftar variant dan size di button.jsx.....13
Gambar 2.14 page.js setelah memakai buttonVariants.....15
Gambar 2.15 Tombol Lihat semua proyek bergaya shadcn/ui.....15
Gambar 3.1 Token tema: satu nama kelas, nilai berbeda di mode terang dan gelap.....17
Gambar 3.2 Bagian-bagian Card pada ProyekCard.....18
Gambar 3.3 Token --primary dan --aksen di globals.css.....20
Gambar 3.4 Find and Replace: text-utama dark:text-sky-200 menjadi text-primary.....21
Gambar 3.5 Beranda dengan token tema di mode terang.....22
Gambar 3.6 Beranda dengan token tema di mode gelap.....23
Gambar 3.7 ProyekCard.jsx dengan Card dan Badge.....24
Gambar 3.8 Kartu proyek baru dengan Card dan Badge.....25
Gambar 3.9 FilterProyek.jsx dengan Button.....26
Gambar 3.10 Tombol kategori CSS aktif di mode terang.....26
Gambar 3.11 Tombol kategori CSS aktif di mode gelap.....27
Gambar 4.1 Alur data dari Supabase sampai tampil di browser.....28
Gambar 4.2 Setiap permintaan Supabase menghasilkan { data, error }.....29
Gambar 4.3 Hasil npm install @supabase/supabase-js.....31

Gambar 4.4 Berkas src/lib/supabase.js....32
Gambar 4.5 page.js halaman Proyek dengan await supabase....33
Gambar 4.6 Kartu tampil tanpa gambar....34
Gambar 4.7 Next.js menunjuk src={proyek.gambar} yang kosong....34
Gambar 4.8 Halaman Proyek menampilkan data dari Supabase....35
Gambar 4.9 Pesan console.log server component muncul di terminal....35
Gambar 4.10 Kartu Proyek Terbaru di Beranda kehilangan gambar....36
Gambar 4.11 Pesan saat nama tabel salah....37
Gambar 4.12 Pesan saat tidak ada data....38
Gambar 4.13 Error supabaseUrl is required karena nama variabel salah ketik....38
Gambar 4.14 page.js Beranda mengambil tiga proyek terbaru....39
Gambar 4.15 Proyek Terbaru di Beranda dari Supabase....40
Gambar 5.1 Dari alamat halaman sampai satu baris data....41
Gambar 5.2 Halaman detail dengan fungsi ambilProyek....44
Gambar 5.3 Halaman detail Kalkulator Nilai Rapor dari Supabase....44
Gambar 5.4 Slug yang tidak ada menampilkan halaman 404....45
Gambar 5.5 Pencarian @/data/proyek tidak menemukan hasil....45
Gambar 5.6 Halaman Proyek tetap tampil setelah proyek.js dihapus....46
Gambar 5.7 Module not found karena proyek.js dihapus terlalu cepat....47
Gambar 6.1 Mengapa Vercel perlu Environment Variables sendiri....48
Gambar 6.2 Kapan Next.js mengambil data dari Supabase....49
Gambar 6.3 Hasil npm run build di laptop....50
Gambar 6.4 git status setelah praktik Minggu 14....51
Gambar 6.5 Push praktik Minggu 14....52
Gambar 6.6 Deployment terbaru berstatus Error....52
Gambar 6.7 Build Logs Vercel: supabaseUrl is required....53
Gambar 6.8 Halaman Environment Variables yang masih kosong....54
Gambar 6.9 Dua Environment Variables bertipe Config sebelum disimpan....55
Gambar 6.10 Dua Environment Variables tersimpan....55
Gambar 6.11 Tipe Secret menolak variabel berawalan NEXT_PUBLIC_....56
Gambar 6.12 Jendela konfirmasi Redeploy....57
Gambar 6.13 Redeploy berhasil dengan status Ready....57

Gambar 6.14 Halaman Proyek versi live membaca Supabase.....58
Gambar 6.15 Halaman detail versi live di mode gelap.....59
Gambar 6.16 Contoh hasil tugas: Beranda mengambil skills dan proyek terbaru.....61
Gambar 6.17 Contoh hasil tugas: SkillCard.jsx.....62
Gambar 6.18 Contoh hasil tugas: Beranda dengan keahlian dari Supabase.....65
Gambar 6.19 Contoh hasil tugas: Beranda di mode gelap.....65
Gambar 6.20 Contoh hasil tugas: halaman Tentang dengan Card.....66
Gambar 6.21 Contoh hasil tugas: halaman Kontak dengan Card.....66
Gambar 6.22 Push tugas Minggu 14.....67
Gambar 6.23 Contoh hasil tugas: Beranda versi live.....67
Gambar 6.24 Contoh hasil tantangan: variant baru di badge.jsx.....69
Gambar 6.25 Contoh hasil tantangan: Badge berwarna sesuai kategori.....69
Gambar 7.1 Contoh hasil soal praktik: jadwal belajar dari Supabase.....75

# DAFTAR TABEL

Tabel 1.1 Alokasi waktu Minggu 14.....3
Tabel 2.1 Dua perintah utama shadcn/ui.....6
Tabel 2.2 Berkas yang dibuat atau diubah shadcn init.....9
Tabel 2.3 Variant komponen Button.....14
Tabel 3.1 Token tema yang dipakai di portofolio.....18
Tabel 3.2 Sepuluh pasangan Find and Replace.....22
Tabel 4.1 Method supabase-js yang dipakai minggu ini.....30
Tabel 6.1 Tipe Environment Variables di Vercel.....49
Tabel 6.2 Isian formulir Add Environment Variable.....54
Tabel 6.3 Cara memeriksa hasil kerja Minggu 14.....59
Tabel 6.4 Petunjuk tugas Minggu 14.....68
Tabel 6.5 Ketentuan pengumpulan P1-W14.....70
Tabel 7.1 Rubrik penilaian tugas P1-W14.....75

# PETUNJUK PENGGUNAAN MODUL

**Petunjuk bagi Siswa**

1. Siapkan project `portofolio-next` hasil tugas Minggu 13, termasuk berkas `.env.local` yang berisi Project URL dan *publishable key* Supabase.
2. Pertemuan dibuka dengan **kuis P1-W13 selama 10 menit**. Pelajari kembali bab Evaluasi Modul P1-W13 sebelum pertemuan.
3. Pastikan internet stabil. Minggu ini kamu memasang paket dari npm dan mengambil data dari Supabase.
4. Ikuti tahap praktik secara berurutan. Setiap tahap hanya menunjukkan bagian kode yang berubah, lalu tangkapan layar hasilnya.
5. Soal pilihan ganda dan uraian bab Evaluasi menjadi **kuis 10 menit** di awal pertemuan P1-W15. Soal praktiknya dikerjakan sebagai remedial jika tugasmu belum memenuhi kriteria selesai. Ketentuan lengkap tercantum di dokumen **Panduan dan Ketentuan Kelas Industri UBIG**.

**Petunjuk bagi Guru dan Mentor**

1. Gunakan modul ini bersama presentasi P1-W14 dan dokumen Kunci Jawaban P1-W14 yang khusus untuk mentor.
2. Buka pertemuan dengan kuis P1-W13 melalui Google Form selama 10 menit, lalu bandingkan kartu proyek buatan sendiri dengan contoh komponen shadcn/ui.
3. Minta siswa memeriksa `.env.local` sebelum pertemuan. Siswa yang belum menyelesaikan Minggu 13 tidak bisa mengerjakan Kegiatan Pembelajaran 3 sampai 5.
4. Target minimal di kelas: shadcn/ui terpasang, kartu proyek memakai `Card`, dan halaman Proyek membaca data Supabase di laptop. Halaman detail dan *deploy* boleh dilanjutkan di rumah.
5. Siapkan Google Form kuis P1-W14 untuk pembuka pertemuan P1-W15. Kunci, skor, dan daftar periksa remedial ada di Kunci Jawaban P1-W14.
6. Tandai butir kompetensi *Integrasi Next.js dengan Supabase (Koneksi Full Stack Dasar)* setelah P1-W15 selesai, karena butir ini dibuktikan di Minggu 14 dan 15.

> **Catatan**
>
> Teks bertanda `huruf seperti ini` adalah kode, nama berkas, perintah, atau nama tombol yang harus ditulis persis sama, termasuk huruf besar dan kecilnya. Perintah diketik di terminal Git Bash VS Code, kecuali disebutkan lain.

# BAB I
# PENDAHULUAN

## 1.1 Deskripsi Modul

Coba buka beberapa website toko daring atau aplikasi sekolah. Tombol, kartu, dan kotak isian di dalamnya terlihat rapi dan seragam. Developer di perusahaan jarang membuat semua komponen itu dari nol; mereka memakai **pustaka komponen** agar tampilan konsisten dan waktu kerja lebih singkat. Minggu ini kamu memakai pustaka komponen yang sangat populer di dunia Next.js, yaitu **shadcn/ui**.

Masih ada satu pekerjaan dari Minggu 13: data portofolio sudah ada di Supabase, tetapi website belum mengambilnya. Setelah tampilan dirapikan, kamu menghubungkan Next.js ke Supabase sehingga kartu proyek dan halaman detail terisi langsung dari database.

Modul P1-W14 terdiri atas lima Kegiatan Pembelajaran: mengenal dan memasang shadcn/ui; mengatur token tema serta memakai `Card`, `Badge`, dan `Button`; mengambil data proyek dari Supabase; membuat halaman detail dari database lalu menghapus data lama di kode; serta menerbitkan hasilnya ke Vercel dengan *Environment Variables*.

Aturan yang berlaku untuk semua minggu, seperti format laporan, standar video, tenggat, dan sanksi, tidak dibahas di modul ini. Semuanya tercantum di dokumen **Panduan dan Ketentuan Kelas Industri UBIG**.

## 1.2 Prasyarat

Sebelum memulai, pastikan kamu sudah memiliki:

- project `portofolio-next` hasil tugas Minggu 13 yang sudah di-*push* ke GitHub dan tampil di Vercel;
- project Supabase dengan tabel `projects` (minimal 3 baris) dan `skills`, keduanya dengan *policy* baca publik (Minggu 13);
- berkas `.env.local` di folder utama project berisi `NEXT_PUBLIC_SUPABASE_URL` dan `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` (Minggu 13);
- pemahaman `async` dan `await` (Minggu 08), *server component* dan *client component* (Minggu 12), serta mode gelap dengan awalan `dark:` (Minggu 12);

• kemampuan menjalankan `npm run dev`, `git add`, `git commit`, dan `git push` di terminal Git Bash VS Code.

### 1.3 Tujuan Pembelajaran

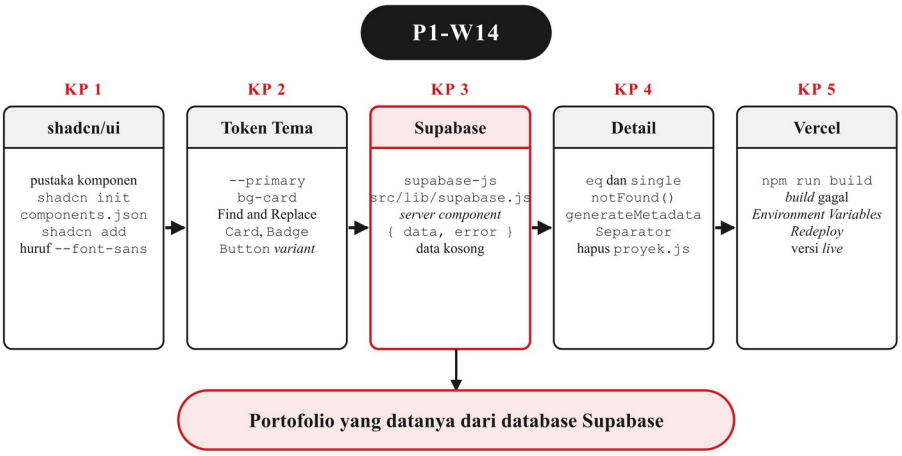
Setelah mempelajari modul ini, kamu diharapkan mampu:

1. menjelaskan cara kerja shadcn/ui dan memasangnya dengan `npx shadcn@latest init` serta `npx shadcn@latest add`;
2. mengatur token tema dan memakai komponen `Button`, `Card`, `Badge`, dan `Separator`;
3. menghubungkan Next.js ke Supabase dengan `@supabase/supabase-js` dan *environment variable*;
4. mengambil data di *server component* dengan `select`, `order`, `limit`, `eq`, dan `single`;
5. menangani *error* dan data kosong, serta menampilkan halaman 404 untuk *slug* yang tidak ada;
6. menerbitkan versi *live* yang membaca Supabase dengan *Environment Variables* Vercel.

Tujuan tersebut menjadi bagian pertama pembuktian butir kompetensi **Integrasi Next.js dengan Supabase (Koneksi *Full Stack* Dasar)** pada Paket 1. Bagian keduanya, menyimpan data dari form, dipelajari di Minggu 15.

### 1.4 Peta Konsep

Urutan materi dalam modul ini digambarkan pada Gambar 1.1. Setiap Kegiatan Pembelajaran menjadi bekal untuk kegiatan berikutnya.




**Gambar 1.1** Peta konsep Modul P1-W14

## 1.5 Alokasi Waktu

Kegiatan minggu ini terdiri atas satu pertemuan daring 90 menit dan tugas mandiri seperti pada Tabel 1.1.

**Tabel 1.1** Alokasi waktu Minggu 14

| Kegiatan | Waktu | Materi |
|---|---|---|
| Pembuka | 15 menit | Kuis P1-W13 (10 menit), lalu membandingkan kartu buatan sendiri dengan komponen shadcn/ui |
| Konsep | 20 menit | Cara kerja shadcn/ui, token tema, dan alur permintaan Next.js ke Supabase |
| Praktik | 45 menit | Memasang shadcn/ui, kartu proyek dengan `Card` dan `Badge`, lalu data proyek dari Supabase (Kegiatan Pembelajaran 1 sampai 3) |
| Penutup | 10 menit | Rangkuman, halaman detail, *deploy*, dan penjelasan tugas `skills` |
| Tugas mandiri | 3 - 4 jam | Kegiatan Pembelajaran 4 dan 5, keahlian dari Supabase, komponen shadcn/ui lainnya, laporan, dan video |

# BAB II
# KEGIATAN PEMBELAJARAN 1: MENGENAL DAN MEMASANG SHADCN/UI

## 2.1 Tujuan Kegiatan Pembelajaran

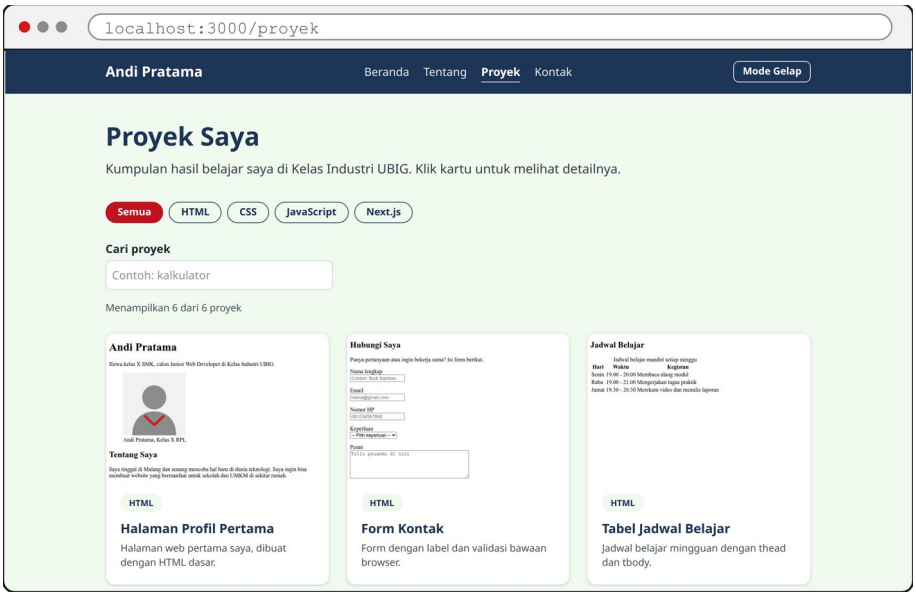
Setelah mengikuti kegiatan ini, kamu mampu menjelaskan perbedaan shadcn/ui dengan pustaka komponen biasa, memasang shadcn/ui di `portofolio-next`, memahami berkas yang dibuatnya, memperbaiki huruf yang berubah setelah pemasangan, dan memakai tombol pertama dari shadcn/ui.

## 2.2 Uraian Materi

### *2.2.1 Mengapa Memakai Pustaka Komponen*

Sejak Minggu 10 kamu membuat semua komponen sendiri: `ProyekCard`, `SkillCard`, tombol kategori, dan tombol tema. Setiap komponen ditulis dengan deretan kelas Tailwind yang panjang, misalnya `rounded-xl border border-gray-200 bg-white shadow-sm`. Cara ini bagus untuk belajar, tetapi melelahkan ketika komponennya makin banyak: kelas mudah berbeda sedikit di setiap tempat, dan mode gelap harus ditulis ulang di setiap kelas dengan awalan `dark:`.

Halaman Proyek hasil Minggu 13 terlihat pada Gambar 2.1. Kartu dan tombolnya sudah rapi, tetapi semuanya kita rakit sendiri.

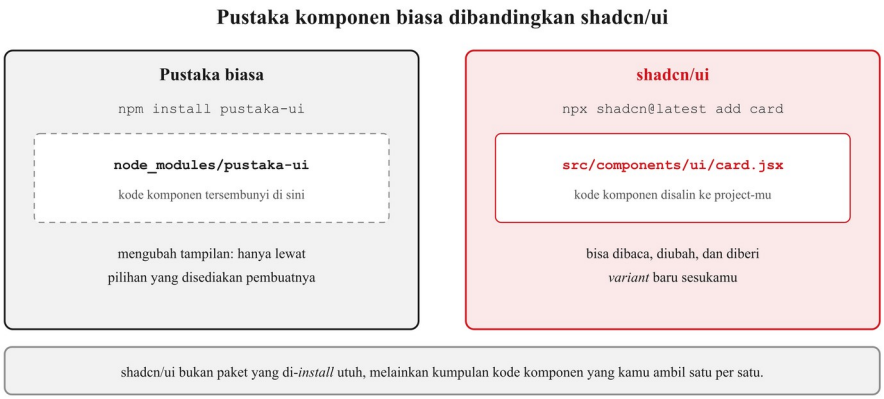



**Gambar 2.1** Halaman Proyek hasil Minggu 13 dengan kartu dan tombol buatan sendiri

**Pustaka komponen** adalah kumpulan komponen siap pakai yang sudah dirancang rapi, mudah diakses dengan *keyboard*, dan seragam. Ibaratnya seperti membeli perabot rakitan: kamu tidak menggergaji kayu sendiri, cukup memasang bagian-bagian yang sudah jadi lalu mengecatnya sesuai selera.

### *2.2.2 Cara Kerja shadcn/ui*

Pustaka komponen biasa di-*install* sebagai paket di folder `node_modules`, sehingga kodenya tersembunyi dan sulit diubah. **shadcn/ui** bekerja dengan cara berbeda: komponen yang kamu minta **disalin** ke dalam project-mu, di folder `src/components/ui`. Kode itu menjadi milikmu, bisa dibaca, diubah, dan diberi variasi baru. Perbedaannya terlihat pada Gambar 2.2.




**Gambar 2.2** Pustaka komponen biasa dibandingkan shadcn/ui

Untuk menyalin komponen, shadcn/ui menyediakan program baris perintah (CLI) yang dijalankan dengan `npx`. Ada dua perintah utama yang kamu pakai minggu ini, seperti pada Tabel 2.1.

**Tabel 2.1** Dua perintah utama shadcn/ui

| No | Perintah | Kegunaan |
|---|---|---|
| 1 | `npx shadcn@latest init` | Menyiapkan project satu kali: membuat `components.json`, `src/lib/utils.js`, komponen `button.jsx`, menambah warna tema di `globals.css`, dan memasang paket pendukung |
| 2 | `npx shadcn@latest add card badge` | Menyalin komponen yang disebut (boleh lebih dari satu) ke `src/components/ui` |

> **Catatan**
>
> Di balik layar, komponen shadcn/ui versi terbaru dibangun di atas **Base UI**, pustaka yang mengurus perilaku komponen (misalnya fokus *keyboard* dan aksesibilitas), sedangkan tampilannya diatur dengan kelas Tailwind. Kamu tidak perlu mempelajari Base UI; cukup memakai komponen jadinya.

## 2.3 Praktik: Memasang shadcn/ui

Praktik berikut memasang shadcn/ui di `portofolio-next`, memeriksa berkas yang dibuatnya, lalu memakai tombol pertama. Ikuti tahapnya secara berurutan.

### *2.3.1 Tahap 1: Menjalankan shadcn init*

Buka folder `portofolio-next` di VS Code, lalu buka terminal Git Bash (**Terminal** lalu **New Terminal**). Pastikan `npm run dev` tidak sedang berjalan; jika masih berjalan, hentikan dengan Ctrl+C. Ketik perintah pada Kode 2.1.

**Kode 2.1** Menyiapkan shadcn/ui di project

```
$ npx shadcn@latest init
```

Karena paket `shadcn` belum ada di laptopmu, npx meminta izin untuk mengunduhnya, seperti pada Gambar 2.3. Tekan Enter untuk menjawab `y` (ya).



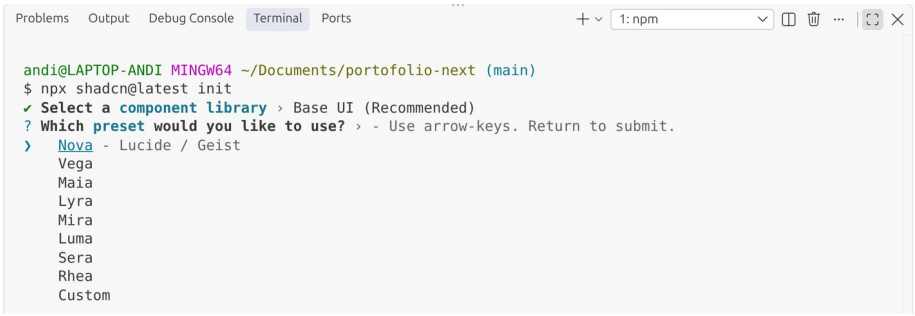

**Gambar 2.3** npx meminta izin mengunduh paket shadcn

Pertanyaan pertama adalah pustaka komponen yang dipakai, seperti pada Gambar 2.4. Biarkan pilihan **Base UI (Recommended)** yang sudah ditandai, lalu tekan Enter.



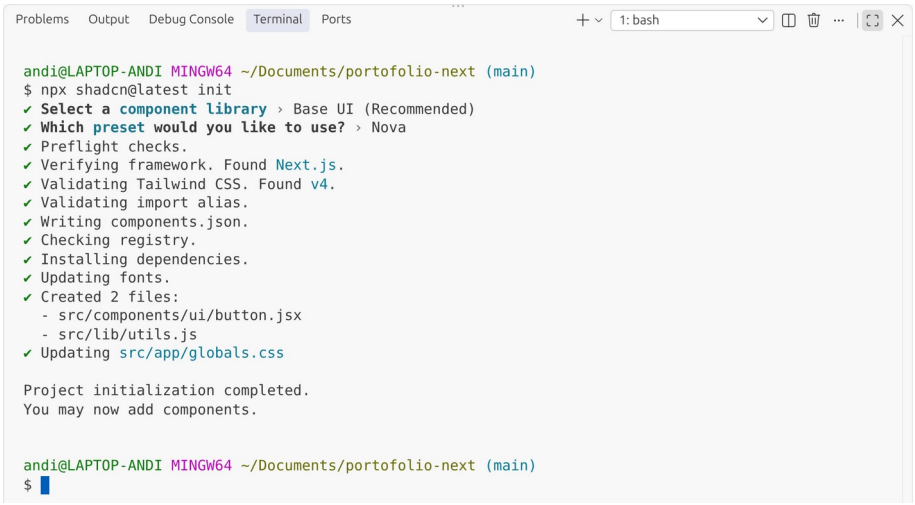

**Gambar 2.4** Pertanyaan pustaka komponen: pilih Base UI

Pertanyaan kedua adalah *preset*, yaitu gaya bawaan komponen, seperti pada Gambar 2.5. Biarkan pilihan pertama **Nova**, lalu tekan Enter.




**Gambar 2.5** Pertanyaan preset: pilih Nova

CLI lalu memeriksa project, menulis berkas, dan memasang paket. Tunggu sampai muncul tulisan **Project initialization completed.**, seperti pada Gambar 2.6.

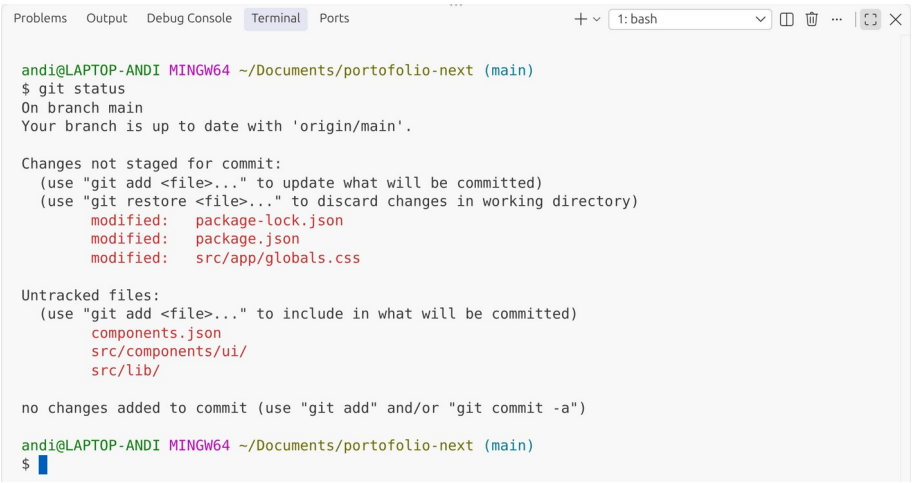



**Gambar 2.6** shadcn init selesai

> **Kesalahan Umum**
>
> Jika PowerShell menolak dengan pesan *npx.ps1 cannot be loaded because running scripts is disabled*, berarti terminalmu bukan Git Bash. Ganti profil terminal ke Git Bash seperti di Minggu 10, lalu jalankan ulang perintahnya. Jika muncul *Verifying framework* gagal, periksa bahwa terminal berada di folder `portofolio-next` (prompt diakhiri `portofolio-next (main)`).

### *2.3.2 Tahap 2: Memeriksa Berkas Baru*

Jalankan `git status` untuk melihat apa saja yang diubah dan dibuat oleh shadcn init. Hasilnya terlihat pada Gambar 2.7: tiga berkas diubah dan tiga hal baru muncul.




**Gambar 2.7** Berkas yang diubah dan dibuat shadcn init

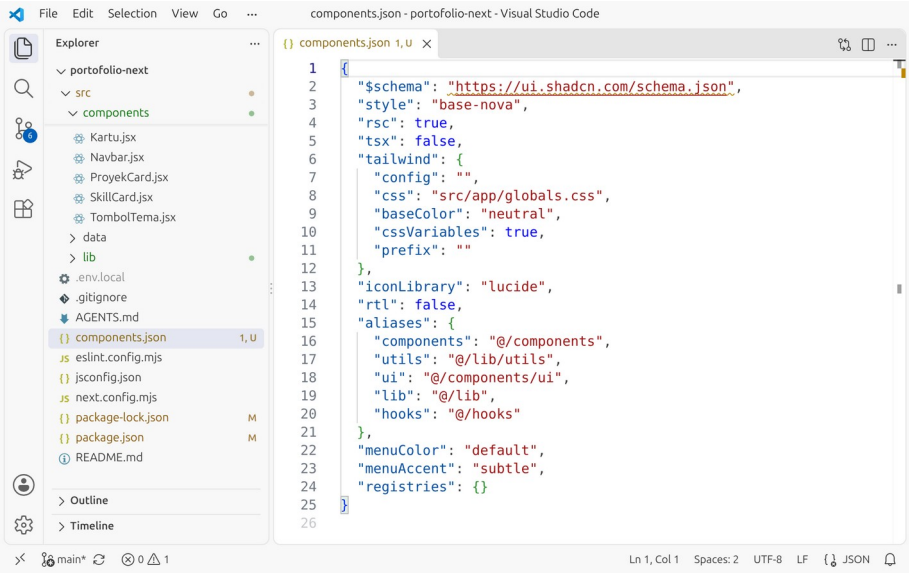
Kegunaan setiap berkas dijelaskan pada Tabel 2.2.

**Tabel 2.2** Berkas yang dibuat atau diubah shadcn init

| No | Berkas | Isi dan Kegunaan |
|---|---|---|
| 1 | `components.json` | Pengaturan shadcn/ui: gaya `base-nova`, JavaScript (`"tsx": false`), letak `globals.css`, dan alias folder komponen |
| 2 | `src/lib/utils.js` | Fungsi `cn` untuk menggabungkan kelas Tailwind dengan rapi |
| 3 | `src/components/ui/button.jsx` | Komponen `Button` pertama |
| 4 | `src/app/globals.css` | Ditambah variabel warna tema untuk mode terang dan gelap |
| 5 | `package.json` dan `package-lock.json` | Ditambah paket pendukung, antara lain `@base-ui/react`, `class-variance-authority`, `cn`, `lucide-react`, dan `shadcn` |

Buka `components.json` di VS Code, seperti pada Gambar 2.8. Bagian `aliases` menjelaskan mengapa komponen nanti diimpor dengan alamat

`@/components/ui/...`: alias `@` menunjuk folder `src`, sama seperti yang kamu pakai sejak Minggu 10.

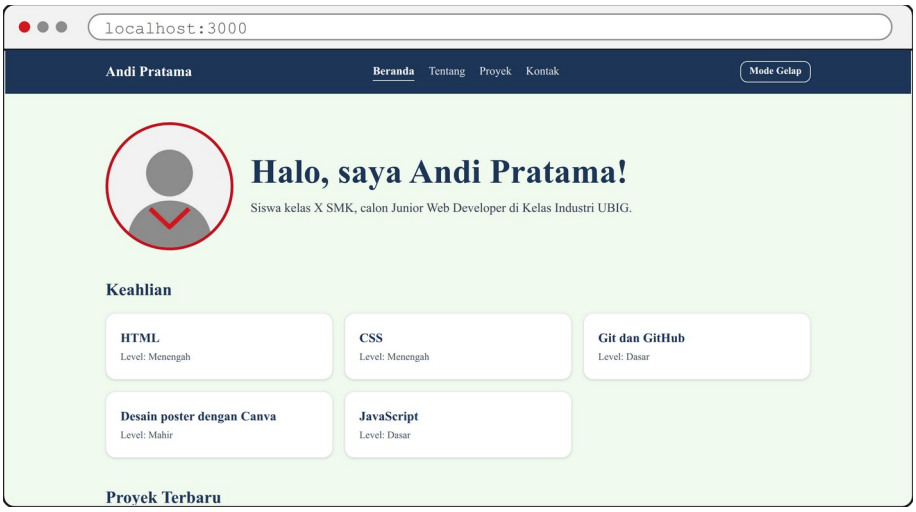



**Gambar 2.8** Isi components.json

> **Perhatian**
>
> Jangan menghapus atau mengubah `components.json`. Berkas ini dibaca setiap kali kamu menjalankan `npx shadcn@latest add`. Tanpa berkas ini, CLI tidak tahu ke folder mana komponen harus disalin.

### *2.3.3 Tahap 3: Memperbaiki Huruf yang Berubah*

Jalankan `npm run dev`, lalu buka `localhost:3000`. Ada kejutan: huruf seluruh halaman berubah menjadi huruf berkait seperti koran, seperti pada Gambar 2.9. Huruf itu adalah Times New Roman, huruf bawaan *browser* ketika huruf yang diminta tidak ditemukan.



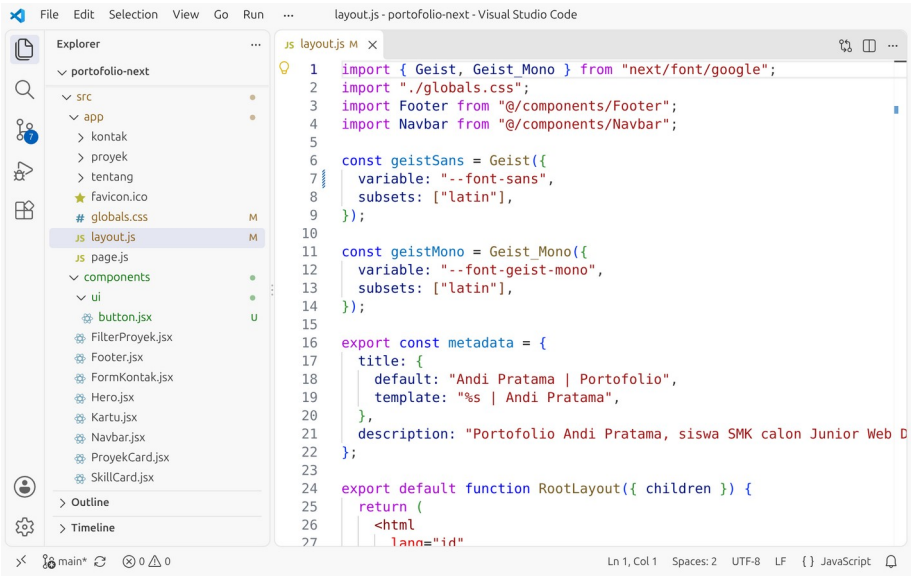

**Gambar 2.9** Setelah shadcn init, huruf halaman berubah menjadi Times New Roman

Penyebabnya, `globals.css` yang baru menyuruh halaman memakai huruf dari variabel `--font-sans`, sedangkan `layout.js` milik kita menamai huruf Geist dengan variabel `--font-geist-sans` (nama bawaan `create-next-app`). Karena variabel `--font-sans` tidak pernah diisi, *browser* memakai huruf cadangan. Perbaikannya cukup mengganti nama variabel di `src/app/layout.js`, seperti pada Kode 2.2.

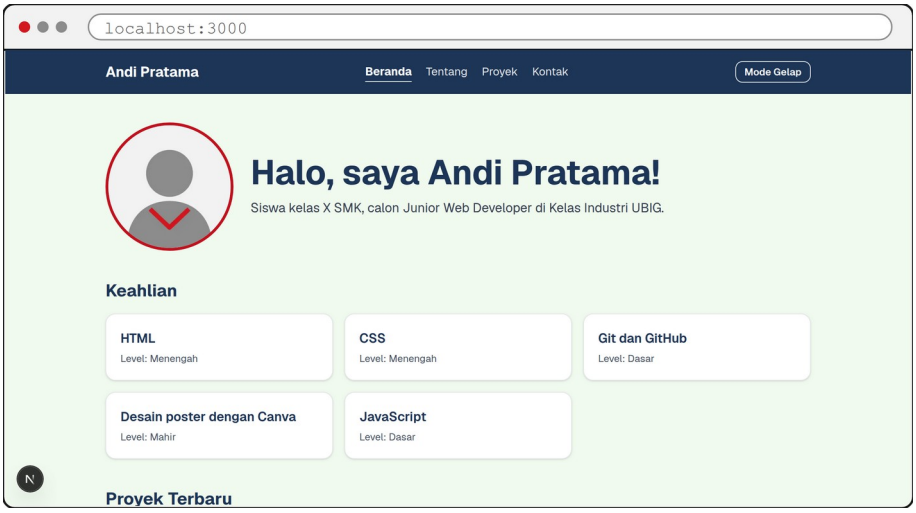
**Kode 2.2** src/app/layout.js: nama variabel huruf Geist diganti --font-sans

```
const geistSans = Geist({
  variable: "--font-sans",
  subsets: ["latin"],
});
```

Simpan dengan Ctrl+S. Kode `layout.js` setelah diubah terlihat pada Gambar 2.10, dan halaman kembali memakai huruf Geist yang rapi seperti pada Gambar 2.11.




**Gambar 2.10** layout.js setelah variabel huruf diganti




**Gambar 2.11** Huruf halaman kembali normal dengan Geist

> **Kesalahan Umum**
>
> Jika huruf masih Times New Roman, periksa ejaan `--font-sans` (dua tanda minus di depan) dan pastikan baris `className` pada tag `html` masih memuat `${geistSans.variable}`. Simpan berkas, lalu muat ulang halaman dengan Ctrl+F5.

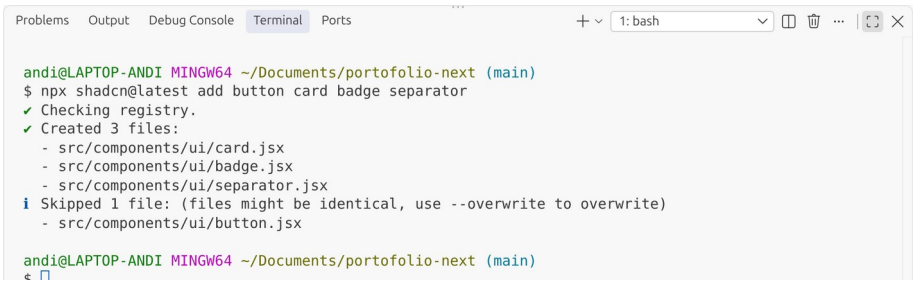
### *2.3.4 Tahap 4: Menambah Komponen Card, Badge, dan Separator*

Komponen lain ditambahkan dengan perintah `add`. Hentikan dulu `npm run dev` dengan Ctrl+C, lalu jalankan perintah pada Kode 2.3.

**Kode 2.3** Menambah empat komponen sekaligus

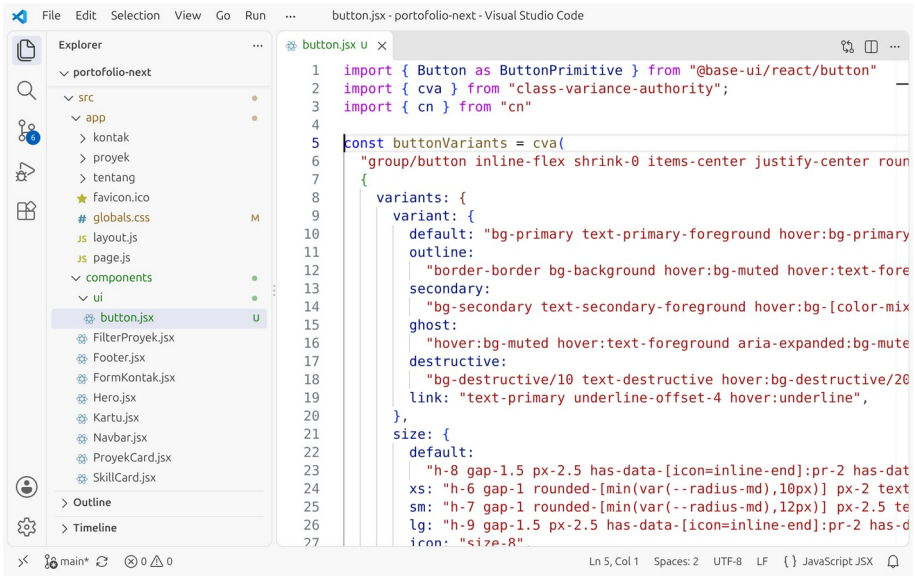
```
$ npx shadcn@latest add button card badge separator
```

Hasilnya terlihat pada Gambar 2.12. Tiga berkas baru dibuat, sedangkan `button.jsx` dilewati (*Skipped*) karena sudah ada dari langkah init.




**Gambar 2.12** Hasil npx shadcn add

Buka berkas `button.jsx` di folder `src/components/ui`. Bagian terpentingnya adalah `buttonVariants`, yang memetakan setiap pilihan gaya ke kelas Tailwind, seperti pada Gambar 2.13. Pilihan gaya ini disebut ***variant***, dan ukurannya disebut `size`.




**Gambar 2.13** Daftar variant dan size di button.jsx

Arti setiap *variant* dirangkum pada Tabel 2.3. Kamu cukup menulis nama *variant*-nya; kelas Tailwind yang panjang sudah diurus `button.jsx`.

**Tabel 2.3** Variant komponen Button

| No | Variant | Tampilan dan Contoh Pemakaian |
|---|---|---|
| 1 | `default` | Latar warna utama (`primary`), untuk tombol utama seperti **Kirim** |
| 2 | `outline` | Bergaris tepi tanpa latar, untuk pilihan yang tidak aktif |
| 3 | `secondary` | Latar abu lembut, untuk tombol pendukung |
| 4 | `ghost` | Tanpa garis dan latar, latar muncul saat kursor di atasnya |
| 5 | `destructive` | Berwarna merah, untuk tindakan berbahaya seperti **Hapus** |
| 6 | `link` | Tampil seperti tautan bergaris bawah |

### *2.3.5 Tahap 5: Tombol Pertama dari shadcn/ui*

Tombol **Lihat semua proyek** di Beranda adalah tautan `Link` ke halaman Proyek. Tautan tidak boleh diubah menjadi `<button>`, karena fungsinya berpindah halaman. shadcn/ui menyediakan fungsi `buttonVariants` yang memberi **gaya** tombol pada elemen lain, termasuk `Link`. Ubah `src/app/page.js`: tambahkan impor `buttonVariants`, lalu ganti `className` tautan, seperti pada Kode 2.4 dan Kode 2.5.

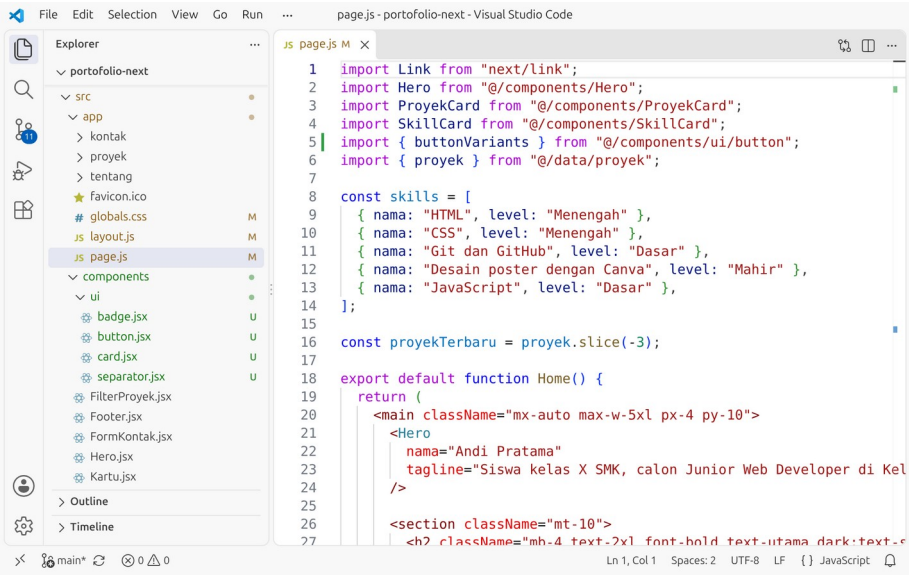
**Kode 2.4** src/app/page.js: impor buttonVariants

```
import Link from "next/link";
import Hero from "@/components/Hero";
import ProyekCard from "@/components/ProyekCard";
import SkillCard from "@/components/SkillCard";
import { buttonVariants } from "@/components/ui/button";
import { proyek } from "@/data/proyek";
```

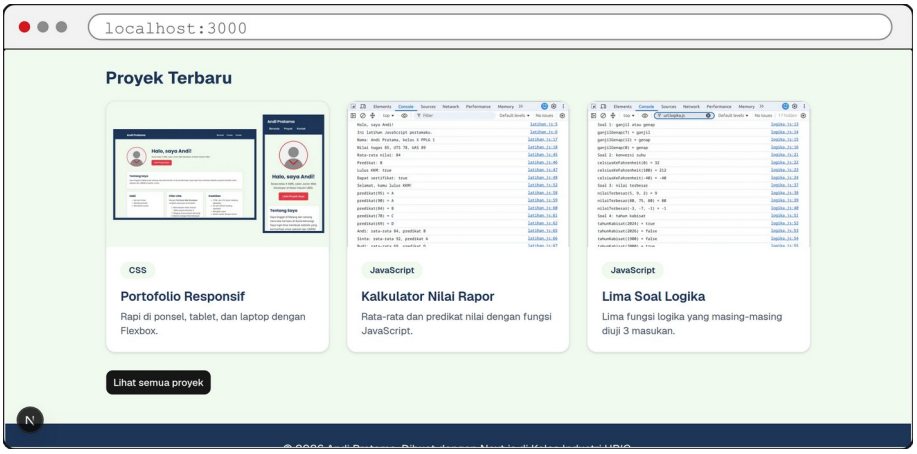
**Kode 2.5** src/app/page.js: tautan bergaya tombol

```
        <Link href="/proyek" className={buttonVariants({ size: "lg",
className: "mt-6" })}>
          Lihat semua proyek
        </Link>
```

`buttonVariants({ size: "lg", className: "mt-6" })` menghasilkan deretan kelas tombol ukuran besar, ditambah kelas `mt-6` milik kita. Hasil di VS Code terlihat pada Gambar 2.14, dan tombolnya di *browser* terlihat pada Gambar 2.15.




**Gambar 2.14** page.js setelah memakai buttonVariants




**Gambar 2.15** Tombol Lihat semua proyek bergaya shadcn/ui

Tombolnya kini berwarna hitam, warna utama bawaan preset Nova. Di Kegiatan Pembelajaran 2 warna utama ini diganti menjadi biru portofolio.

> **Coba Sendiri**
>
> Ganti `size: "lg"` menjadi `size: "sm"`, simpan, lalu lihat ukuran tombol berubah. Kembalikan ke `"lg"` sebelum melanjutkan.

### 2.4 Rangkuman

- Pustaka komponen menyediakan komponen siap pakai yang rapi dan seragam.
- shadcn/ui menyalin kode komponen ke `src/components/ui`, sehingga kodenya milikmu dan bisa diubah.
- `npx shadcn@latest init` menyiapkan project satu kali; `npx shadcn@latest add` menambah komponen.
- Setelah init, nama variabel huruf di `layout.js` harus diganti menjadi `--font-sans`.
- `Button` punya `variant` dan `size`; `buttonVariants` memberi gaya tombol pada `Link`.

### 2.5 Latihan

1. Apa perbedaan utama shadcn/ui dengan pustaka komponen yang di-*install* di `node_modules`?
2. Sebutkan tiga berkas yang dibuat atau diubah oleh `npx shadcn@latest init` beserta kegunaannya.
3. Mengapa huruf halaman berubah menjadi Times New Roman setelah init, dan bagaimana memperbaikinya?
4. Mengapa tombol **Lihat semua proyek** memakai `buttonVariants`, bukan komponen `Button`?
5. Tuliskan perintah untuk menambah komponen `card` dan `badge` sekaligus.

# BAB III
# KEGIATAN PEMBELAJARAN 2: TOKEN TEMA DAN KOMPONEN CARD

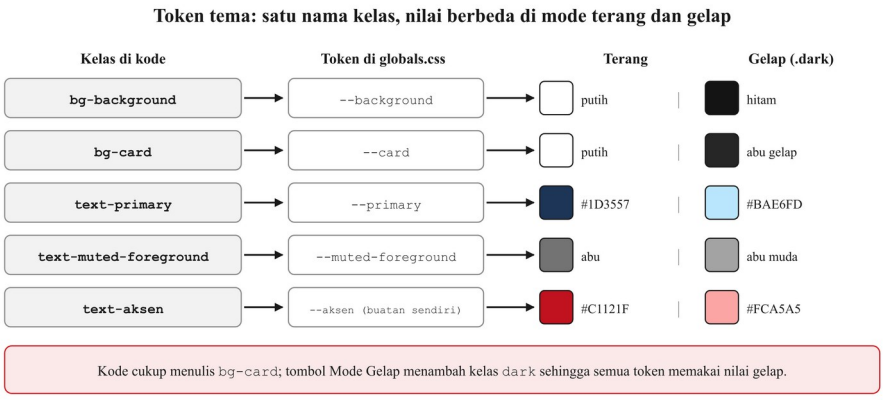
## 3.1 Tujuan Kegiatan Pembelajaran

Setelah mengikuti kegiatan ini, kamu mampu menjelaskan token tema, mengganti warna utama portofolio di `globals.css`, mengganti pasangan kelas `dark:` dengan token, serta menyusun `ProyekCard` dari `Card` dan `Badge` dan tombol kategori dari `Button`.

## 3.2 Uraian Materi

### *3.2.1 Token Tema*

Di Minggu 12, setiap warna ditulis dua kali: satu untuk mode terang dan satu untuk mode gelap, misalnya `bg-white dark:bg-gray-800`. shadcn/ui memakai cara yang lebih ringkas, yaitu **token tema**. Token adalah nama warna yang nilainya disimpan di `globals.css`. Kode cukup menulis satu kelas, misalnya `bg-card`, lalu nilainya otomatis berganti ketika elemen `html` diberi kelas `dark` oleh tombol Mode Gelap, seperti pada Gambar 3.1.




**Gambar 3.1** Token tema: satu nama kelas, nilai berbeda di mode terang dan gelap

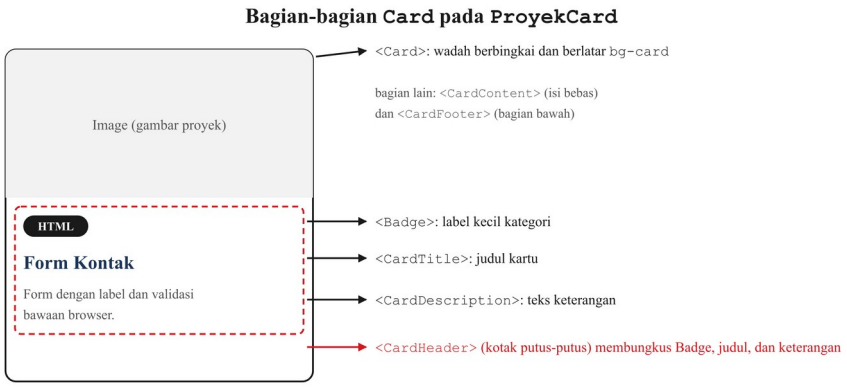
Ibaratnya seperti seragam sekolah: jadwal cukup menulis "pakai seragam", sedangkan seragam hari Senin dan Jumat bisa berbeda tanpa mengubah tulisan jadwal. Token yang paling sering dipakai di portofolio dirangkum pada Tabel 3.1.

**Tabel 3.1** Token tema yang dipakai di portofolio

| No | Kelas Tailwind | Dipakai untuk |
|---|---|---|
| 1 | `bg-background`, `text-foreground` | Latar dan warna teks utama halaman |
| 2 | `bg-card` | Latar kartu dan kotak |
| 3 | `text-primary`, `bg-primary`, `border-primary` | Warna utama: judul, tombol aktif, garis penekanan |
| 4 | `text-muted-foreground` | Teks keterangan yang lebih pudar |
| 5 | `border-border`, `border-input` | Garis tepi kartu dan kotak isian |
| 6 | `text-aksen` | Warna aksen merah portofolio (token buatan sendiri) |

### *3.2.2 Komponen Card dan Badge*

`Card` adalah wadah berbingkai untuk satu kesatuan informasi, sedangkan `Badge` adalah label kecil seperti stiker kategori. `Card` dibagi menjadi beberapa bagian dengan nama yang menjelaskan isinya, seperti pada Gambar 3.2. Pola ini disebut ***composition***: komponen besar disusun dari komponen-komponen kecil.




**Gambar 3.2** Bagian-bagian Card pada ProyekCard

> **Catatan**
> Komponen shadcn/ui diimpor dengan kurung kurawal karena satu berkas berisi beberapa komponen, misalnya `import { Card, CardHeader, CardTitle } from "@/components/ui/card";`. Ini berbeda dengan komponen buatanmu sendiri yang memakai `export default`, sehingga diimpor tanpa kurung kurawal.

## 3.3 Praktik: Tema dan Komponen

Praktik berikut mengganti warna utama, merapikan kelas warna di seluruh folder `src`, lalu menyusun ulang kartu proyek dan tombol kategori. Setelah tahap yang menambah kelas baru, hentikan `npm run dev` (Ctrl+C) lalu jalankan lagi agar Tailwind membaca semua kelas.

### *3.3.1 Tahap 1: Mengatur Warna Utama di globals.css*

Buka `src/app/globals.css`. Gulir ke bagian `:root`, yaitu daftar nilai token untuk mode terang. Ganti nilai `--primary` dengan biru portofolio `#1D3557`, lalu tambahkan token baru `--aksen` tepat di bawah `--primary-foreground`, seperti pada Kode 3.1.

**Kode 3.1** globals.css: token mode terang

```
:root {
  --background: oklch(1 0 0);
  --foreground: oklch(0.145 0 0);
  --card: oklch(1 0 0);
  --card-foreground: oklch(0.145 0 0);
  --popover: oklch(1 0 0);
  --popover-foreground: oklch(0.145 0 0);
  --primary: #1D3557;
  --primary-foreground: oklch(0.985 0 0);
  --aksen: #C1121F;
  /* ... baris lain tetap ... */
}
```

Lakukan hal yang sama di bagian `.dark`, yaitu nilai token saat mode gelap. Warna utama dibuat biru muda agar terbaca di latar gelap, dan teks di atas tombol utama dibuat biru tua, seperti pada Kode 3.2.

**Kode 3.2** globals.css: token mode gelap

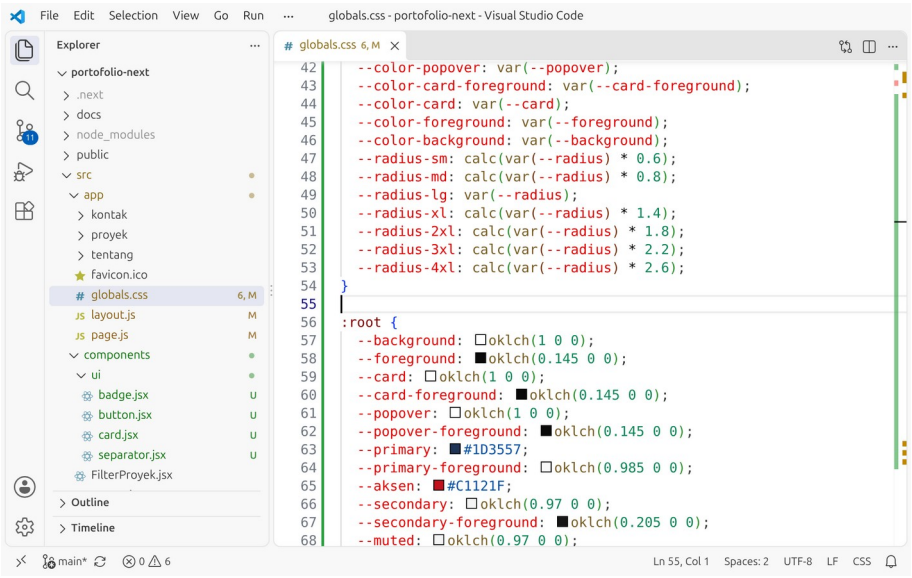
```
.dark {
  --background: oklch(0.145 0 0);
  --foreground: oklch(0.985 0 0);
  --card: oklch(0.205 0 0);
  --card-foreground: oklch(0.985 0 0);
  --popover: oklch(0.205 0 0);
  --popover-foreground: oklch(0.985 0 0);
  --primary: #BAE6FD;
  --primary-foreground: #1D3557;
  --aksen: #FCA5A5;
  /* ... baris lain tetap ... */
}
```

Terakhir, daftarkan token `--aksen` agar bisa dipakai sebagai kelas `text-aksen`. Hapus baris `--color-aksen: #C1121F;` dari blok `@theme`, lalu tambahkan `--color-aksen: var(--aksen);` di baris pertama blok `@theme inline`, seperti pada Kode 3.3. Hasil di VS Code terlihat pada Gambar 3.3.

**Kode 3.3** globals.css: token aksen didaftarkan di @theme inline

```
@theme {
  --color-utama: #1D3557;
  --color-latar: #F1FAEE;
}

@theme inline {
  --color-aksen: var(--aksen);
  /* ... baris lain tetap ... */
}
```

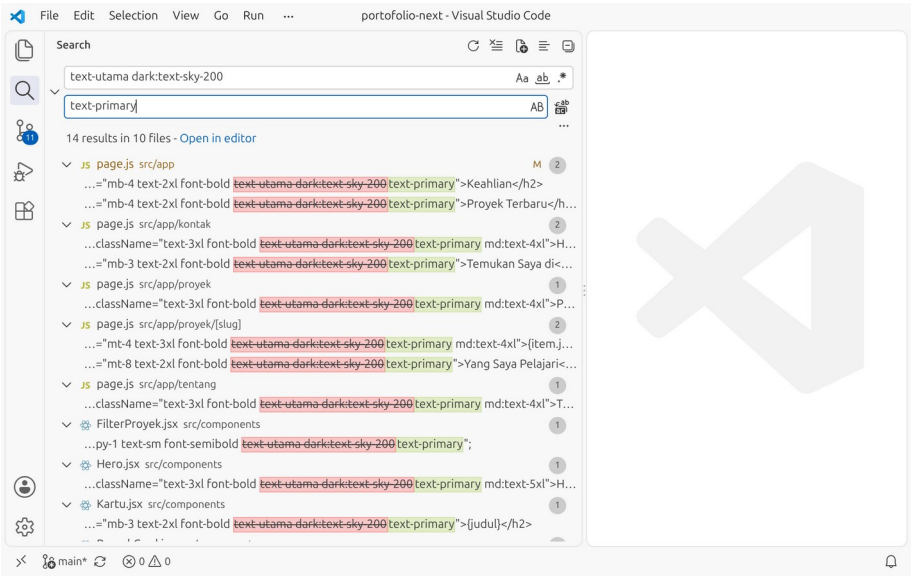



**Gambar 3.3** Token --primary dan --aksen di globals.css

> **Catatan**
>
> Nilai bawaan shadcn/ui ditulis dengan format `oklch(...)`, cara lain menulis warna. Kamu boleh menulis warna sendiri dengan format heksadesimal seperti `#1D3557` yang sudah kamu kenal sejak Minggu 03.

### *3.3.2 Tahap 2: Mengganti Kelas dark: dengan Token*

Sekarang semua pasangan kelas `dark:` dari Minggu 12 diganti token. Kamu memakai **Find and Replace** di seluruh folder sekaligus, kebalikan dari yang kamu lakukan di Minggu 12. Tekan Ctrl+Shift+H, ketik teks di kotak **Search**, ketik penggantinya di kotak **Replace**, lalu klik ikon **Replace All** (atau Ctrl+Alt+Enter). Contoh satu pasangan terlihat pada Gambar 3.4.



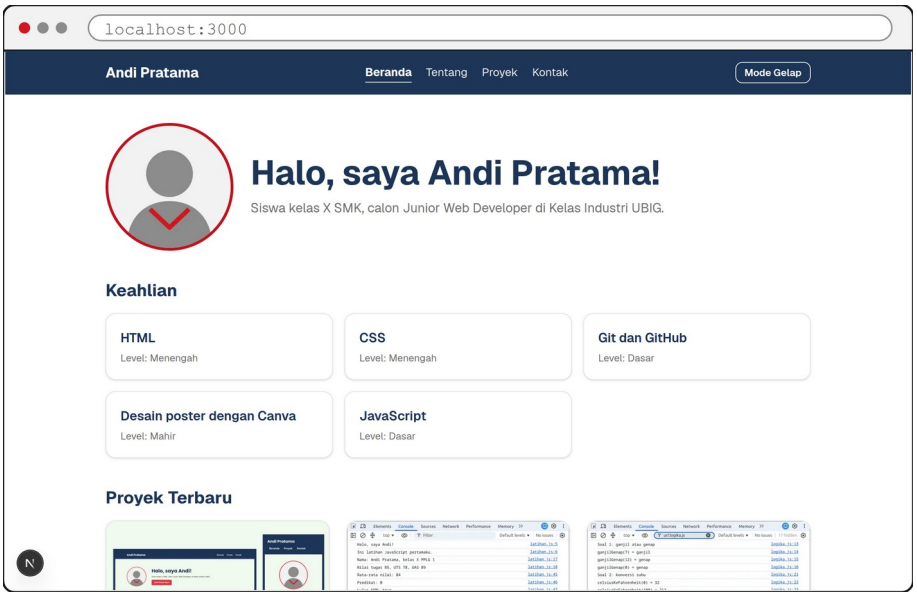

**Gambar 3.4** Find and Replace: text-utama dark:text-sky-200 menjadi text-primary

Ulangi untuk sepuluh pasangan pada Tabel 3.2, dari atas ke bawah. Ketik persis sama, termasuk spasi di antara dua kelas.

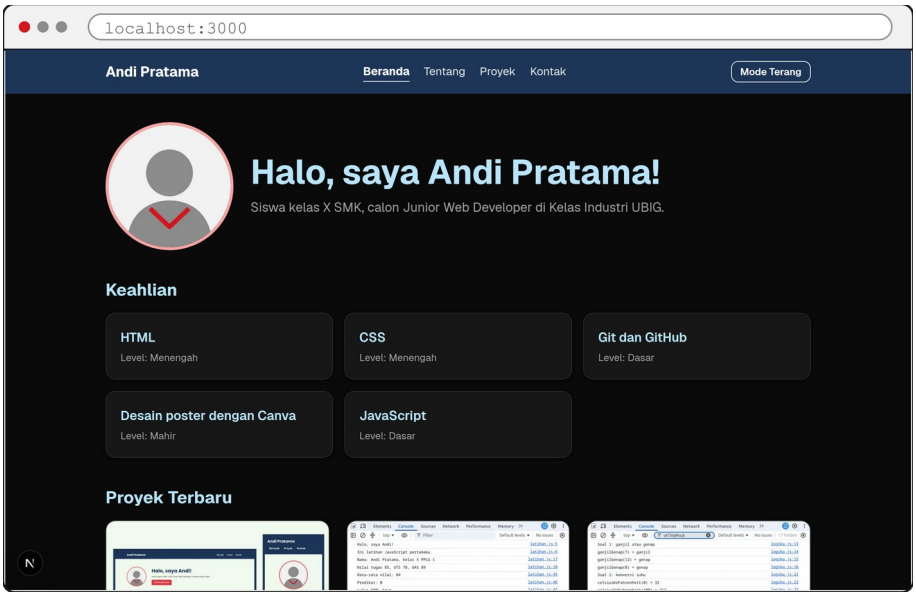
**Tabel 3.2** Sepuluh pasangan Find and Replace

| No | Search (cari) | Replace (ganti) |
|---|---|---|
| 1 | `bg-latar dark:bg-gray-900` | `bg-background` |
| 2 | `text-gray-800 dark:text-gray-100` | `text-foreground` |
| 3 | `bg-white dark:bg-gray-800` | `bg-card` |
| 4 | `text-utama dark:text-sky-200` | `text-primary` |
| 5 | `border-utama dark:border-sky-200` | `border-primary` |
| 6 | `text-gray-700 dark:text-gray-300` | `text-muted-foreground` |
| 7 | `text-gray-600 dark:text-gray-400` | `text-muted-foreground` |
| 8 | `border-gray-200 dark:border-gray-700` | `border-border` |
| 9 | `border-gray-300 dark:border-gray-600` | `border-input` |
| 10 | `text-aksen dark:text-red-300` | `text-aksen` |

Hentikan `npm run dev`, jalankan lagi, lalu buka Beranda. Tampilannya kini memakai token, seperti pada Gambar 3.5. Klik **Mode Gelap**: semua warna ikut berganti tanpa satu pun kelas `dark:` di kode kita, seperti pada Gambar 3.6.




**Gambar 3.5** Beranda dengan token tema di mode terang




**Gambar 3.6** Beranda dengan token tema di mode gelap

> **Kesalahan Umum**
>
> Jika setelah Replace All masih ada teks seperti `bg-white dark:bg-gray-800`, kemungkinan ada spasi ganda atau kelas lain di antara keduanya. Cari kata `dark:` dengan Ctrl+Shift+F; yang boleh tersisa hanya pesan berhasil berwarna hijau di `FormKontak.jsx`.

### *3.3.3 Tahap 3: ProyekCard dengan Card dan Badge*

Buka `src/components/ProyekCard.jsx`, lalu ganti seluruh isinya dengan Kode 3.4. Gambar tetap memakai `Image` dari Next.js, tetapi wadahnya kini `Card`, label kategorinya `Badge`, dan teksnya `CardTitle` serta `CardDescription`.

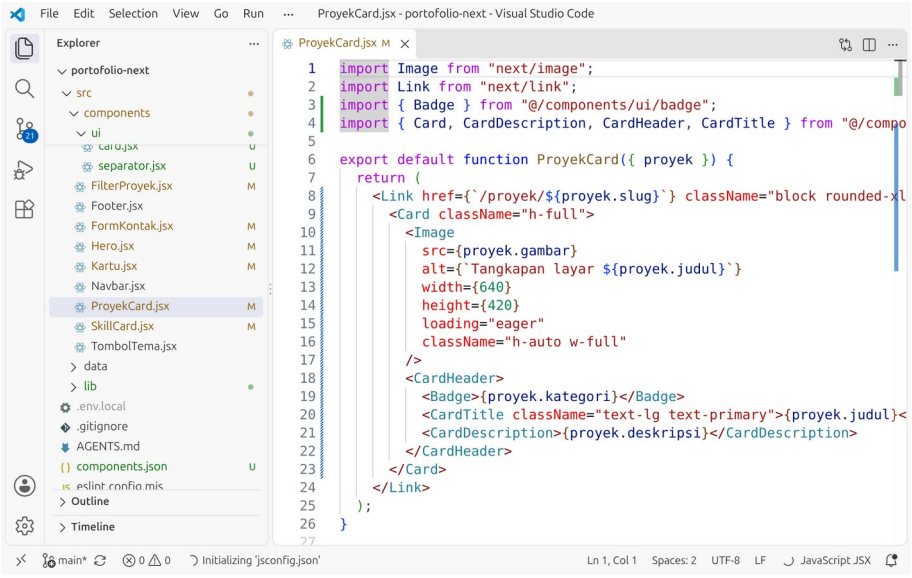
**Kode 3.4** src/components/ProyekCard.jsx memakai Card dan Badge

```
import Image from "next/image";
import Link from "next/link";
import { Badge } from "@/components/ui/badge";
import { Card, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card";

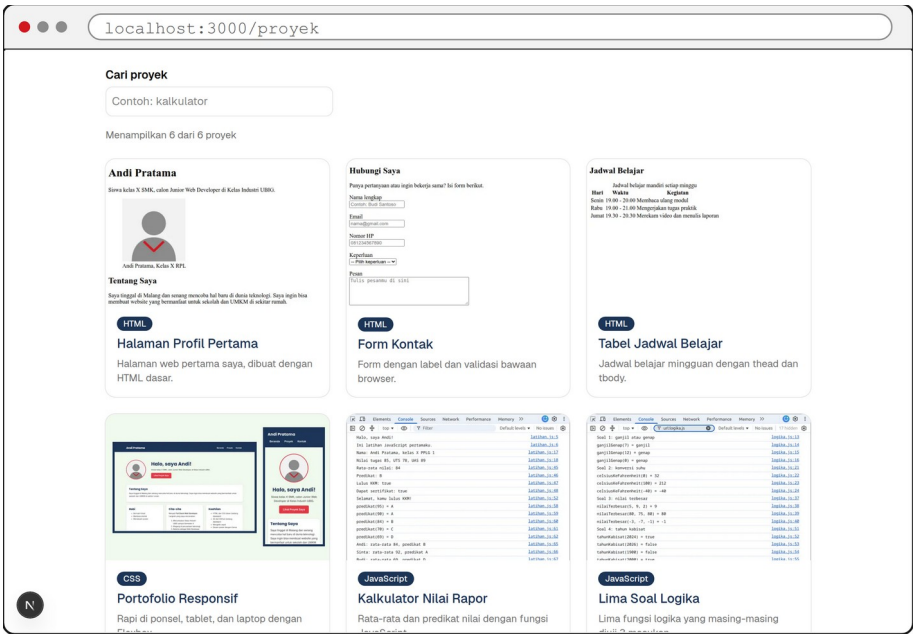
export default function ProyekCard({ proyek }) {
  return (
    <Link href={`/proyek/${proyek.slug}`} className="block rounded-
xl hover:shadow-md">
      <Card className="h-full">
        <Image
          src={proyek.gambar}
          alt={`Tangkapan layar ${proyek.judul}`}
          width={640}
```

```
            height={420}
            loading="eager"
            className="h-auto w-full"
          />
          <CardHeader>
            <Badge>{proyek.kategori}</Badge>
            <CardTitle className="text-lg text-primary">{proyek.judul}
</CardTitle>
            <CardDescription>{proyek.deskripsi}</CardDescription>
          </CardHeader>
        </Card>
      </Link>
    );
}
```

Seluruh kartu tetap dibungkus `Link` agar bisa diklik menuju halaman detail. Kelas `h-full` membuat semua kartu dalam satu baris sama tinggi. Hasil di VS Code terlihat pada Gambar 3.7, dan halaman Proyek terlihat pada Gambar 3.8.




**Gambar 3.7** ProyekCard.jsx dengan Card dan Badge




**Gambar 3.8** Kartu proyek baru dengan Card dan Badge

### *3.3.4 Tahap 4: Tombol Kategori dengan Button*

Buka `src/components/FilterProyek.jsx`. Tambahkan impor `Button` di bawah impor `ProyekCard`, lalu hapus dua konstanta `tombolBiasa` dan `tombolAktif`, karena perannya diambil alih *variant*. Ganti daftar tombol kategori dengan Kode 3.5.

**Kode 3.5** FilterProyek.jsx: tombol kategori memakai Button

```
import { Button } from "@/components/ui/button";

...

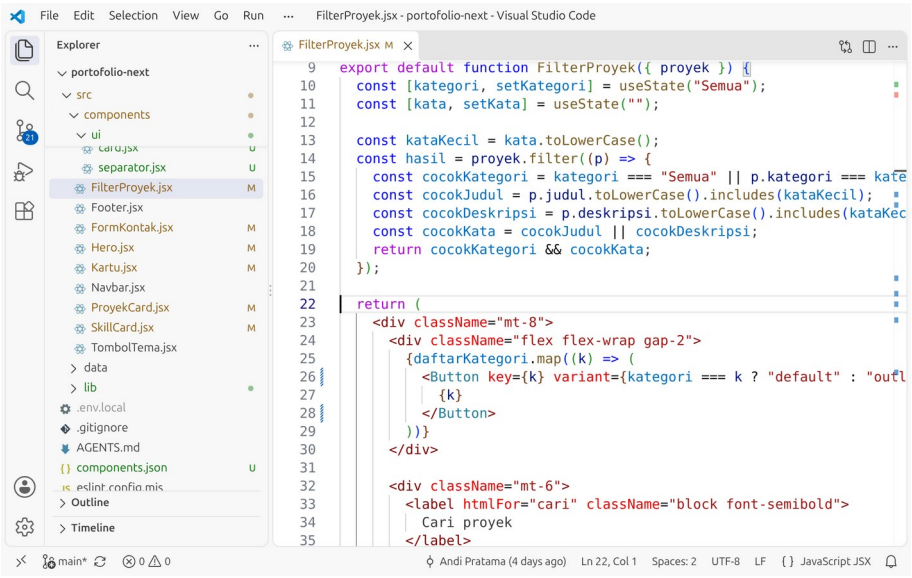
      <div className="flex flex-wrap gap-2">
        {daftarKategori.map((k) => (
          <Button key={k} variant={kategori === k ? "default" :
"outline"} onClick={() => setKategori(k)}>
            {k}
          </Button>
        ))}
      </div>
```

`variant={kategori === k ? "default" : "outline"}` memakai *ternary* dari Minggu 12: tombol yang sedang dipilih tampil penuh berwarna utama, sedangkan tombol lain hanya bergaris tepi. Tombol **Hapus pencarian** juga diganti, seperti pada Kode 3.6.

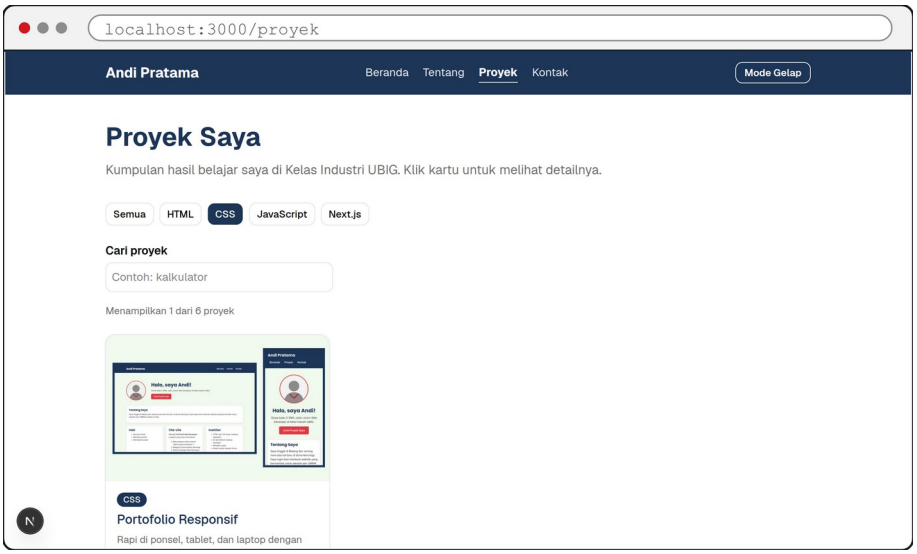
**Kode 3.6** FilterProyek.jsx: tombol Hapus pencarian memakai variant link

```
          <Button variant="link" onClick={() => setKata("")}
className="mt-2 px-0 text-aksen">
            Hapus pencarian
          </Button>
```

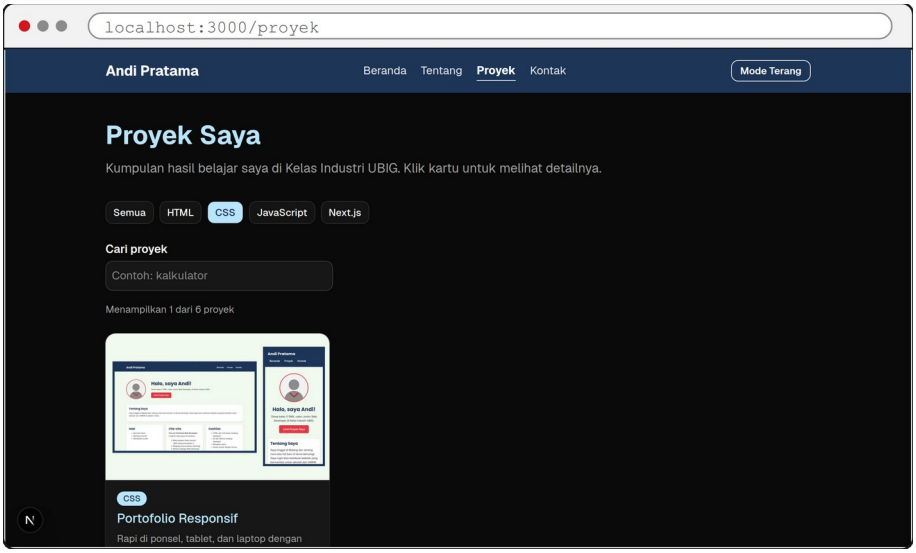
Hasil di VS Code terlihat pada Gambar 3.9. Pilih kategori **CSS** di *browser*: tombol aktifnya berwarna biru portofolio, seperti pada Gambar 3.10, dan tetap terbaca di mode gelap, seperti pada Gambar 3.11.




**Gambar 3.9** FilterProyek.jsx dengan Button




**Gambar 3.10** Tombol kategori CSS aktif di mode terang




**Gambar 3.11** Tombol kategori CSS aktif di mode gelap

> **Coba Sendiri**
>
> Ganti `"outline"` menjadi `"ghost"` atau `"secondary"`, simpan, lalu bandingkan tampilannya. Pilih yang menurutmu paling jelas, tetapi kembalikan ke `"outline"` agar sama dengan modul.

## 3.4 Rangkuman

- Token tema: nama warna di `globals.css` yang berganti otomatis di mode gelap.
- `--primary` diatur di `:root` dan `.dark`; token baru didaftarkan di `@theme inline`.
- Find and Replace mengganti pasangan `dark:` dengan token di seluruh folder.
- `Card` tersusun dari `CardHeader`, `CardTitle`, `CardDescription`; `Badge` label kecil.
- `Button` dengan `variant` menggantikan konstanta kelas tombol buatan sendiri.

## 3.5 Latihan

1. Apa keuntungan `bg-card` dibandingkan `bg-white dark:bg-gray-800`?
2. Di bagian mana `globals.css` nilai warna mode gelap ditulis?
3. Mengapa token `--aksen` harus didaftarkan di `@theme inline`?
4. Sebutkan tiga bagian `Card` yang dipakai `ProyekCard` beserta isinya.
5. Tuliskan kode tombol `Button` yang memakai *variant* `outline`.

# BAB IV
# KEGIATAN PEMBELAJARAN 3: MENGAMBIL DATA PROYEK DARI SUPABASE

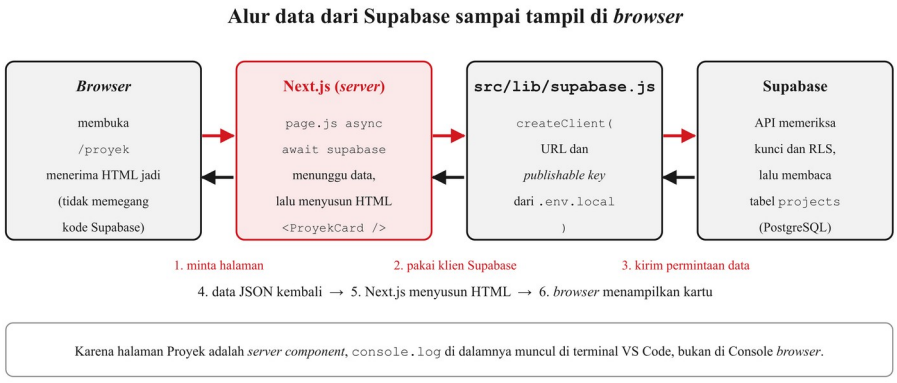
## 4.1 Tujuan Kegiatan Pembelajaran

Setelah mengikuti kegiatan ini, kamu mampu memasang `@supabase/supabase-js`, membuat klien Supabase di `src/lib/supabase.js`, mengambil data di *server component* dengan `await`, menangani *error* dan data kosong, serta menampilkan proyek terbaru di Beranda.

## 4.2 Uraian Materi

### *4.2.1 Alur Data dari Supabase ke Browser*

Di Minggu 13 kamu sudah membaca data tabel `projects` lewat alamat API di *browser*. Next.js membaca data yang sama, tetapi lewat pustaka resmi **`@supabase/supabase-js`** sehingga kita tidak perlu menyusun alamat API sendiri. Karena halaman Proyek adalah *server component*, permintaan data dikirim dari *server* Next.js, bukan dari *browser* pengunjung, seperti pada Gambar 4.1.




**Gambar 4.1** Alur data dari Supabase sampai tampil di browser

Bayangkan kamu memesan makanan di restoran (analogi Minggu 13). Pelayan (Next.js) membawa pesananmu ke dapur (Supabase), menunggu masakannya jadi, lalu

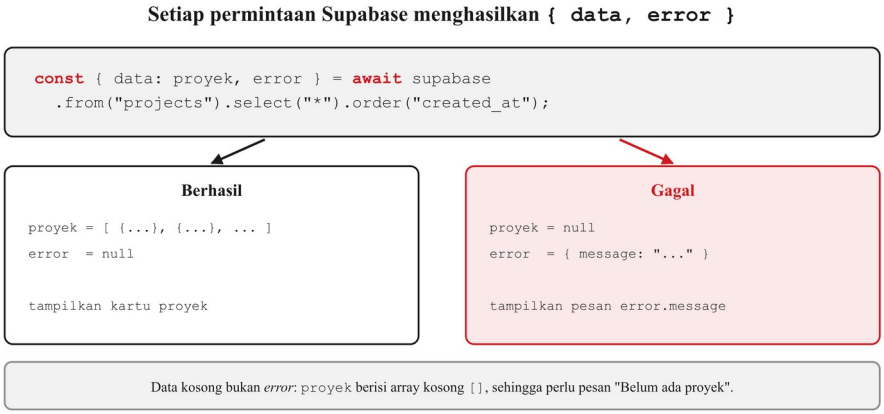
menyajikan piring yang sudah tertata (HTML). Kamu tidak pernah masuk dapur, sama seperti *browser* yang tidak pernah memegang kode Supabase.

### *4.2.2 Klien Supabase dan Environment Variable*

Untuk berbicara dengan Supabase, Next.js membutuhkan **klien**, yaitu objek yang tahu alamat project dan kuncinya. Klien dibuat sekali dengan fungsi `createClient` di berkas `src/lib/supabase.js`, lalu dipakai ulang oleh semua halaman. Alamat dan kunci dibaca dari *environment variable* yang sudah kamu simpan di `.env.local` pada Minggu 13, melalui `process.env.NAMA_VARIABEL`.

### *4.2.3 Mengambil Data dengan await dan { data, error }*

Mengambil data membutuhkan waktu, seperti `fetch` di Minggu 08. Karena itu fungsi halaman diberi kata `async` dan permintaannya diberi `await`. Hasilnya selalu berupa object berisi dua bagian: `data` (isi tabel) dan `error` (keterangan jika gagal), seperti pada Gambar 4.2.




**Gambar 4.2** Setiap permintaan Supabase menghasilkan { data, error }

Permintaan disusun dengan menyambung beberapa *method* secara berurutan, seperti kalimat SQL di Minggu 13. *Method* yang dipakai minggu ini dirangkum pada Tabel 4.1.

**Tabel 4.1** Method supabase-js yang dipakai minggu ini

| No | Method | Arti | Padanan SQL |
|---|---|---|---|
| 1 | `.from("projects")` | Dari tabel `projects` | `from projects` |
| 2 | `.select("*")` | Ambil semua kolom | `select *` |
| 3 | `.order("created_at")` | Urutkan dari yang terlama | `order by created_at` |
| 4 | `.order("created_at", { ascending: false })` | Urutkan dari yang terbaru | `order by created_at desc` |
| 5 | `.limit(3)` | Ambil paling banyak 3 baris | `limit 3` |
| 6 | `.eq("slug", slug)` | Hanya baris yang `slug`-nya sama | `where slug = '...'` |
| 7 | `.single()` | Ambil tepat satu baris sebagai object | (tidak ada) |

## 4.3 Praktik: Data Proyek dari Supabase

Praktik berikut memasang pustaka Supabase, membuat klien, lalu mengubah halaman Proyek dan Beranda agar membaca database. Pastikan `.env.local` dari Minggu 13 masih ada di folder utama project.

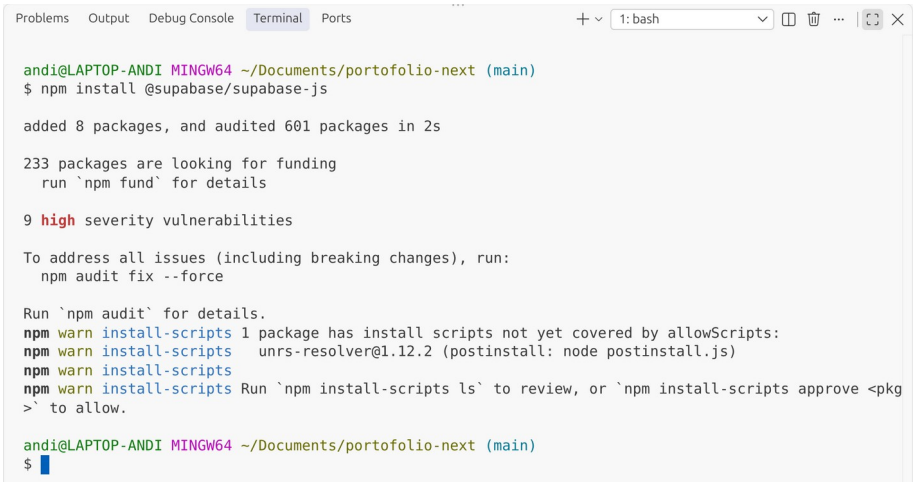
### *4.3.1 Tahap 1: Memasang supabase-js*

Hentikan `npm run dev` (Ctrl+C), lalu jalankan perintah pada Kode 4.1.

**Kode 4.1** Memasang pustaka Supabase untuk JavaScript

```
$ npm install @supabase/supabase-js
```

Hasilnya terlihat pada Gambar 4.3: npm menambahkan beberapa paket dan mencatat `@supabase/supabase-js` versi `^2.118.0` di `package.json`.




**Gambar 4.3** Hasil npm install @supabase/supabase-js

> **Perhatian**
>
> Tulisan *vulnerabilities* dan *npm warn install-scripts* adalah pemberitahuan rutin npm tentang paket-paket di project, bukan tanda pemasangan gagal. **Jangan** menjalankan `npm audit fix --force`, karena perintah itu bisa mengganti versi Next.js dan merusak project. Pemasangan berhasil jika ada tulisan *added ... packages*.

### *4.3.2 Tahap 2: Membuat src/lib/supabase.js*

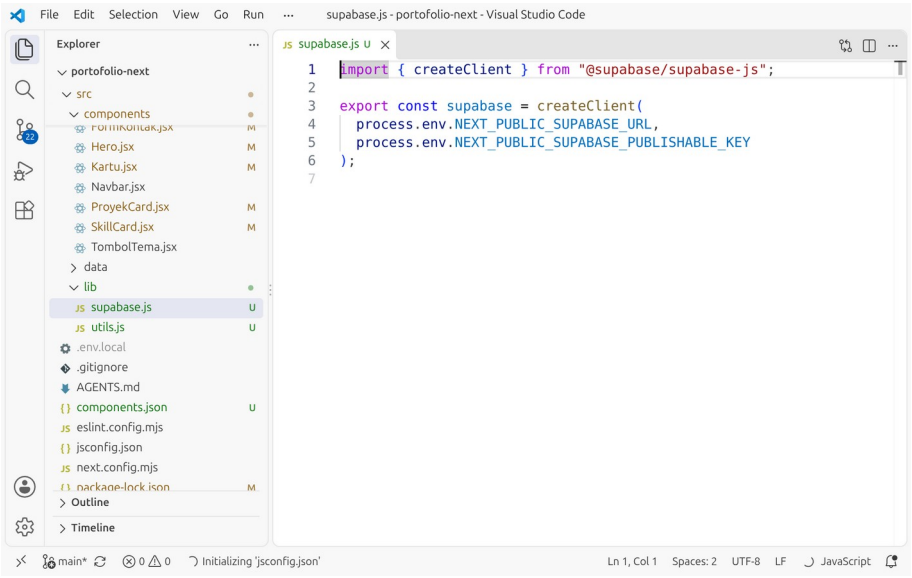
Di Explorer VS Code, klik kanan folder `src/lib` (dibuat oleh shadcn init), pilih **New File**, lalu beri nama `supabase.js`. Isi dengan Kode 4.2.

**Kode 4.2** src/lib/supabase.js

```
import { createClient } from "@supabase/supabase-js";

export const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY
);
```

Baris pertama mengimpor `createClient`. `export const supabase` membuat klien dengan dua nilai dari `.env.local`, lalu mengekspornya agar bisa diimpor halaman lain dengan `import { supabase } from "@/lib/supabase";`. Hasilnya terlihat pada Gambar 4.4.




**Gambar 4.4** Berkas src/lib/supabase.js

> **Perhatian**
>
> Jangan pernah menempel nilai kunci langsung di `supabase.js`, misalnya `createClient("https://...", "sb_publishable_...")`. Berkas ini ikut ter-*commit* ke GitHub. Nilai kunci cukup berada di `.env.local` yang diabaikan Git.

### *4.3.3 Tahap 3: Halaman Proyek Mengambil Data*

Buka `src/app/proyek/page.js`. Ganti impor data lama `import { proyek } from "@/data/proyek";` dengan impor klien Supabase, ubah fungsi menjadi `async`, lalu ambil data di awal fungsi. Sementara, tambahkan `console.log` untuk memeriksa jumlah data yang datang, seperti pada Kode 4.3.

**Kode 4.3** src/app/proyek/page.js membaca tabel projects

```
import FilterProyek from "@/components/FilterProyek";
import { supabase } from "@/lib/supabase";

export const metadata = {
  title: "Proyek",
  description: "Daftar proyek HTML, CSS, dan JavaScript karya Andi
Pratama.",
};

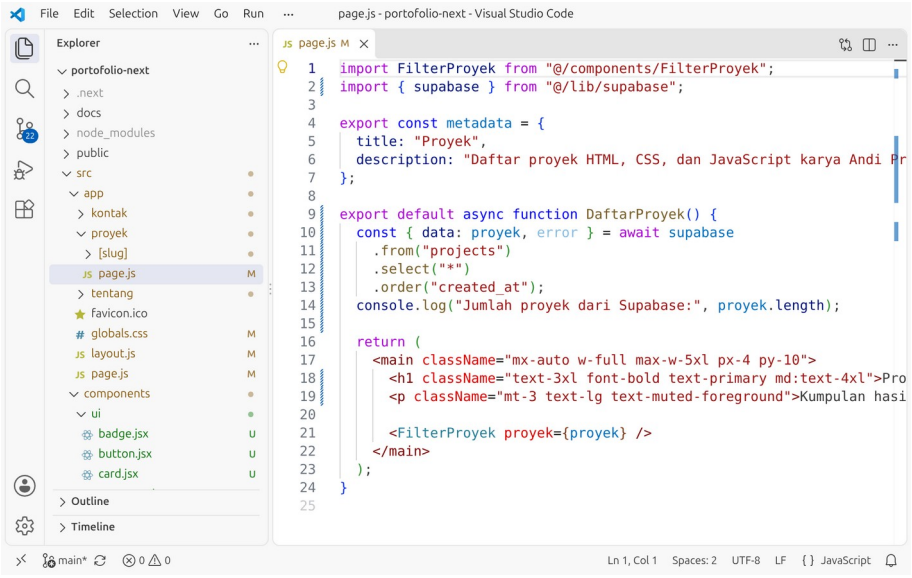
export default async function DaftarProyek() {
  const { data: proyek, error } = await supabase
    .from("projects")
    .select("*")
    .order("created_at");
```

```
  console.log("Jumlah proyek dari Supabase:", proyek.length);

  return (
    <main className="mx-auto w-full max-w-5xl px-4 py-10">
      <h1 className="text-3xl font-bold text-primary
md:text-4xl">Proyek Saya</h1>
      <p className="mt-3 text-lg text-muted-foreground">Kumpulan
hasil belajar saya di Kelas Industri UBIG. Klik kartu untuk melihat
detailnya.</p>

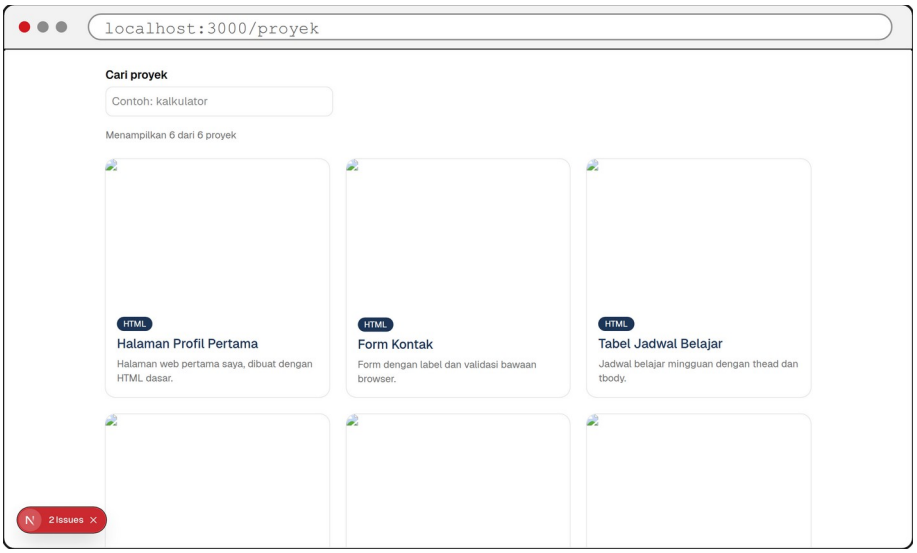
      <FilterProyek proyek={proyek} />
    </main>
  );
}
```

`const { data: proyek, error } = ...` mengambil bagian `data` dan menamainya `proyek`, sehingga sisa kode tetap memakai nama `proyek` seperti sebelumnya. Kode di VS Code terlihat pada Gambar 4.5.



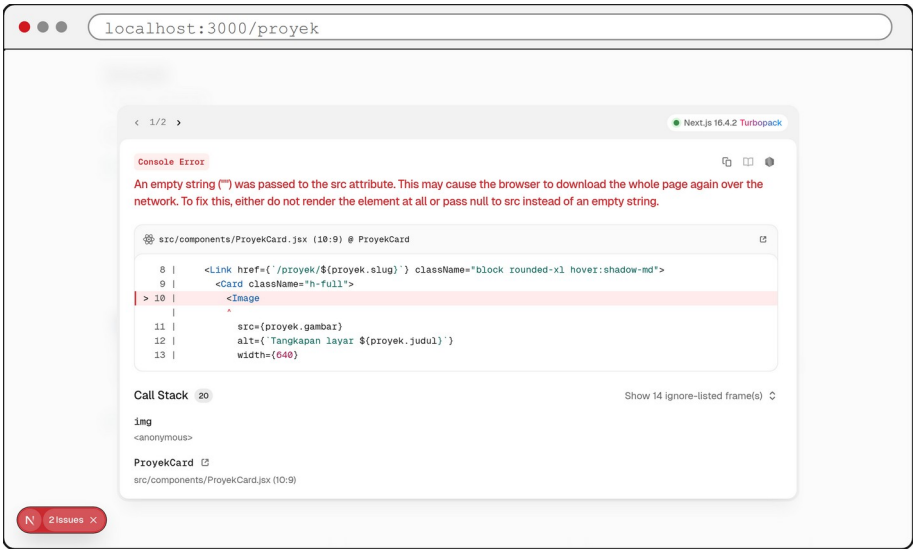

**Gambar 4.5** page.js halaman Proyek dengan await supabase

Jalankan `npm run dev`, lalu buka halaman Proyek. Data sudah datang (ada enam kartu dan tulisan *Menampilkan 6 dari 6 proyek*), tetapi semua gambar kosong, seperti pada Gambar 4.6.




**Gambar 4.6** Kartu tampil tanpa gambar

Klik lencana merah **2 Issues** di kiri bawah untuk membaca penyebabnya. Next.js menunjuk baris `src={proyek.gambar}` di `ProyekCard.jsx`, seperti pada Gambar 4.7.

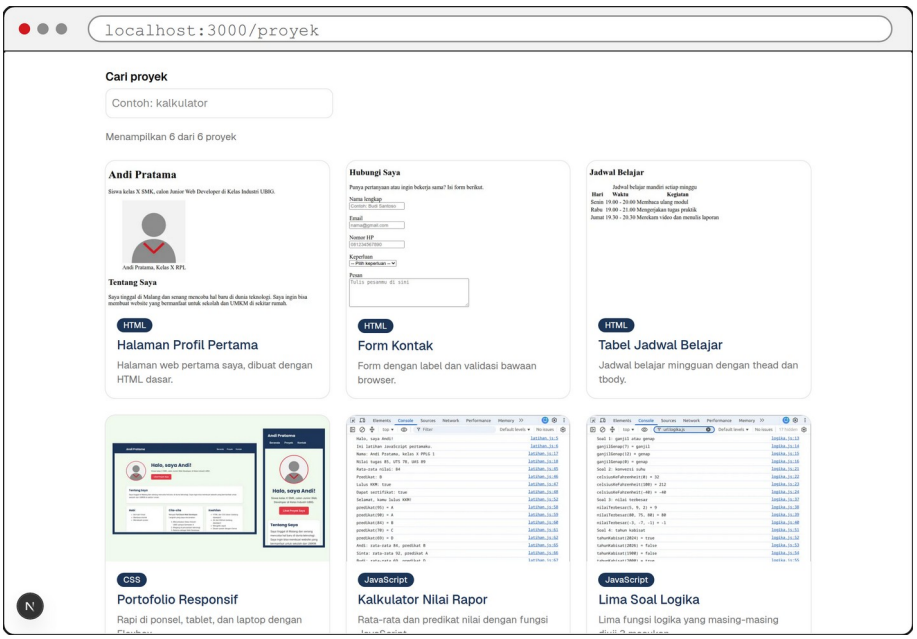



**Gambar 4.7** Next.js menunjuk src={proyek.gambar} yang kosong

Penyebabnya, data lama di `proyek.js` menyimpan alamat gambar di properti `gambar`, sedangkan tabel Supabase menamai kolomnya `gambar_url` (Minggu 13). Properti `proyek.gambar` tidak ada, sehingga nilainya kosong.

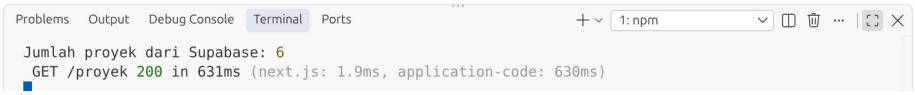
### *4.3.4 Tahap 4: Menyesuaikan ProyekCard dengan Kolom gambar_url*

Buka berkas `ProyekCard.jsx` di folder `src/components`. Pada atribut `src` milik `Image`, ganti `proyek.gambar` menjadi `proyek.gambar_url`, lalu simpan. Muat ulang halaman Proyek; gambar kembali tampil, seperti pada Gambar 4.8.



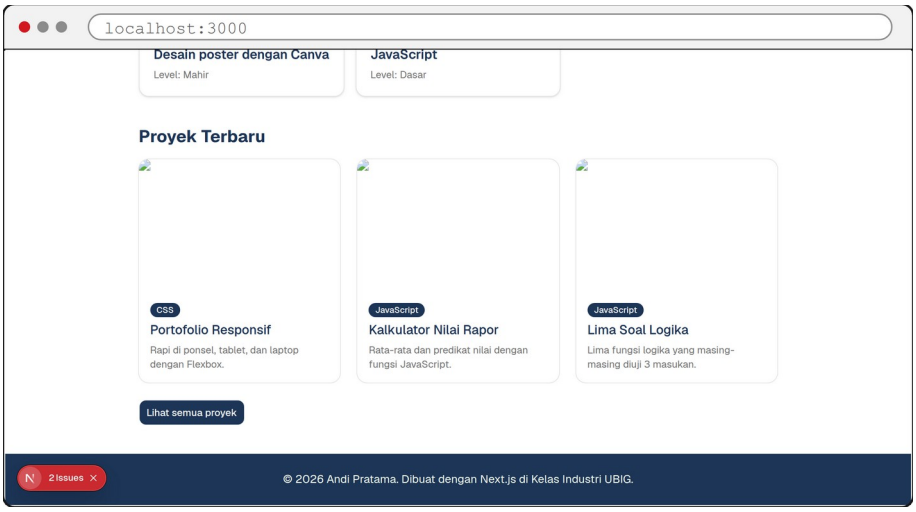

**Gambar 4.8** Halaman Proyek menampilkan data dari Supabase

Lihat terminal VS Code. Pesan dari `console.log` muncul di sana, seperti pada Gambar 4.9, bukan di Console *browser*. Ini bukti bahwa kode halaman Proyek berjalan di *server*.




**Gambar 4.9** Pesan console.log server component muncul di terminal

Namun Beranda sekarang bermasalah: kartu Proyek Terbaru kehilangan gambarnya, seperti pada Gambar 4.10. Beranda masih mengambil data dari `proyek.js` yang tidak punya `gambar_url`, padahal `ProyekCard` dipakai bersama oleh kedua halaman. Beranda diperbaiki di Tahap 6.




**Gambar 4.10** Kartu Proyek Terbaru di Beranda kehilangan gambar

### *4.3.5 Tahap 5: Menangani Error dan Data Kosong*

Hapus baris `console.log` yang tadi ditambahkan. Lalu ganti baris berisi komponen `FilterProyek` dengan tiga kemungkinan tampilan: pesan saat gagal, pesan saat tabel kosong, dan daftar proyek saat datanya ada. Isi lengkap halaman Proyek menjadi seperti Kode 4.4.

**Kode 4.4** src/app/proyek/page.js lengkap dengan pesan error dan data kosong

```
import FilterProyek from "@/components/FilterProyek";
import { supabase } from "@/lib/supabase";

export const metadata = {
  title: "Proyek",
  description: "Daftar proyek HTML, CSS, dan JavaScript karya Andi
Pratama.",
};

export default async function DaftarProyek() {
  const { data: proyek, error } = await supabase
    .from("projects")
    .select("*")
    .order("created_at");

  return (
    <main className="mx-auto w-full max-w-5xl px-4 py-10">
      <h1 className="text-3xl font-bold text-primary
md:text-4xl">Proyek Saya</h1>
      <p className="mt-3 text-lg text-muted-foreground">Kumpulan
hasil belajar saya di Kelas Industri UBIG. Klik kartu untuk melihat
detailnya.</p>

      {error && (
        <p className="mt-8 rounded-xl border border-dashed border-
```

```
input p-6 text-center text-aksen">
          Data proyek gagal dimuat: {error.message}
        </p>
      )}
      {proyek && proyek.length === 0 && (
        <p className="mt-8 rounded-xl border border-dashed border-
input p-6 text-center text-muted-foreground">
          Belum ada proyek di database.
        </p>
      )}
      {proyek && proyek.length > 0 && <FilterProyek
proyek={proyek} />}
    </main>
  );
}
```

Ketiga baris memakai `&&` dari Minggu 12: bagian kanan hanya ditampilkan jika syarat di kiri benar. Jika permintaan gagal, `proyek` bernilai `null`, sehingga baris kedua dan ketiga tidak dijalankan.

Untuk membuktikan pesan *error* bekerja, ganti sementara `.from("projects")` menjadi `.from("project")` (tanpa huruf s). Supabase menolak dengan pesan *Could not find the table 'public.project'*, dan halaman menampilkan pesan pada Gambar 4.11.



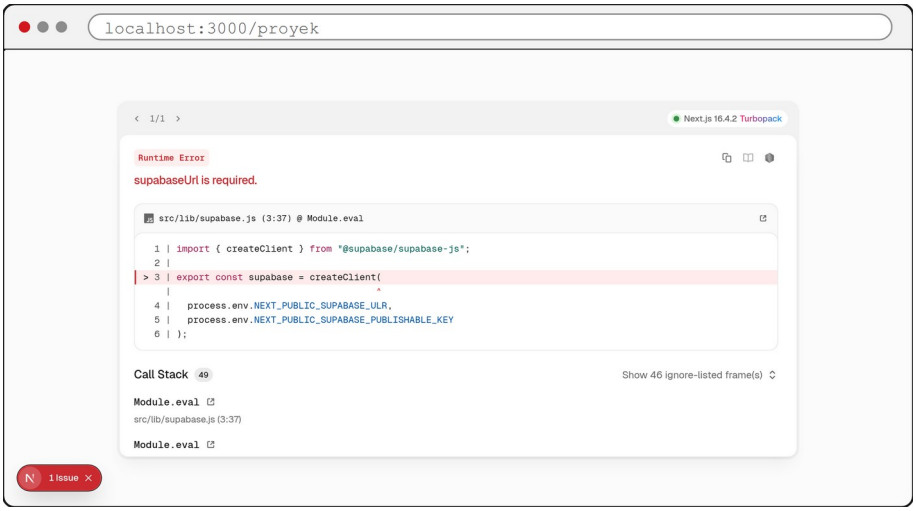

**Gambar 4.11** Pesan saat nama tabel salah

Kembalikan menjadi `projects`. Untuk membuktikan pesan data kosong, tambahkan sementara `.eq("kategori", "Next.js")` di bawah `.select("*")`. Belum ada proyek berkategori Next.js, sehingga halaman menampilkan pesan pada Gambar 4.12. Hapus lagi baris tersebut setelah mencoba.




**Gambar 4.12** Pesan saat tidak ada data

> **Kesalahan Umum**
>
> Jika halaman menampilkan *Runtime Error: supabaseUrl is required.* seperti pada Gambar 4.13, alamat Supabase dari `.env.local` tidak terbaca. Penyebabnya: nama variabel salah ketik (misalnya `ULR`), `.env.local` tidak berada di folder utama project, atau `npm run dev` belum dijalankan ulang setelah `.env.local` dibuat. Samakan nama variabel di `supabase.js` dan `.env.local`, lalu jalankan ulang `npm run dev`.




**Gambar 4.13** Error supabaseUrl is required karena nama variabel salah ketik

### *4.3.6 Tahap 6: Proyek Terbaru di Beranda dari Supabase*

Buka berkas `page.js` di folder `src/app`, lalu lakukan empat perubahan berikut:

1. ganti impor `proyek` dengan impor `supabase`;
2. hapus baris `proyekTerbaru` yang memakai `slice`;
3. ubah fungsi `Home` menjadi `async`;
4. ambil tiga proyek terbaru di awal fungsi, seperti pada Kode 4.5.

**Kode 4.5** src/app/page.js: tiga proyek terbaru dari Supabase

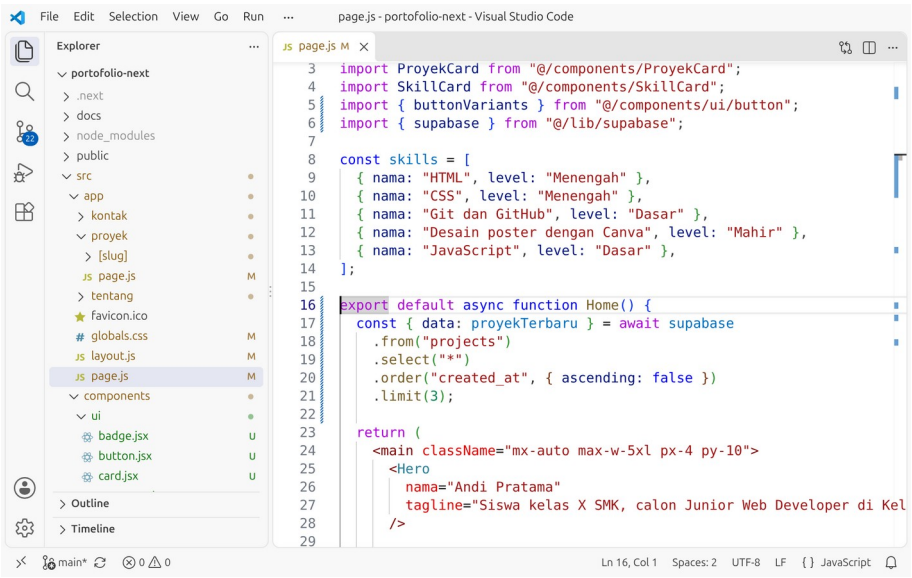
```
import { supabase } from "@/lib/supabase";

...

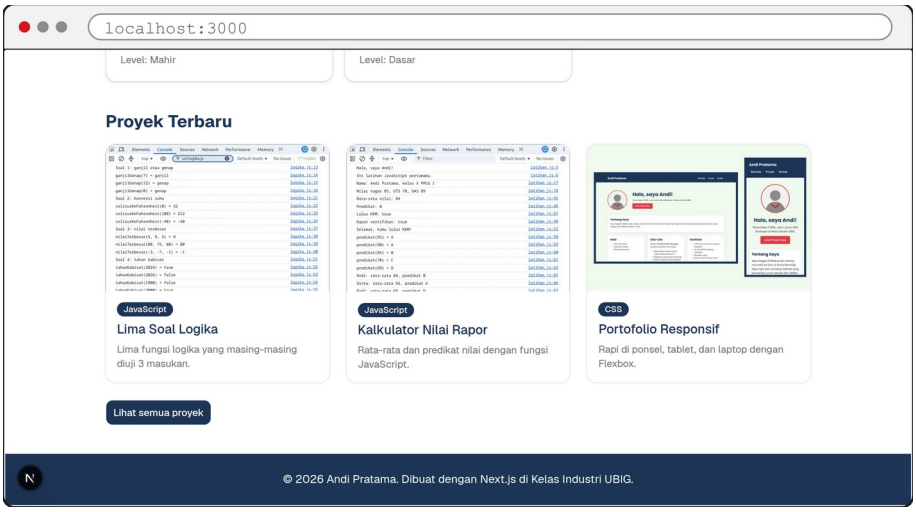
export default async function Home() {
  const { data: proyekTerbaru } = await supabase
    .from("projects")
    .select("*")
    .order("created_at", { ascending: false })
    .limit(3);

  return (
    ...
```

Pilihan `{ ascending: false }` mengurutkan dari yang terbaru, lalu `.limit(3)` mengambil tiga baris pertama. Di bagian daftar, tambahkan `proyekTerbaru &&` tepat di depan `proyekTerbaru.map(` agar halaman tidak *error* jika data gagal dimuat. Kode di VS Code terlihat pada Gambar 4.14 dan Beranda terlihat pada Gambar 4.15.




**Gambar 4.14** page.js Beranda mengambil tiga proyek terbaru




**Gambar 4.15** Proyek Terbaru di Beranda dari Supabase

## 4.4 Rangkuman

- `@supabase/supabase-js` adalah pustaka resmi untuk berbicara dengan Supabase dari JavaScript.
- Klien dibuat sekali di `src/lib/supabase.js` dengan `createClient` dan nilai dari `.env.local`.
- Halaman yang mengambil data diberi `async`, permintaannya diberi `await`, dan hasilnya `{ data, error }`.
- Nama kolom di database (`gambar_url`) harus sama dengan nama yang dipakai di komponen.
- Selalu tangani tiga keadaan: gagal (`error`), kosong (`length === 0`), dan ada data.

## 4.5 Latihan

1. Mengapa klien Supabase dibuat di `src/lib/supabase.js`, bukan di setiap halaman?
2. Apa isi `data` dan `error` ketika permintaan berhasil, dan ketika gagal?
3. Mengapa `console.log` di halaman Proyek muncul di terminal VS Code, bukan di Console *browser*?
4. Tuliskan permintaan untuk mengambil tiga proyek terbaru dari tabel `projects`.
5. Apa penyebab pesan *supabaseUrl is required.* dan bagaimana memperbaikinya?

# BAB V
# KEGIATAN PEMBELAJARAN 4: HALAMAN DETAIL DARI DATABASE

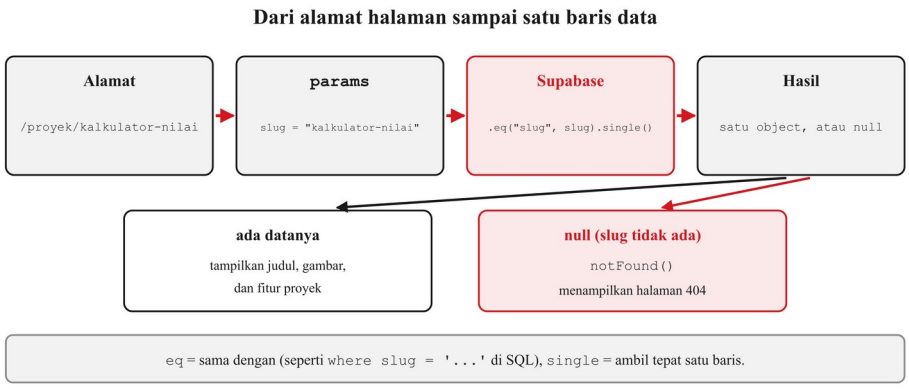
## 5.1 Tujuan Kegiatan Pembelajaran

Setelah mengikuti kegiatan ini, kamu mampu mengambil satu proyek berdasarkan *slug* dengan `eq` dan `single`, menampilkan halaman 404 untuk *slug* yang tidak ada, memakai `Badge` dan `Separator` di halaman detail, lalu menghapus data lama `src/data/proyek.js` dengan aman.

## 5.2 Uraian Materi

### *5.2.1 Mengambil Satu Baris dengan eq dan single*

Halaman detail di Minggu 11 mencari proyek dengan `proyek.find(...)` di array. Sekarang pencariannya dikerjakan database: `.eq("slug", slug)` menyaring baris yang `slug`-nya sama dengan alamat halaman, lalu `.single()` mengambil tepat satu baris sebagai object, bukan array. Jika tidak ada baris yang cocok, `data` bernilai `null`, sehingga halaman memanggil `notFound()` seperti di Minggu 11. Alurnya terlihat pada Gambar 5.1.




**Gambar 5.1** Dari alamat halaman sampai satu baris data

### *5.2.2 Satu Fungsi untuk Dua Tempat*

Halaman detail membutuhkan data proyek di dua tempat: `generateMetadata` untuk judul tab, dan komponen halaman untuk isinya. Agar permintaan tidak ditulis dua kali, permintaan dibungkus dalam fungsi `ambilProyek(slug)`, lalu fungsi itu dipanggil di kedua tempat. Ini menerapkan prinsip fungsi dari Minggu 06: tulis sekali, pakai berkali-kali.

## 5.3 Praktik: Halaman Detail dari Database

Praktik berikut mengubah halaman detail agar membaca Supabase, menguji *slug* yang salah, lalu menghapus `src/data/proyek.js` yang sudah tidak dipakai.

### *5.3.1 Tahap 1: Mengubah Halaman Detail*

Buka berkas `page.js` di folder `src/app/proyek/[slug]`. Ganti bagian atas berkas, yaitu impor, fungsi `ambilProyek`, dan `generateMetadata`, dengan Kode 5.1. Impor `proyek` dari `@/data/proyek` dihapus dan diganti impor `supabase`, `Badge`, dan `Separator`.

**Kode 5.1** src/app/proyek/[slug]/page.js bagian atas

```
import Image from "next/image";
import Link from "next/link";
import { notFound } from "next/navigation";
import { Badge } from "@/components/ui/badge";
import { Separator } from "@/components/ui/separator";
import { supabase } from "@/lib/supabase";

async function ambilProyek(slug) {
  const { data } = await supabase
    .from("projects")
    .select("*")
    .eq("slug", slug)
    .single();
  return data;
}

export async function generateMetadata({ params }) {
  const { slug } = await params;
  const item = await ambilProyek(slug);

  if (!item) {
    return { title: "Proyek tidak ditemukan" };
  }
  return { title: item.judul, description: item.deskripsi };
}
```

Lalu ganti komponen halaman dengan Kode 5.2. Perhatikan tiga perubahan: data diambil dengan `await ambilProyek(slug)`, kategori ditampilkan dengan `Badge`, gambar memakai `item.gambar_url`, dan garis pemisah sebelum judul *Yang Saya Pelajari* memakai `Separator`.

**Kode 5.2** src/app/proyek/[slug]/page.js bagian komponen halaman

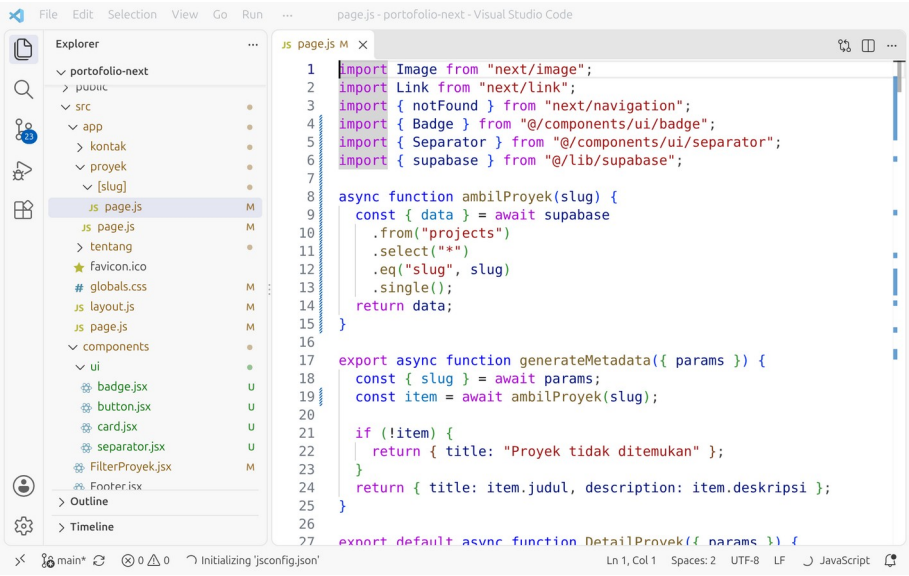
```
export default async function DetailProyek({ params }) {
  const { slug } = await params;
  const item = await ambilProyek(slug);

  if (!item) {
    notFound();
  }

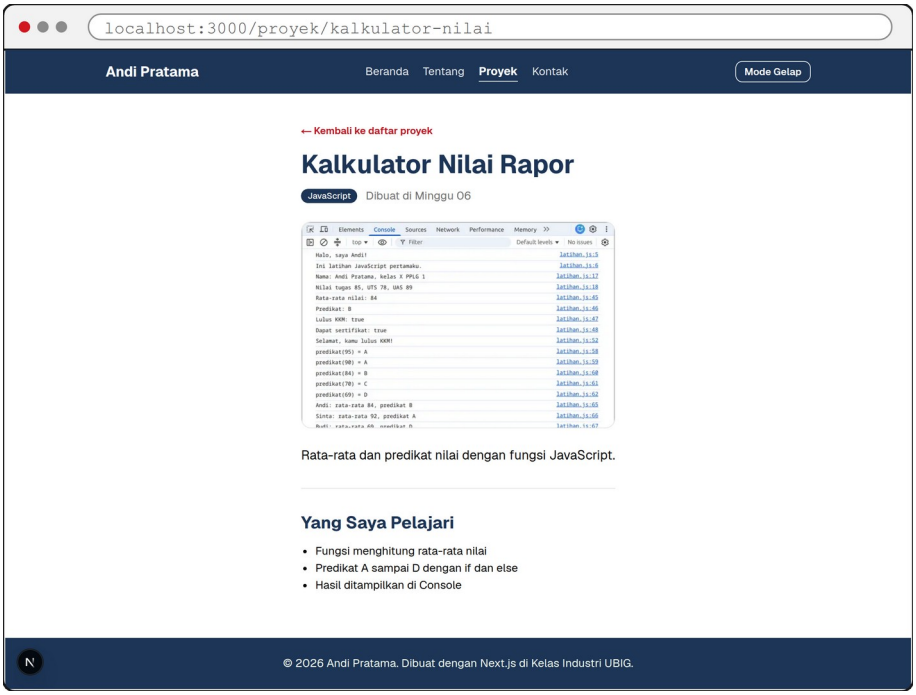
  return (
    <main className="mx-auto max-w-3xl px-4 py-10">
      <Link href="/proyek" className="text-sm font-semibold text-
aksen hover:underline">
        &larr; Kembali ke daftar proyek
      </Link>
      <h1 className="mt-4 text-3xl font-bold text-primary
md:text-4xl">{item.judul}</h1>
      <div className="mt-3 flex items-center gap-3">
        <Badge>{item.kategori}</Badge>
        <span className="text-muted-foreground">Dibuat di
{item.minggu}</span>
      </div>
      <Image
        src={item.gambar_url}
        alt={`Tangkapan layar ${item.judul}`}
        width={640}
        height={420}
        loading="eager"
        className="mt-6 h-auto w-full rounded-xl border border-
border"
      />
      <p className="mt-6 text-lg">{item.deskripsi}</p>

      <Separator className="my-8" />
      <h2 className="text-2xl font-bold text-primary">Yang Saya
Pelajari</h2>
      <ul className="mt-3 list-disc pl-5">
        {item.fitur.map((f) => (
          <li key={f}>{f}</li>
        ))}
      </ul>
    </main>
  );
}
```

Kode di VS Code terlihat pada Gambar 5.2. Buka salah satu proyek dari halaman Proyek, misalnya Kalkulator Nilai Rapor. Halamannya terlihat pada Gambar 5.3.



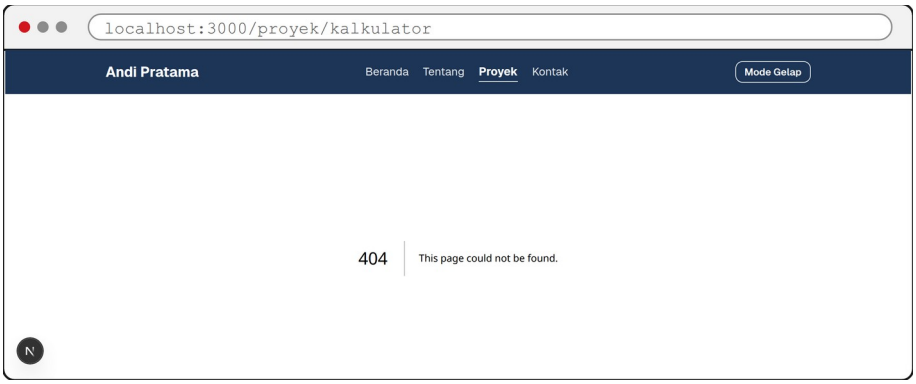

**Gambar 5.2** Halaman detail dengan fungsi ambilProyek




**Gambar 5.3** Halaman detail Kalkulator Nilai Rapor dari Supabase

### *5.3.2 Tahap 2: Mencoba Slug yang Salah*

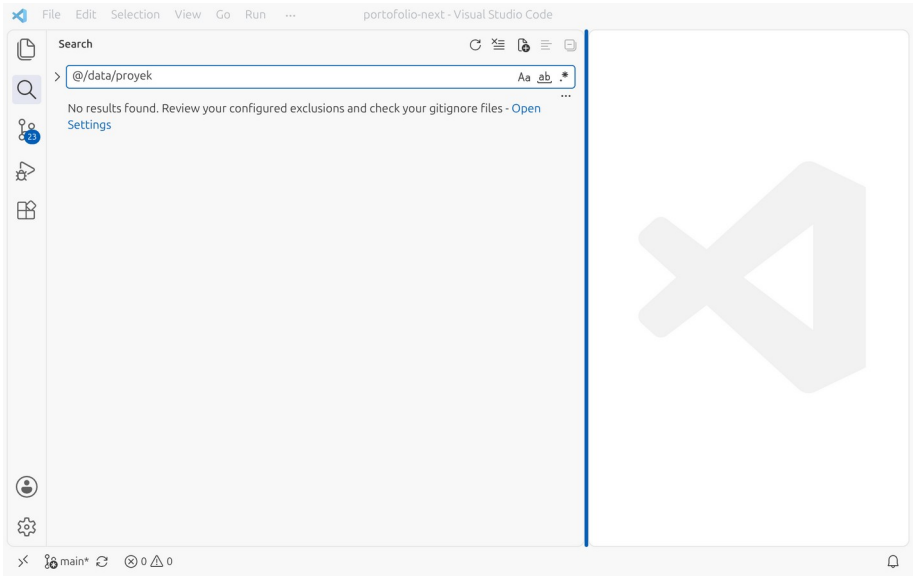
Sekarang coba buka proyek yang tidak ada. Di bilah alamat, ganti bagian akhir alamat menjadi `/proyek/kalkulator`, lalu tekan Enter. Supabase tidak menemukan baris yang cocok, `ambilProyek` mengembalikan `null`, dan `notFound()` menampilkan halaman 404, seperti pada Gambar 5.4.




**Gambar 5.4** Slug yang tidak ada menampilkan halaman 404

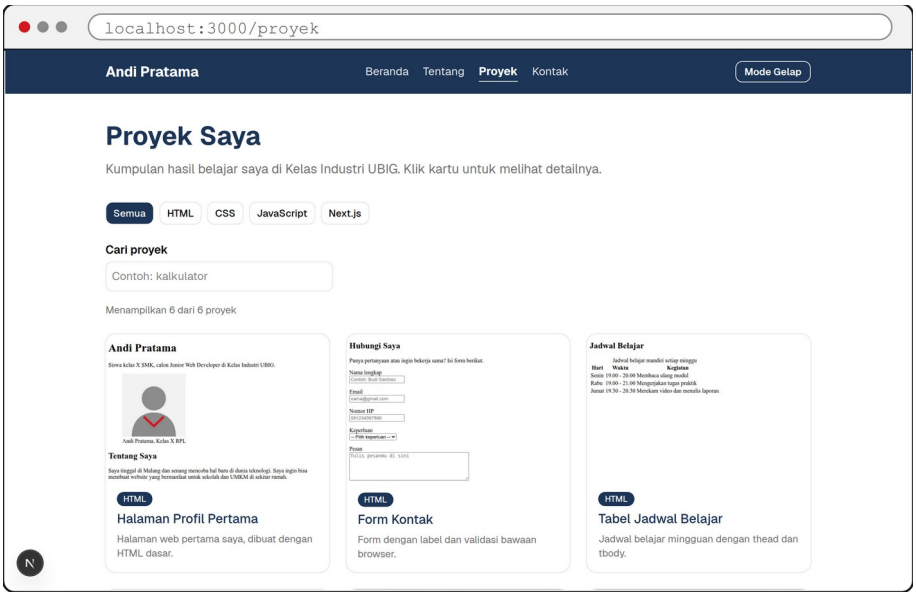
### *5.3.3 Tahap 3: Menghapus src/data/proyek.js*

Sekarang Beranda, halaman Proyek, dan halaman detail sudah membaca Supabase. Sebelum menghapus berkas lama, pastikan tidak ada lagi yang memakainya. Tekan Ctrl+Shift+F, lalu cari `@/data/proyek`. Hasilnya harus *No results found*, seperti pada Gambar 5.5.



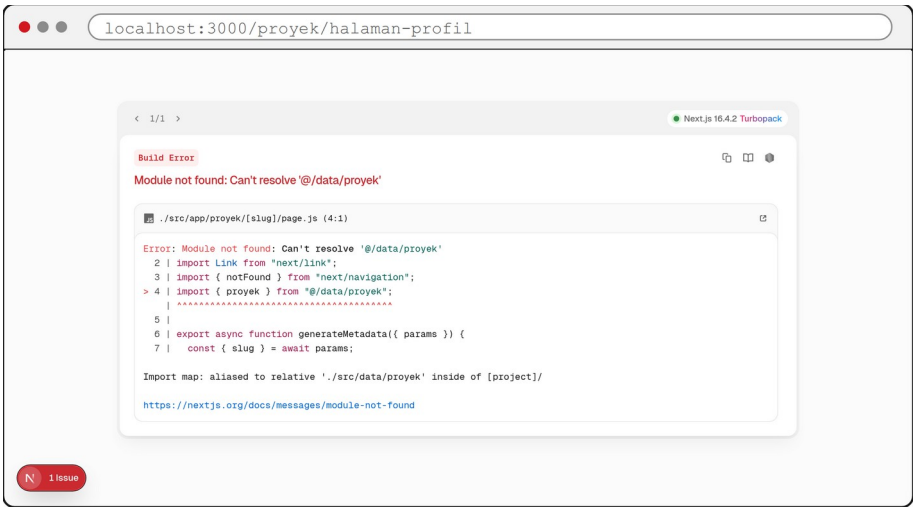

**Gambar 5.5** Pencarian @/data/proyek tidak menemukan hasil

Setelah itu, klik kanan folder `src/data` di Explorer, pilih **Delete**, lalu konfirmasi dengan **Move to Trash** (atau **Delete**). Muat ulang halaman Proyek; tampilannya tetap sama karena semua data kini berasal dari Supabase, seperti pada Gambar 5.6.




**Gambar 5.6** Halaman Proyek tetap tampil setelah proyek.js dihapus

> **Kesalahan Umum**
>
> Jika `proyek.js` dihapus sebelum semua halaman diubah, Next.js menampilkan *Build Error: Module not found: Can't resolve '@/data/proyek'* seperti pada Gambar 5.7. Pesan itu menunjukkan berkas dan baris yang masih memakainya, misalnya halaman detail baris 4. Ubah berkas tersebut sesuai Tahap 1, atau kembalikan `proyek.js` dari Trash.


localhost:3000/proyek/halaman-profil
1/1
Next.js 16.4.2 Turbopack
Build Error
Module not found: Can't resolve '@/data/proyek'
./src/app/proyek/[slug]/page.js (4:1)
Error: Module not found: Can't resolve '@/data/proyek'
2 | import Link from "next/link";
3 | import { notFound } from "next/navigation";
> 4 | import { proyek } from "@/data/proyek";
| ^^^

# BAB VI
# KEGIATAN PEMBELAJARAN 5: DEPLOY KE VERCEL DENGAN ENVIRONMENT VARIABLES

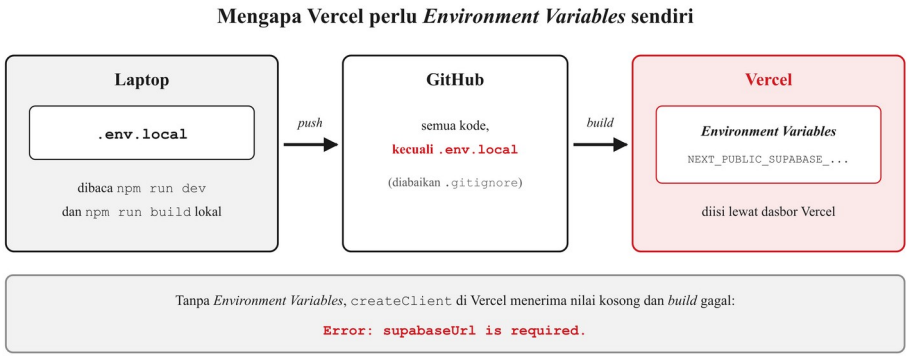
## 6.1 Tujuan Kegiatan Pembelajaran

Setelah mengikuti kegiatan ini, kamu mampu memeriksa project dengan `npm run lint` dan `npm run build`, membaca log *build* Vercel yang gagal, menambahkan *Environment Variables* dengan tipe yang tepat, menjalankan *Redeploy*, serta memastikan versi *live* membaca data dari Supabase.

## 6.2 Uraian Materi

### *6.2.1 Mengapa Vercel Perlu Environment Variables Sendiri*

Berkas `.env.local` hanya ada di laptopmu. Di Minggu 13 kamu memastikannya tidak ikut ter-*commit*, sehingga GitHub dan Vercel tidak pernah menerimanya. Akibatnya, saat Vercel menjalankan `npm run build`, alamat Supabase bernilai kosong dan `createClient` menolak bekerja. Nilai yang sama harus diisikan sendiri di dasbor Vercel, di menu **Environment Variables**, seperti pada Gambar 6.1.




**Gambar 6.1** Mengapa Vercel perlu Environment Variables sendiri

Vercel menyediakan dua tipe variabel, seperti pada Tabel 6.1. Variabel berawalan `NEXT_PUBLIC_` memang dikirim ke *browser*, jadi Vercel hanya menerimanya sebagai tipe **Config**.

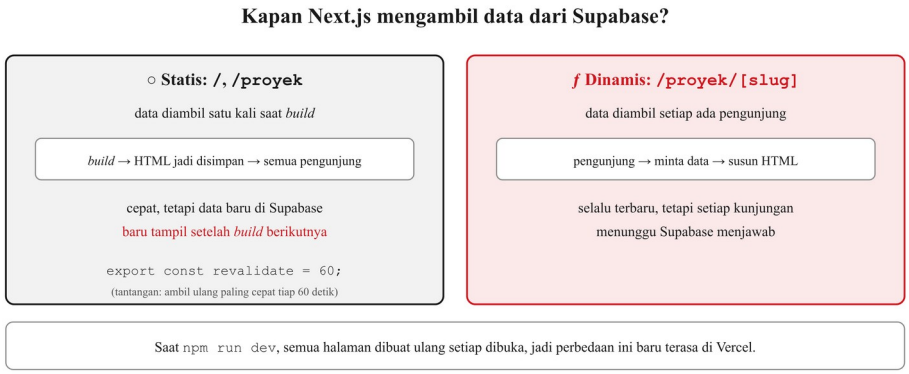
**Tabel 6.1** Tipe Environment Variables di Vercel

| No | Tipe | Sifat | Contoh |
|---|---|---|---|
| 1 | Secret | Nilai tidak bisa dilihat lagi setelah disimpan; untuk *password*, *secret key*, dan token | PIN admin di Minggu 15 |
| 2 | Config | Nilai bisa dibaca anggota project; untuk nilai yang aman diketahui umum | Dua variabel `NEXT_PUBLIC_` Supabase minggu ini |

*Publishable key* aman diketahui umum karena setiap permintaan tetap dijaga RLS (Minggu 13). *Secret key* tidak pernah boleh diberi awalan `NEXT_PUBLIC_`.

### *6.2.2 Halaman Statis dan Dinamis*

Saat *build*, Next.js memilih cara membuat setiap halaman, seperti pada Gambar 6.2. Halaman **statis** (○) dibuat sekali saat *build*, sehingga datanya diambil dari Supabase pada saat itu. Halaman **dinamis** (*ƒ*) dibuat setiap ada pengunjung, sehingga datanya selalu terbaru. Beranda dan halaman Proyek termasuk statis, sedangkan halaman detail dinamis karena alamatnya bergantung pada *slug*.




**Gambar 6.2** Kapan Next.js mengambil data dari Supabase

Artinya, jika kamu menambah proyek di Supabase, Beranda dan halaman Proyek versi *live* baru berubah setelah *deploy* berikutnya. Cara membuat halaman statis diperbarui berkala dibahas di bagian tantangan.

## 6.3 Praktik: Deploy dengan Environment Variables

Praktik berikut memeriksa project di laptop, mengirimnya ke GitHub, membaca *build* Vercel yang gagal, menambahkan *Environment Variables*, lalu menerbitkan ulang. Kegagalan pertama sengaja ditunjukkan agar kamu mengenali pesannya.

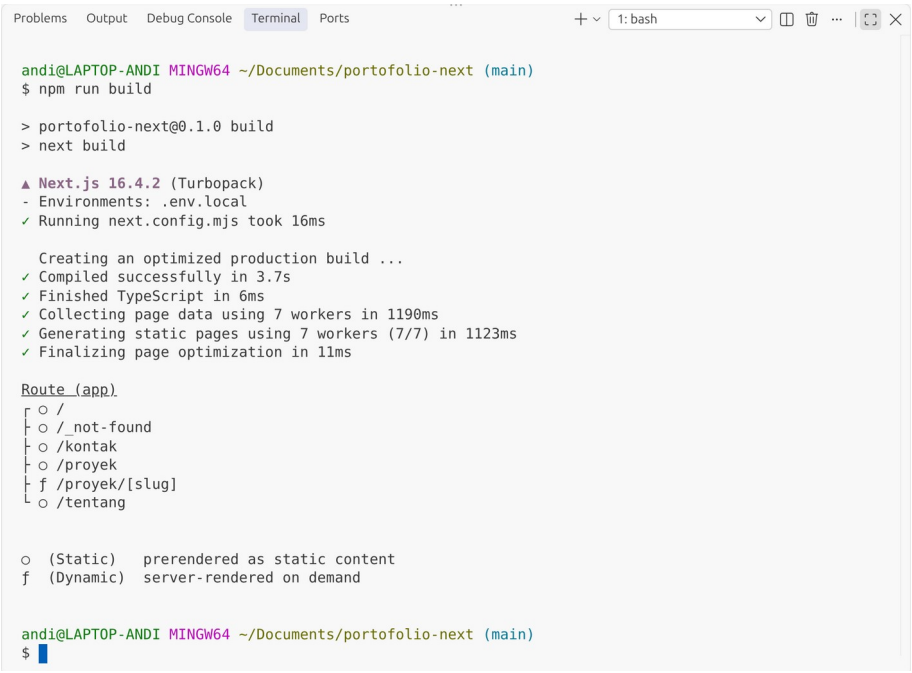
### *6.3.1 Tahap 1: Memeriksa dengan lint dan build*

Hentikan `npm run dev`, lalu jalankan dua perintah pada Kode 6.1. `npm run lint` memeriksa kesalahan penulisan kode; jika tidak ada keluaran selain judul perintah, berarti bersih. `npm run build` membuat versi produksi seperti yang dilakukan Vercel.

**Kode 6.1** Pemeriksaan sebelum push

```
$ npm run lint
$ npm run build
```

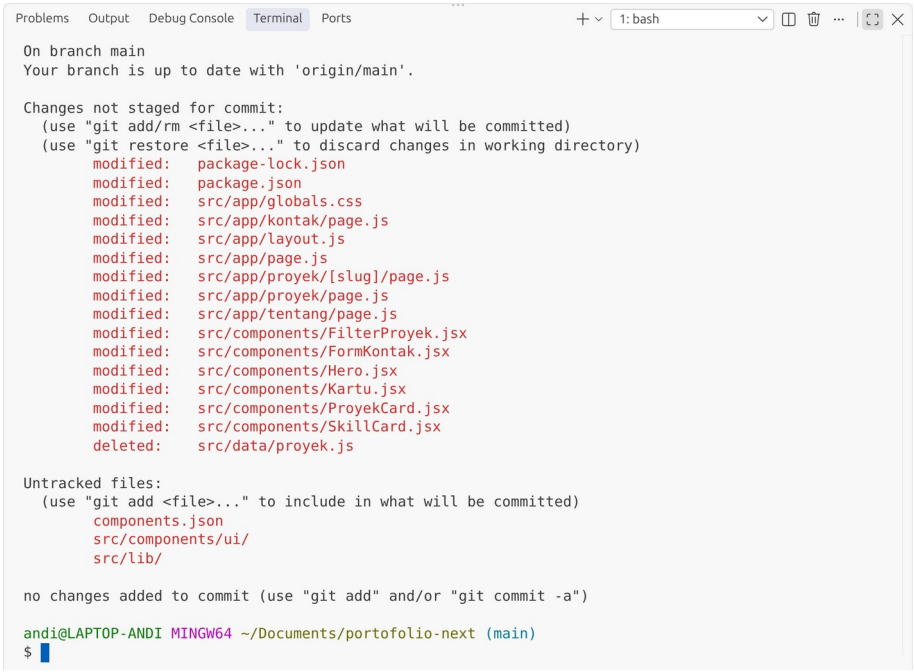
Hasil *build* terlihat pada Gambar 6.3. Perhatikan baris *Environments: .env.local*, yang menandakan *build* di laptop membaca `.env.local`, serta tanda ○ dan *ƒ* di daftar halaman.




**Gambar 6.3** Hasil npm run build di laptop

### *6.3.2 Tahap 2: Commit dan Push*

Jalankan `git status`. Daftarnya panjang, seperti pada Gambar 6.4, karena minggu ini banyak berkas berubah: berkas shadcn/ui, `package.json`, halaman, komponen, `src/lib`, dan `src/data/proyek.js` yang dihapus. Pastikan `.env.local` **tidak** ada di daftar.

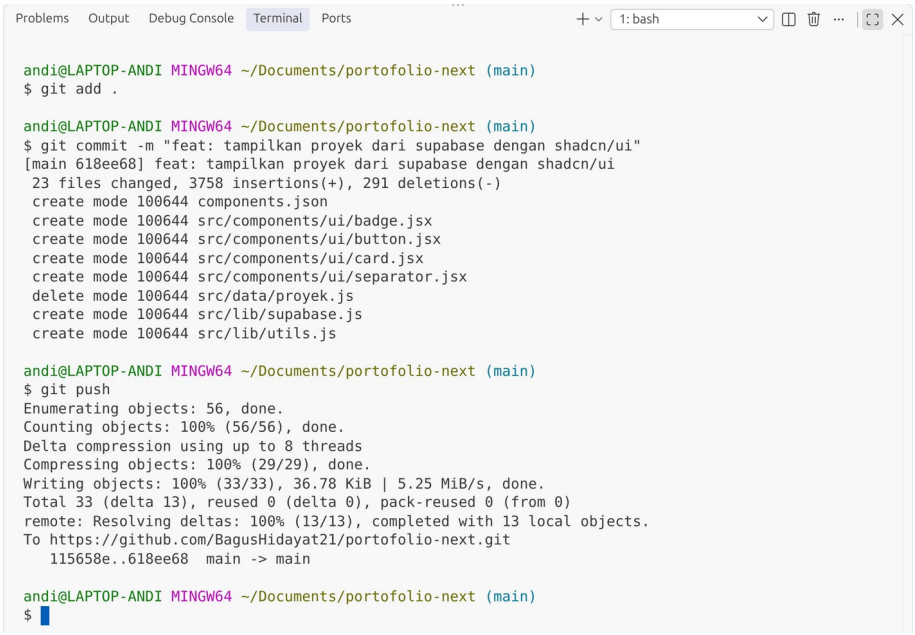



**Gambar 6.4** git status setelah praktik Minggu 14

Kirim semuanya dengan perintah pada Kode 6.2. Hasil *push* terlihat pada Gambar 6.5.

**Kode 6.2** Commit dan push praktik Minggu 14

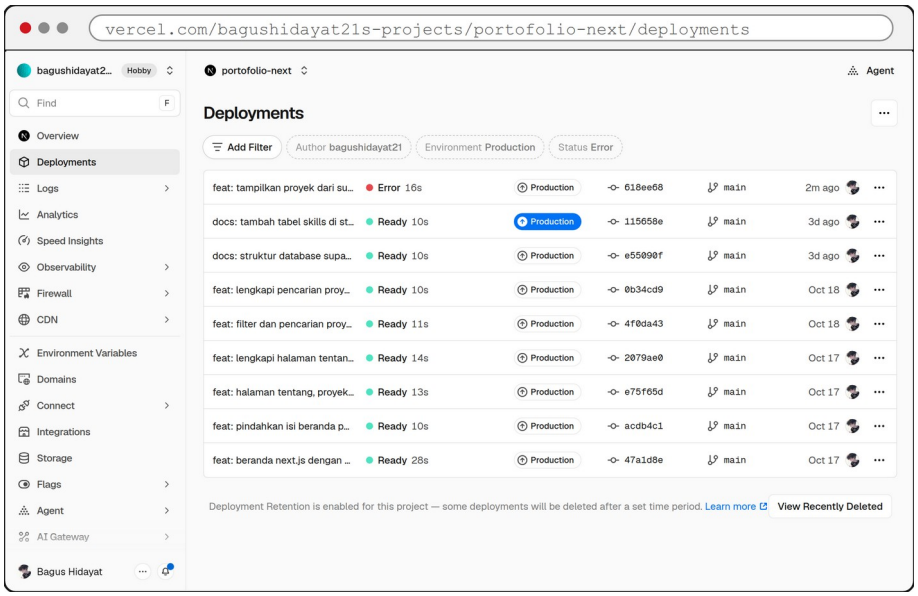
```
$ git add .
$ git commit -m "feat: tampilkan proyek dari supabase dengan
shadcn/ui"
$ git push
```




**Gambar 6.5** Push praktik Minggu 14

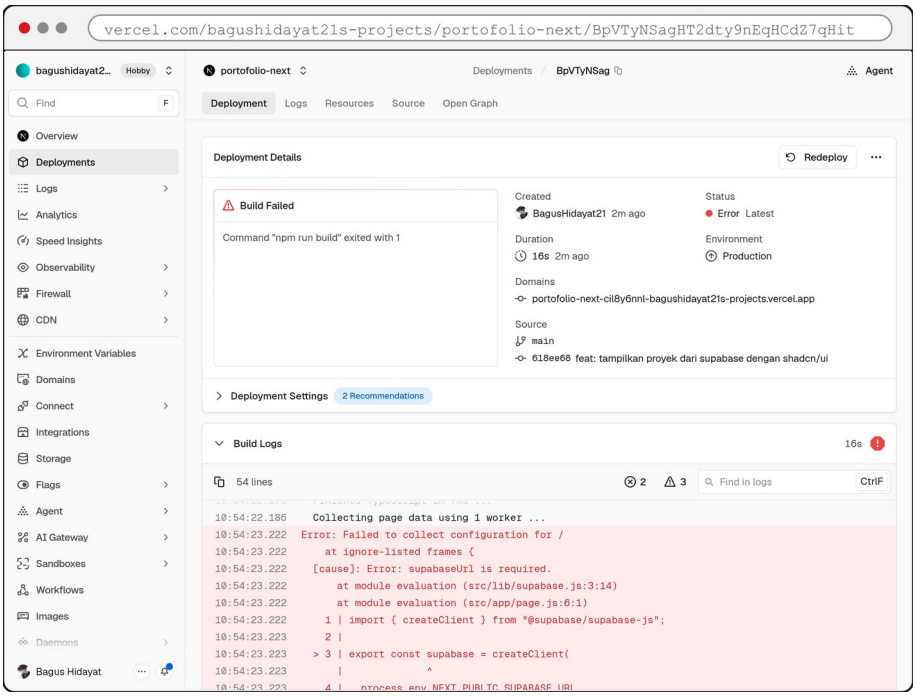
### *6.3.3 Tahap 3: Membaca Build yang Gagal*

Buka dasbor Vercel, pilih project `portofolio-next`, lalu klik **Deployments**. *Deployment* terbaru berstatus **Error**, seperti pada Gambar 6.6. Perhatikan bahwa label **Production** masih menempel di *deployment* sebelumnya: Vercel tidak mengganti website *live* dengan *build* yang gagal, sehingga pengunjung tetap melihat versi lama.




**Gambar 6.6** Deployment terbaru berstatus Error

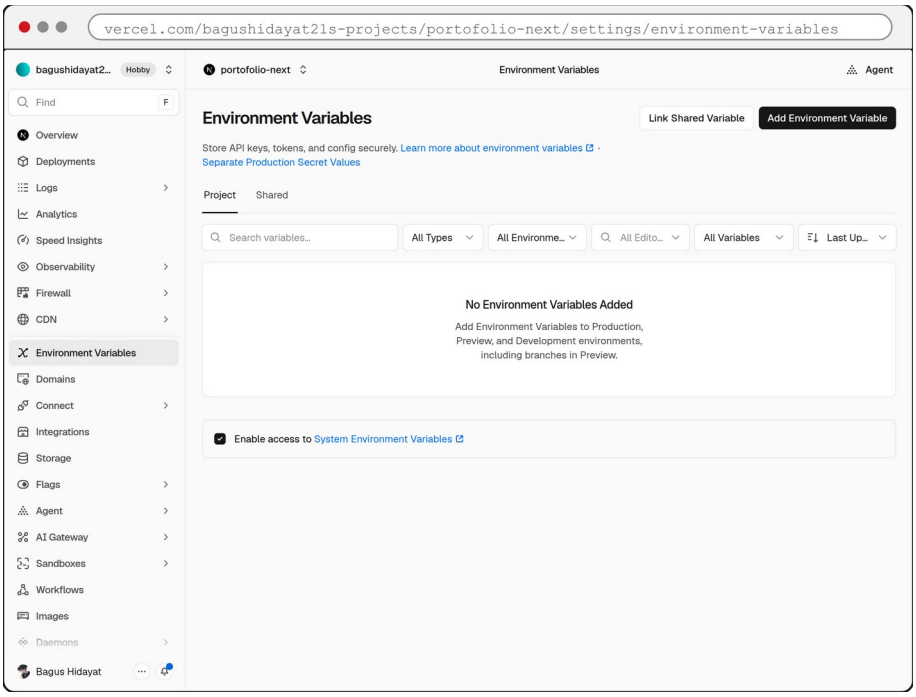
Klik baris *deployment* yang gagal untuk membaca lognya. Bagian **Build Logs** menunjukkan pesan yang sama dengan Kesalahan Umum di Kegiatan Pembelajaran 3: *supabaseUrl is required.*, seperti pada Gambar 6.7.




**Gambar 6.7** Build Logs Vercel: supabaseUrl is required

### *6.3.4 Tahap 4: Menambahkan Environment Variables*

Di menu kiri Vercel, klik **Environment Variables**. Daftarnya masih kosong, seperti pada Gambar 6.8. Klik **Add Environment Variable**.

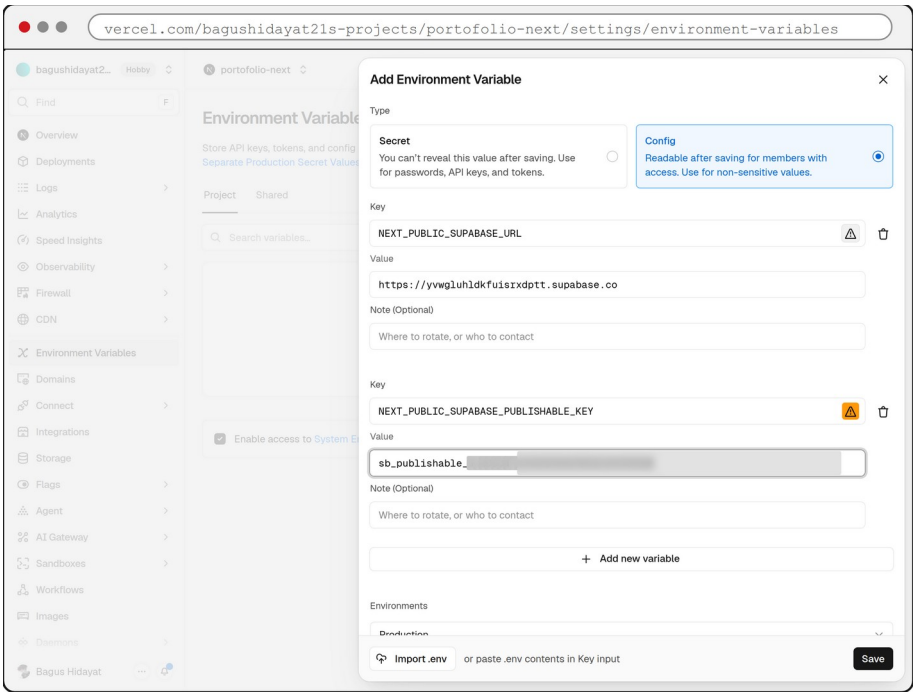



**Gambar 6.8** Halaman Environment Variables yang masih kosong

Pilih **Type Config**, lalu isi dua variabel seperti pada Tabel 6.2. Setelah variabel pertama terisi, klik **Add new variable** untuk memunculkan isian variabel kedua. Biarkan **Environments** berisi **Production**. Formulir yang sudah terisi terlihat pada Gambar 6.9; klik **Save** untuk menyimpan.

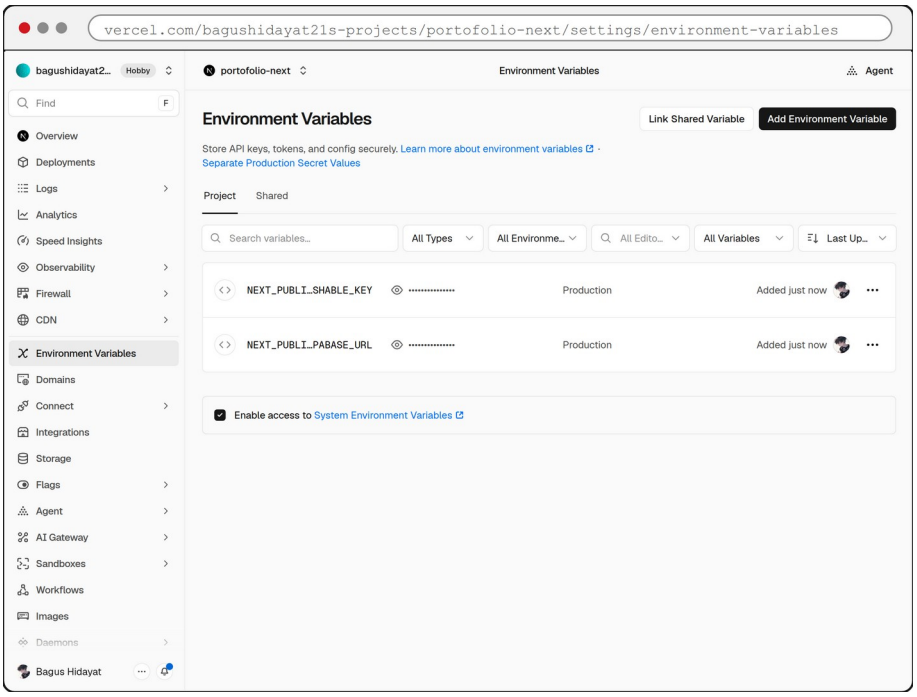
**Tabel 6.2** Isian formulir Add Environment Variable

| No | Key | Value |
|---|---|---|
| 1 | `NEXT_PUBLIC_SUPABASE_URL` | Project URL dari `.env.local` |
| 2 | `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` | *Publishable key* dari `.env.local` |




**Gambar 6.9** Dua Environment Variables bertipe Config sebelum disimpan

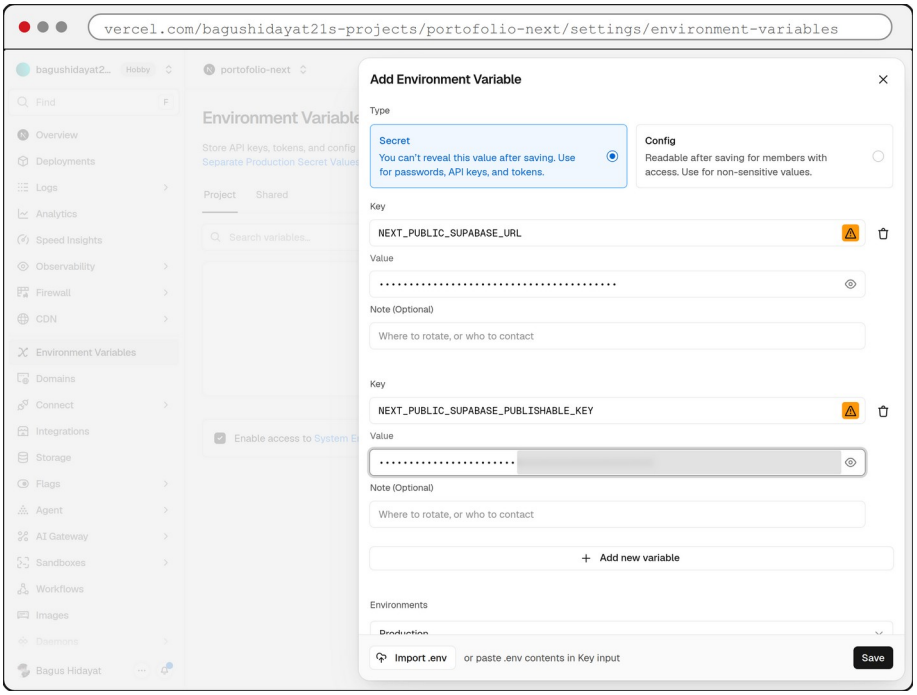
Setelah disimpan, kedua variabel tampil di daftar, seperti pada Gambar 6.10. Salin nilai dari `.env.local` tanpa spasi atau tanda kutip, dan pastikan nama **Key** sama persis dengan di `.env.local`.




**Gambar 6.10** Dua Environment Variables tersimpan

**Kesalahan Umum**

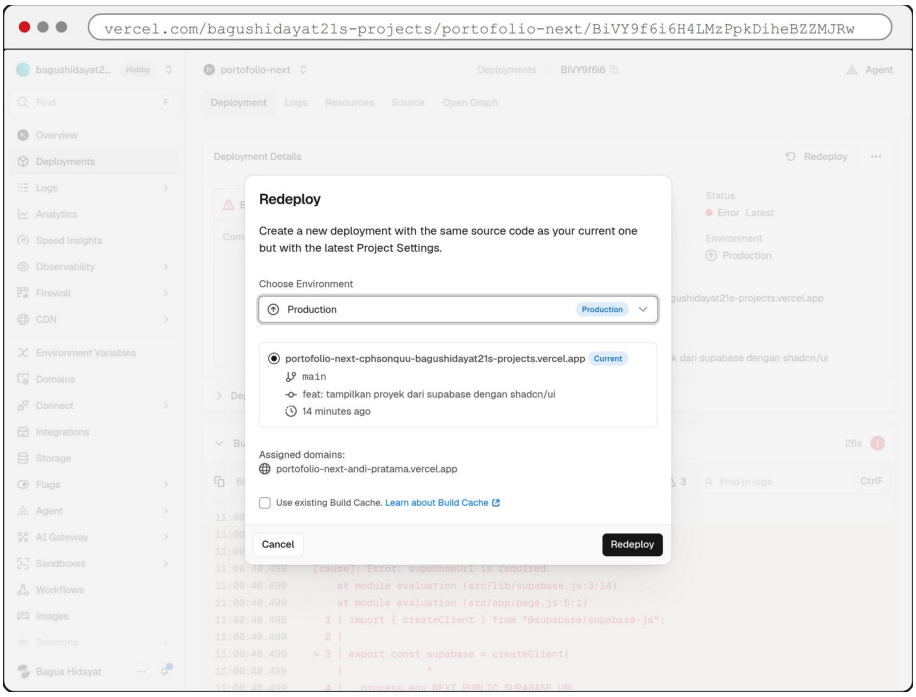
Jika **Type** dibiarkan **Secret**, muncul ikon segitiga di samping **Key** seperti pada Gambar 6.11, dan tombol **Save** tidak menyimpan apa pun. Pesannya berbunyi *The NEXT_PUBLIC_ prefix exposes this value to the browser. Remove the prefix, or change the variable to Config if it's safe to expose.* Pilih **Config**, lalu simpan lagi.




**Gambar 6.11** Tipe Secret menolak variabel berawalan NEXT_PUBLIC_

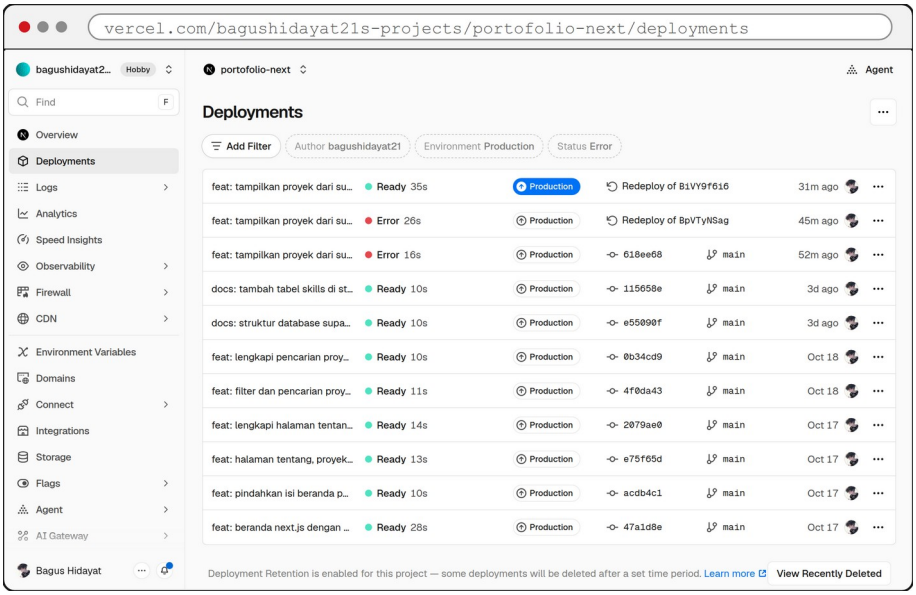
### *6.3.5 Tahap 5: Redeploy*

*Environment Variables* baru hanya dibaca oleh *build* berikutnya. Kembali ke **Deployments**, klik *deployment* yang gagal, lalu klik **Redeploy**. Jendela konfirmasi muncul seperti pada Gambar 6.12; klik **Redeploy** di dalam jendela itu.




**Gambar 6.12** Jendela konfirmasi Redeploy

Tunggu sekitar satu menit. *Deployment* baru bertuliskan *Redeploy of ...* berstatus **Ready** dan label **Production**-nya berwarna biru, tanda bahwa versi inilah yang kini tampil di website *live*, seperti pada Gambar 6.13.

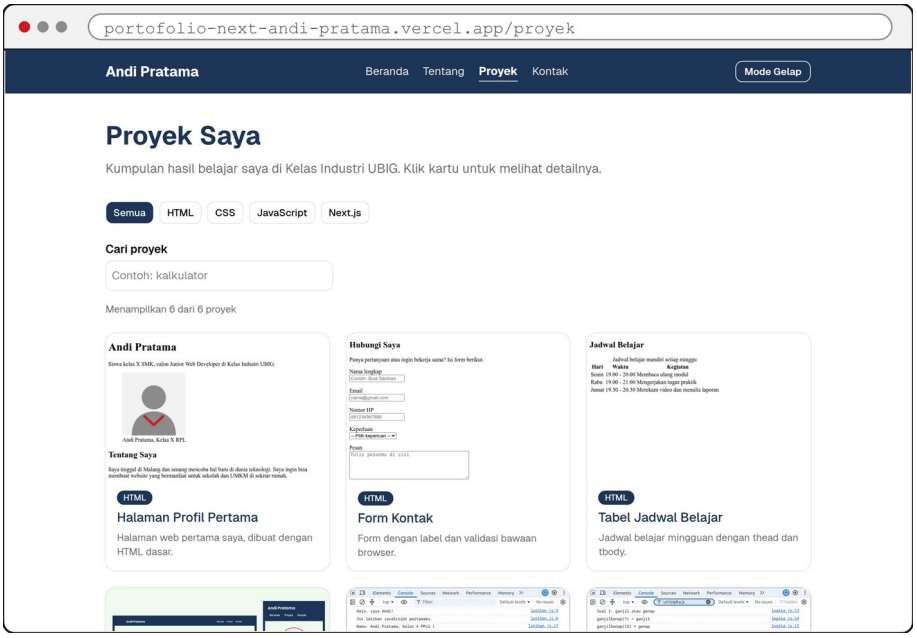



**Gambar 6.13** Redeploy berhasil dengan status Ready

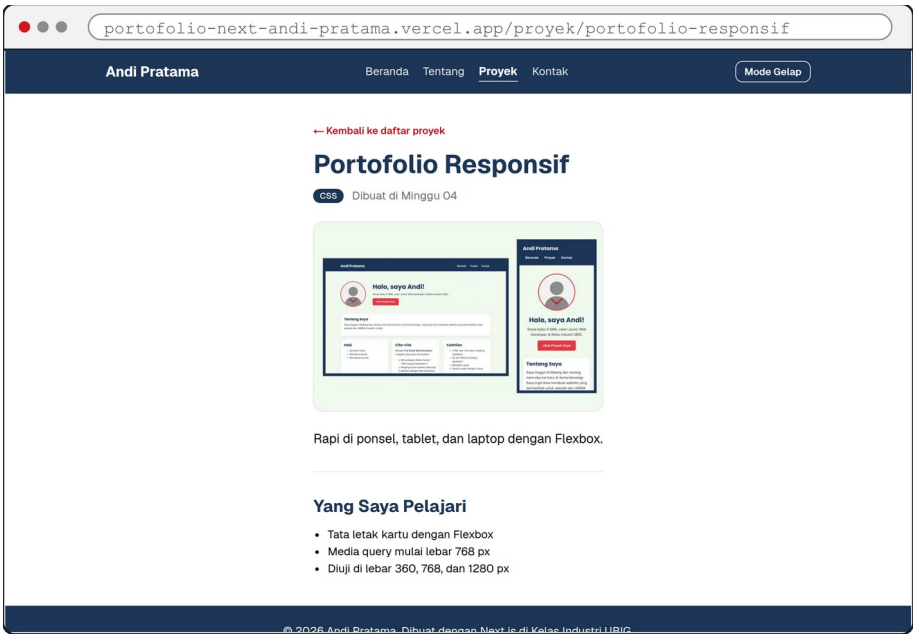
Pada contoh Andi tampak dua baris **Error** di bawahnya: *build* pertama dari *push*, dan satu *Redeploy* yang terlanjur dijalankan ketika variabel belum tersimpan karena masih bertipe Secret. Baris yang gagal boleh dibiarkan; yang penting baris teratas **Ready**.

### *6.3.6 Tahap 6: Memeriksa Versi Live*

Buka halaman Proyek di website *live* milikmu (alamat `vercel.app`). Kartu proyek kini tampil dengan `Card` dan datanya berasal dari Supabase, seperti pada Gambar 6.14. Buka satu halaman detail dan coba mode gelap, seperti pada Gambar 6.15.




**Gambar 6.14** Halaman Proyek versi live membaca Supabase




**Gambar 6.15** Halaman detail versi live di mode gelap

## 6.4 Memeriksa Hasil Kerja

Sebelum mengerjakan tugas, periksa hasil praktikmu dengan Tabel 6.3.

**Tabel 6.3** Cara memeriksa hasil kerja Minggu 14

| No | Yang Diperiksa | Cara Memeriksa |
|---|---|---|
| 1 | Huruf halaman | Beranda memakai huruf Geist, bukan Times New Roman |
| 2 | Token tema | Ctrl+Shift+F mencari `dark:` hanya menemukan pesan berhasil di `FormKontak.jsx` |
| 3 | Kartu proyek | Halaman Proyek memakai `Card` dan `Badge`; tombol kategori aktif berwarna biru portofolio |
| 4 | Data Supabase | Halaman Proyek, Beranda, dan detail tetap tampil walaupun `src/data` sudah dihapus |
| 5 | Halaman 404 | `/proyek/kalkulator` menampilkan 404 |
| 6 | Pemeriksaan | `npm run lint` tanpa pesan kesalahan dan `npm run build` berhasil |
| 7 | Versi *live* | Vercel berstatus Ready, dan halaman Proyek *live* memakai kartu baru |

## 6.5 Rangkuman

- `.env.local` tidak ikut ke GitHub, sehingga Vercel butuh *Environment Variables* sendiri.

- Variabel `NEXT_PUBLIC_` disimpan dengan tipe **Config**; *secret* tidak boleh diberi awalan itu.
- *Build* yang gagal tidak mengganti website *live*; baca **Build Logs** untuk menemukan penyebabnya.
- *Environment Variables* baru hanya berlaku setelah *Redeploy*.
- Halaman statis (○) mengambil data saat *build*, halaman dinamis (*ƒ*) setiap ada pengunjung.

## 6.6 Latihan

1. Mengapa *build* pertama di Vercel gagal walaupun di laptop berhasil?
2. Mengapa variabel `NEXT_PUBLIC_SUPABASE_URL` harus bertipe Config?
3. Apa yang terjadi pada website *live* ketika *build* terbaru gagal?
4. Mengapa setelah menambah *Environment Variables* kamu harus menjalankan *Redeploy*?
5. Apa arti tanda ○ dan *ƒ* pada hasil `npm run build`?

## 6.7 Tugas dan Pengumpulan Minggu Ini

Bagian ini hanya memuat tugas dan ketentuan pengumpulan yang khusus untuk Minggu 14. Format laporan lengkap, standar video, tenggat, dan sanksi yang berlaku setiap minggu tercantum di dokumen **Panduan dan Ketentuan Kelas Industri UBIG**.

### *6.7.1 Tugas Rumah*

1. Tampilkan keahlian di Beranda dari tabel `skills` Supabase memakai `Card` dan `Badge`.
2. Ganti kartu di halaman Tentang dan Kontak dengan `Card`.
3. Ganti tombol lain dengan komponen shadcn/ui: dua tombol di Hero, tombol Mode Gelap, dan tombol **Kirim Pesan**.
4. Periksa dengan `npm run lint` dan `npm run build`, lalu lakukan *commit* dan *push*, dan pastikan versi *live* menampilkan data dari database.
5. Ambil 3 tangkapan layar sesuai Tabel 6.5, rekam video, lalu kirim laporan.

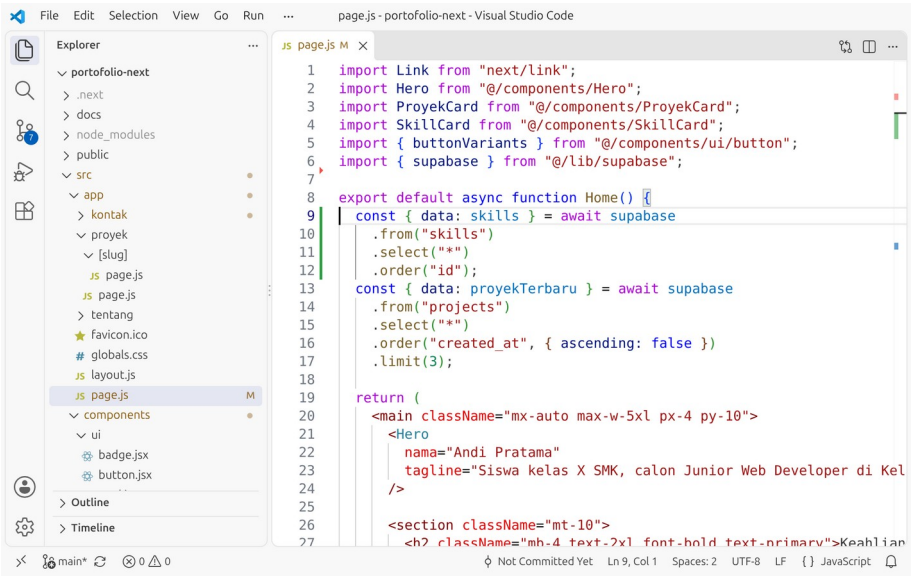
**1. Keahlian dari Supabase.** Di `src/app/page.js`, hapus konstanta `const skills = [...]` beserta isinya, lalu ambil data dari tabel `skills` di awal fungsi `Home`,

tepat sebelum permintaan proyek terbaru, seperti pada Kode 6.3. Ganti juga `{skills.map(` menjadi `{skills && skills.map(`.

**Kode 6.3** src/app/page.js: keahlian dari tabel skills

```
export default async function Home() {
  const { data: skills } = await supabase
    .from("skills")
    .select("*")
    .order("id");
  const { data: proyekTerbaru } = await supabase
    .from("projects")
    .select("*")
    .order("created_at", { ascending: false })
    .limit(3);
```

Hasil di VS Code terlihat pada Gambar 6.16.



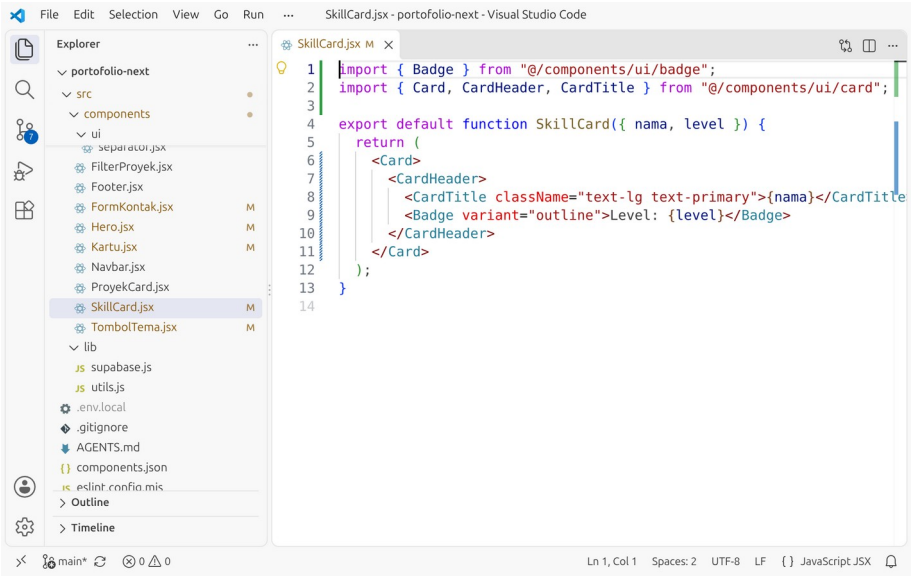

**Gambar 6.16** Contoh hasil tugas: Beranda mengambil skills dan proyek terbaru

**2. SkillCard dengan Card dan Badge.** Buka berkas `SkillCard.jsx` di folder `src/components`, lalu ganti seluruh isinya dengan Kode 6.4. Level keahlian ditampilkan sebagai `Badge` ber-*variant* `outline`. Hasilnya terlihat pada Gambar 6.17.

**Kode 6.4** src/components/SkillCard.jsx

```
import { Badge } from "@/components/ui/badge";
import { Card, CardHeader, CardTitle } from "@/components/ui/card";

export default function SkillCard({ nama, level }) {
  return (
    <Card>
      <CardHeader>
        <CardTitle className="text-lg
text-primary">{nama}</CardTitle>
        <Badge variant="outline">Level: {level}</Badge>
      </CardHeader>
    </Card>
  );
}
```




**Gambar 6.17** Contoh hasil tugas: SkillCard.jsx

**3. Kartu Tentang dan Kontak.** Ganti isi `src/components/Kartu.jsx` dengan Kode 6.5, sehingga semua kartu di halaman Tentang ikut memakai `Card`. Di `src/app/kontak/page.js`, impor `Card`, `CardContent`, `CardHeader`, dan `CardTitle`, lalu ganti kedua `<section>` berbingkai dengan Kode 6.6.

**Kode 6.5** src/components/Kartu.jsx

```
import { Card, CardContent, CardHeader, CardTitle } from
"@/components/ui/card";

export default function Kartu({ judul, children }) {
  return (
    <Card>
      <CardHeader>
        <CardTitle className="text-2xl font-bold text-
primary">{judul}</CardTitle>
      </CardHeader>
      <CardContent>{children}</CardContent>
    </Card>
  );
}
```

**Kode 6.6** src/app/kontak/page.js: dua kartu memakai Card

```
        <Card className="md:col-span-2">
          <CardContent>
            <FormKontak />
          </CardContent>
        </Card>

        <Card>
          <CardHeader>
            <CardTitle className="text-2xl font-bold text-
primary">Temukan Saya di</CardTitle>
          </CardHeader>
          <CardContent>
            <ul className="space-y-2">
              <li>
                <a href="https://github.com/BagusHidayat21"
target="_blank" className="text-aksen hover:underline">
                  GitHub
                </a>
              </li>
              <li>
                <a href="mailto:andi.pratama@example.com"
className="text-aksen hover:underline">
                  andi.pratama@example.com
                </a>
              </li>
            </ul>
          </CardContent>
        </Card>
```

**4. Tombol lain.** Buka berkas `Hero.jsx` di folder `src/components`, impor `Link` dan `buttonVariants`, lalu tambahkan dua tombol tepat di bawah paragraf `tagline`, seperti pada Kode 6.7. Di `TombolTema.jsx`, impor `Button` lalu ganti tag

`button` dengan Kode 6.8. Di `FormKontak.jsx`, ganti tombol **Kirim Pesan** dengan `<Button type="submit" size="lg">`.

**Kode 6.7** src/components/Hero.jsx: impor dan dua tombol baru

```
import Image from "next/image";
import Link from "next/link";
import { buttonVariants } from "@/components/ui/button";

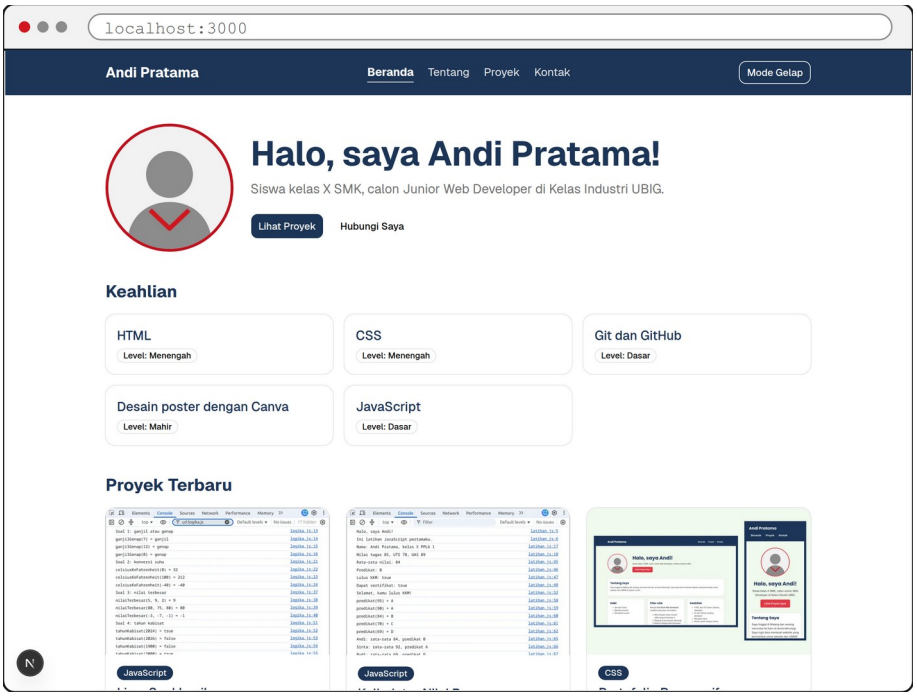
...

        <div className="mt-5 flex justify-center gap-3 md:justify-
start">
          <Link href="/proyek" className={buttonVariants({ size:
"lg" })}>
            Lihat Proyek
          </Link>
          <Link href="/kontak" className={buttonVariants({ variant:
"outline", size: "lg" })}>
            Hubungi Saya
          </Link>
        </div>
```

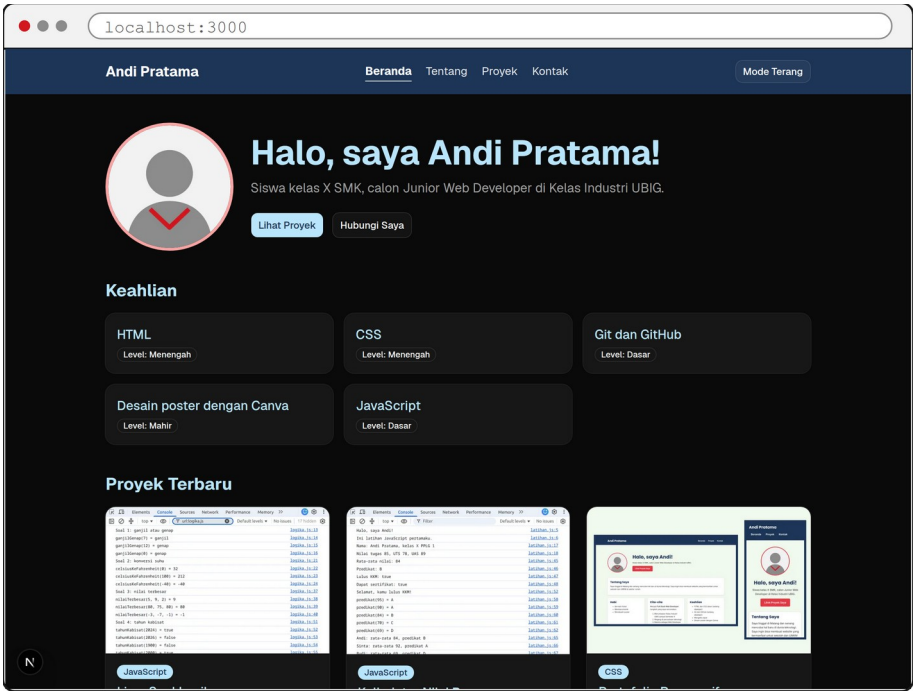
**Kode 6.8** src/components/TombolTema.jsx: tombol tema memakai Button

```
    <Button variant="outline" onClick={gantiTema} className="border-
white bg-transparent text-white hover:bg-aksen hover:text-white">
      {gelap ? "Mode Terang" : "Mode Gelap"}
    </Button>
```

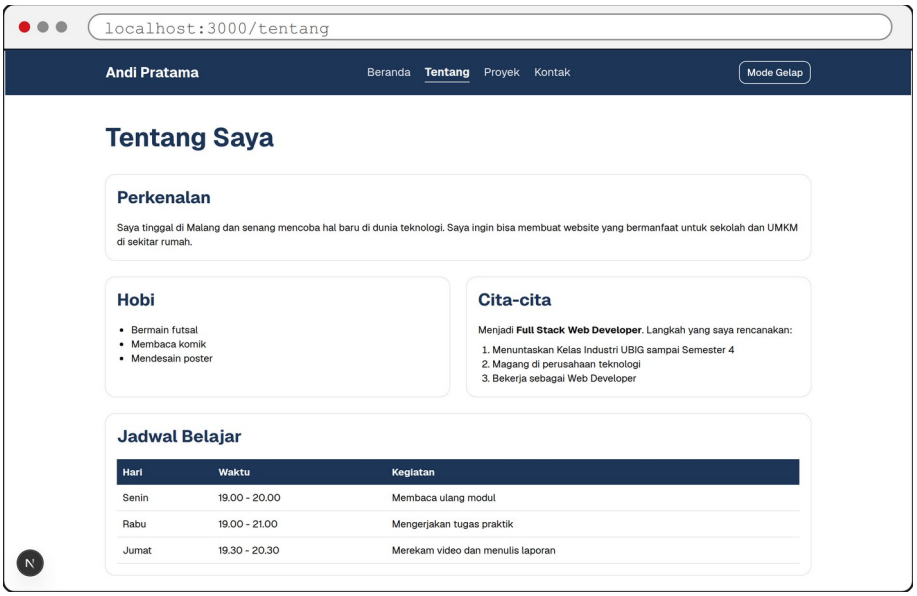
Hasil akhir Beranda terlihat pada Gambar 6.18 dan Gambar 6.19, sedangkan halaman Tentang dan Kontak terlihat pada Gambar 6.20 dan Gambar 6.21.



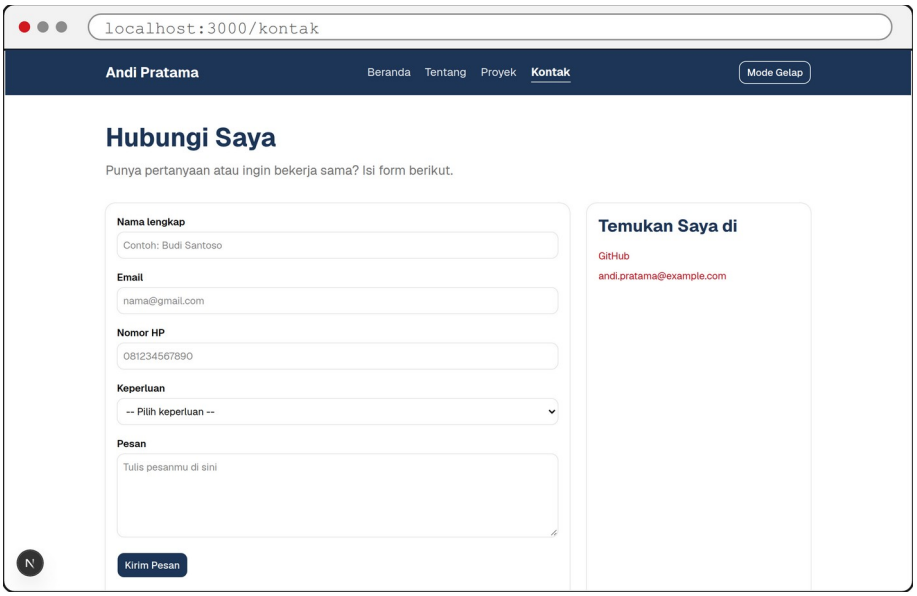

**Gambar 6.18** Contoh hasil tugas: Beranda dengan keahlian dari Supabase




**Gambar 6.19** Contoh hasil tugas: Beranda di mode gelap




**Gambar 6.20** Contoh hasil tugas: halaman Tentang dengan Card

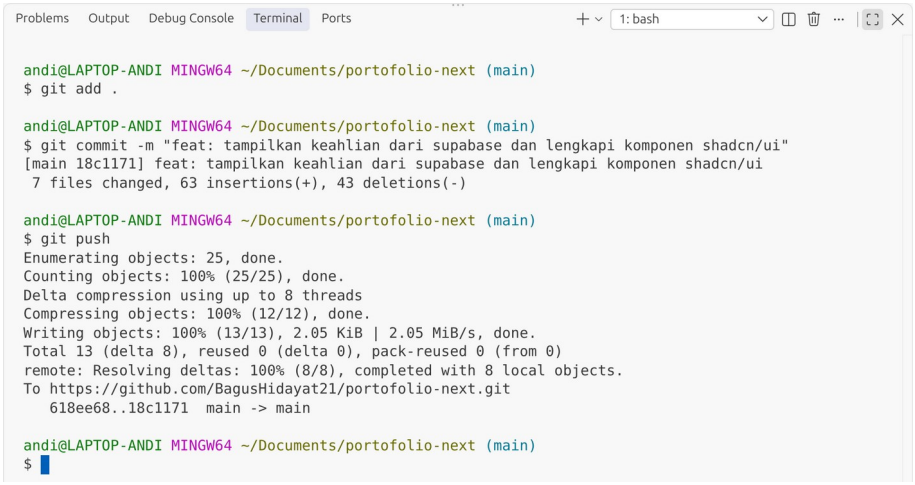



**Gambar 6.21** Contoh hasil tugas: halaman Kontak dengan Card

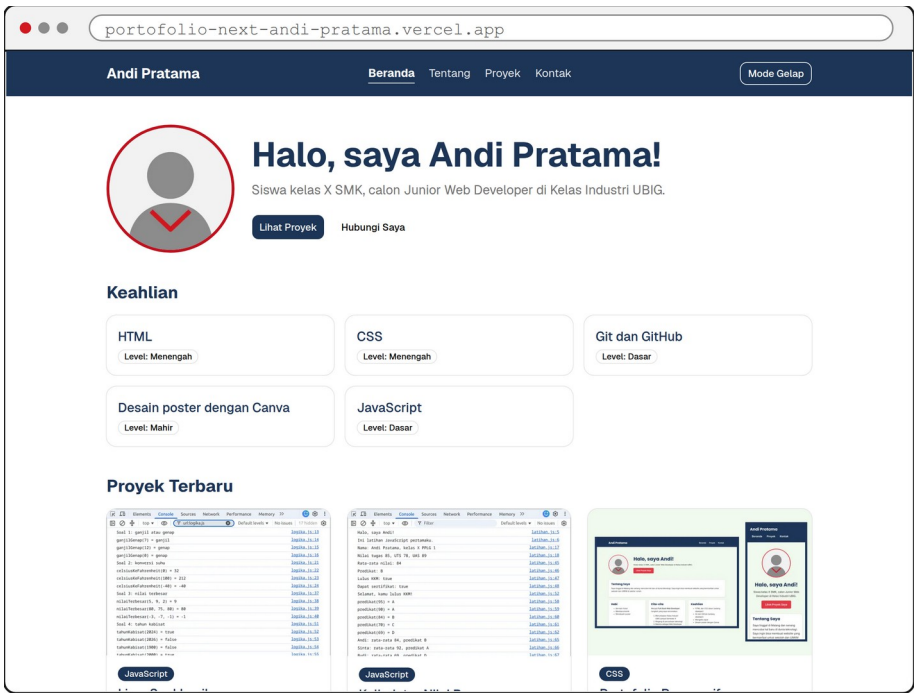
**5. Kirim.** Jalankan `npm run lint` dan `npm run build`, lalu kirim dengan perintah pada Kode 6.9. Vercel sudah punya *Environment Variables*, sehingga *build* langsung berhasil. Hasil *push* terlihat pada Gambar 6.22 dan versi *live*-nya pada Gambar 6.23.

**Kode 6.9** Commit dan push tugas Minggu 14

```
$ git add .
$ git commit -m "feat: tampilkan keahlian dari supabase dan lengkapi
komponen shadcn/ui"
$ git push
```




**Gambar 6.22** Push tugas Minggu 14




**Gambar 6.23** Contoh hasil tugas: Beranda versi live

Petunjuk tambahan untuk tugas ada pada Tabel 6.4.

**Tabel 6.4** Petunjuk tugas Minggu 14

| No | Kode atau Langkah | Kegunaan |
|---|---|---|
| 1 | `.from("skills").select("*").order("id")` | Mengambil semua keahlian, urut sesuai urutan dibuat |
| 2 | `{skills && skills.map(...)}` | Daftar hanya digambar jika data berhasil diambil |
| 3 | `<Badge variant="outline">` | Label bergaris tepi tanpa latar penuh |
| 4 | `buttonVariants({ variant: "outline", size: "lg" })` | Gaya tombol bergaris untuk `Link` **Hubungi Saya** |
| 5 | `className="... hover:bg-aksen ..."` pada `Button` | Kelas tambahan tetap boleh diberikan; kelas ini menimpa gaya *variant* |

### *6.7.2 Tantangan (Opsional)*

**Tantangan 1: data diperbarui berkala.** Halaman statis mengambil data hanya saat *build* (Kegiatan Pembelajaran 5). Tambahkan baris pada Kode 6.10 di atas fungsi halaman Proyek dan Beranda. Next.js akan mengambil ulang data paling cepat setiap 60 detik, tanpa *deploy* ulang. Jalankan `npm run build` dan perhatikan kolom *Revalidate* bertuliskan `1m` di samping kedua halaman, lalu jelaskan efeknya di video.

**Kode 6.10** Halaman diperbarui paling cepat setiap 60 detik

```
export const revalidate = 60;
```

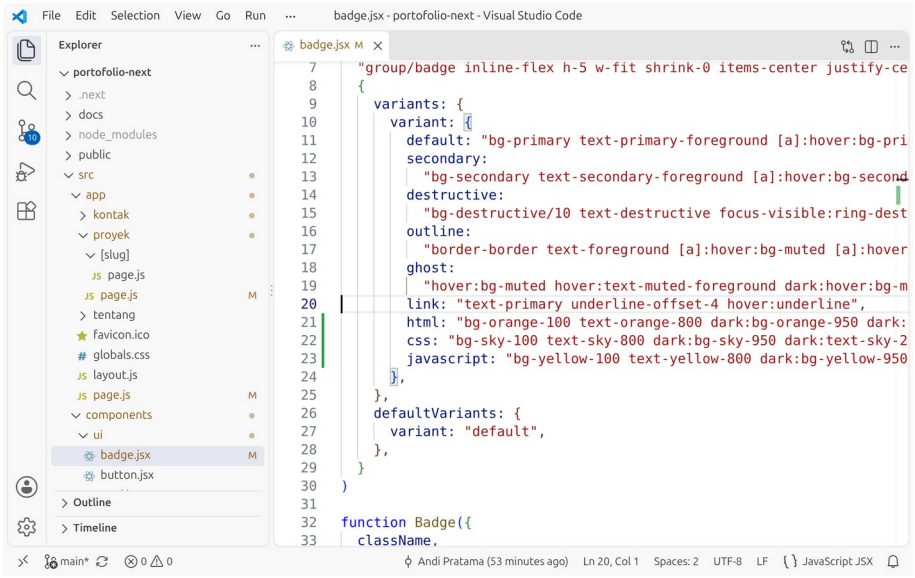
**Tantangan 2: Badge untuk setiap kategori.** Kode shadcn/ui adalah milikmu, jadi kamu boleh menambah *variant*. Di `src/components/ui/badge.jsx`, tambahkan tiga *variant* pada Kode 6.11 tepat di bawah *variant* `link`, lalu di `ProyekCard.jsx` ganti `Badge` menjadi Kode 6.12. Hasilnya terlihat pada Gambar 6.24 dan Gambar 6.25.

**Kode 6.11** badge.jsx: tiga variant kategori baru

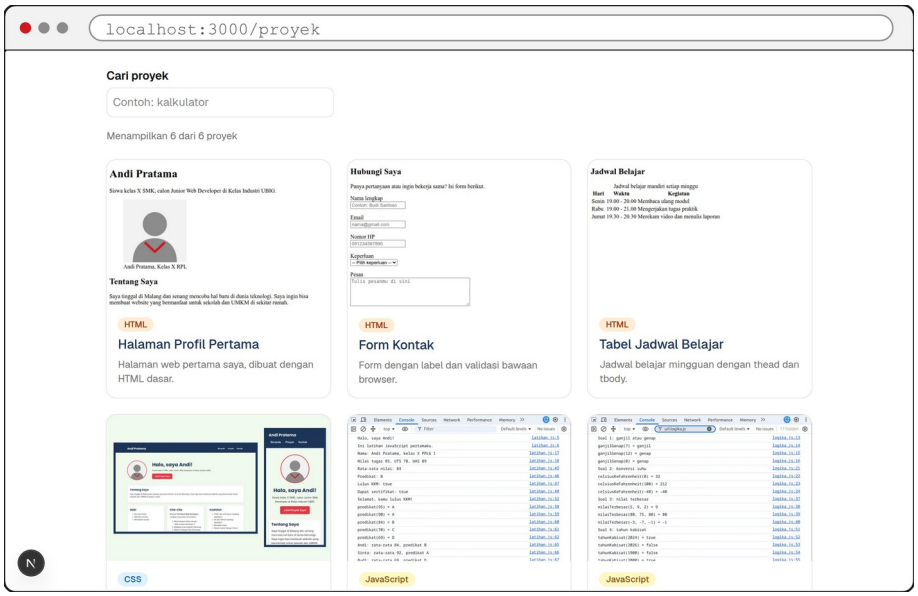
```
        html: "bg-orange-100 text-orange-800 dark:bg-orange-950
dark:text-orange-200",
        css: "bg-sky-100 text-sky-800 dark:bg-sky-950 dark:text-
sky-200",
        javascript: "bg-yellow-100 text-yellow-800 dark:bg-
yellow-950 dark:text-yellow-200",
```

**Kode 6.12** ProyekCard.jsx: variant mengikuti kategori

```
          <Badge variant={proyek.kategori.toLowerCase()}
>{proyek.kategori}</Badge>
```




**Gambar 6.24** Contoh hasil tantangan: variant baru di badge.jsx




**Gambar 6.25** Contoh hasil tantangan: Badge berwarna sesuai kategori

Nama *variant* ditulis dengan huruf kecil, sedangkan kategori di database diawali huruf besar. Karena itu kategori diubah dulu dengan fungsi `toLowerCase()`, misalnya `JavaScript` menjadi `javascript`. Tantangan tidak wajib di-*push*. Jika kamu

mengerjakannya, tuliskan hasilnya sebagai salah satu butir Pencapaian Praktikum di laporan.

### *6.7.3 Kriteria Selesai*

Tugasmu dianggap selesai jika memenuhi semua kriteria berikut:

- data proyek berasal dari Supabase, bukan array di kode (`src/data/proyek.js` sudah dihapus);
- detail proyek diambil berdasarkan *slug*, dan *slug* yang salah menampilkan 404;
- kartu proyek dan keahlian memakai komponen shadcn/ui dengan token tema, dan mode gelap tetap berfungsi;
- versi *live* menampilkan data dari database;
- tidak ada *key* di repositori (`.env.local` tidak ter-*commit*).

### *6.7.4 Ketentuan Pengumpulan P1-W14*

Isi laporan, tangkapan layar, dan video minggu ini mengikuti Tabel 6.5.

**Tabel 6.5** Ketentuan pengumpulan P1-W14

| Aspek | Ketentuan Minggu Ini |
|---|---|
| Kode minggu | `P1-W14` |
| Baris *Project* di laporan | `Project: Portofolio Pribadi – Modul P1-W14: Integrasi Next.js dengan Supabase dan shadcn/ui` |
| Tangkapan layar | 1. Halaman Proyek versi *live* dengan `Card` dan `Badge`: `P1W14_NamaSiswa_01_proyek.png`<br>2. Kode `src/lib/supabase.js` di VS Code: `P1W14_NamaSiswa_02_kode.png`<br>3. Beranda dengan keahlian dari Supabase: `P1W14_NamaSiswa_03_keahlian.png` |
| Link GitHub dan Vercel | Tautan repositori `portofolio-next` dan website `vercel.app` yang sudah membaca Supabase |
| Judul video | `[UBIG P1-W14] Connecting Next.js to Supabase with shadcn/ui \| [Nama Lengkap] – [Nama Sekolah]` |
| Isi demo video | Tunjukkan berkas `src/components/ui` dan `components.json`, tombol kategori dan mode gelap dengan token tema, `src/lib/supabase.js`, halaman Proyek dan Beranda yang membaca Supabase, halaman detail dan 404, *Environment Variables* di Vercel (hanya nama variabelnya), serta versi *live*. Jangan menampilkan isi `.env.local` |
| Tenggat | H-1 sebelum pertemuan Minggu 15, pukul 22.00 WIB |

Contoh tangkapan layar pertama sama dengan Gambar 6.14, contoh kedua sama dengan Gambar 4.4, dan contoh ketiga sama dengan Gambar 6.18.

### *6.7.5 Urutan Mengerjakan Tugas*

1. Selesaikan praktik Kegiatan Pembelajaran 1 sampai 5, lalu periksa dengan Tabel 6.3.
2. Kerjakan tugas nomor 1 sampai 4, lalu periksa tampilan di mode terang dan gelap.
3. Jalankan `npm run lint` dan `npm run build`, lalu lakukan *commit* dan *push*.
4. Pastikan Vercel berstatus Ready dan versi *live* menampilkan data dari Supabase.
5. Ambil 3 tangkapan layar sesuai Tabel 6.5, rekam video, unggah ke YouTube sebagai *Unlisted*, dan masukkan ke *playlist* Paket 1.
6. Tulis laporan dengan format baku di Panduan, lalu kirim bersama 3 tangkapan layar melalui Telegram sebelum tenggat.
7. Pelajari bab Evaluasi, karena soal pilihan ganda dan uraiannya menjadi kuis di awal pertemuan P1-W15.

# BAB VII
# EVALUASI

Soal pilihan ganda dan uraian berikut menjadi **kuis 10 menit** di awal pertemuan P1-W15 melalui Google Form dari mentor, dikerjakan perorangan tanpa membuka modul.

Soal praktik wajib dikerjakan sebagai **remedial** jika tugas minggu ini belum memenuhi kriteria selesai. Jika sudah memenuhi, soal praktik menjadi latihan tambahan. Ketentuan lengkap kuis dan remedial tercantum di dokumen **Panduan dan Ketentuan Kelas Industri UBIG** bab Penilaian.

## 7.1 Soal Pilihan Ganda

Pilih satu jawaban yang paling tepat.

1. Perbedaan utama shadcn/ui dengan pustaka komponen biasa adalah ....
   a. shadcn/ui hanya bisa dipakai di TypeScript
   b. kode komponen shadcn/ui disalin ke dalam project sehingga bisa diubah
   c. shadcn/ui tidak memakai Tailwind
   d. shadcn/ui harus dibeli
2. Perintah untuk menambah komponen `card` ke project adalah ....
   a. `npm install card`
   b. `npx shadcn@latest init card`
   c. `npx shadcn@latest add card`
   d. `npm run add card`
3. Setelah `shadcn init`, huruf halaman berubah menjadi Times New Roman karena ....
   a. Tailwind belum terpasang
   b. variabel huruf di `layout.js` belum bernama `--font-sans`
   c. internet terputus
   d. `components.json` terhapus
4. Kelas yang warnanya otomatis berganti di mode gelap tanpa awalan `dark:` adalah ....
   a. `bg-white`

b. `bg-gray-800`

c. `bg-card`

d. `bg-[#ffffff]`

5. Komponen untuk label kecil kategori pada kartu proyek adalah ....

   a. `Card`

   b. `Badge`

   c. `Separator`

   d. `Button`

6. Hasil setiap permintaan `supabase-js` berupa object berisi ....

   a. `data` dan `error`

   b. `html` dan `css`

   c. `url` dan `key`

   d. `props` dan `state`

7. Halaman yang memakai `await supabase` harus memiliki kata kunci ....

   a. `"use client"`

   b. `async` pada fungsinya

   c. `export const metadata`

   d. `useState`

8. Permintaan untuk mengambil satu proyek berdasarkan *slug* adalah ....

   a. `.select(slug)`

   b. `.limit(slug)`

   c. `.eq("slug", slug).single()`

   d. `.order("slug")`

9. Build pertama di Vercel gagal dengan pesan *supabaseUrl is required.* karena ....

   a. Supabase sedang mati

   b. `.env.local` tidak ikut ke GitHub dan Vercel belum punya *Environment Variables*

   c. `npm run lint` belum dijalankan

   d. nama repositori salah

10. Di Vercel, variabel *publishable key* yang berawalan `NEXT_PUBLIC_` disimpan dengan tipe ....

a. Secret
b. Config
c. Password
d. Private

### 7.2 Soal Uraian

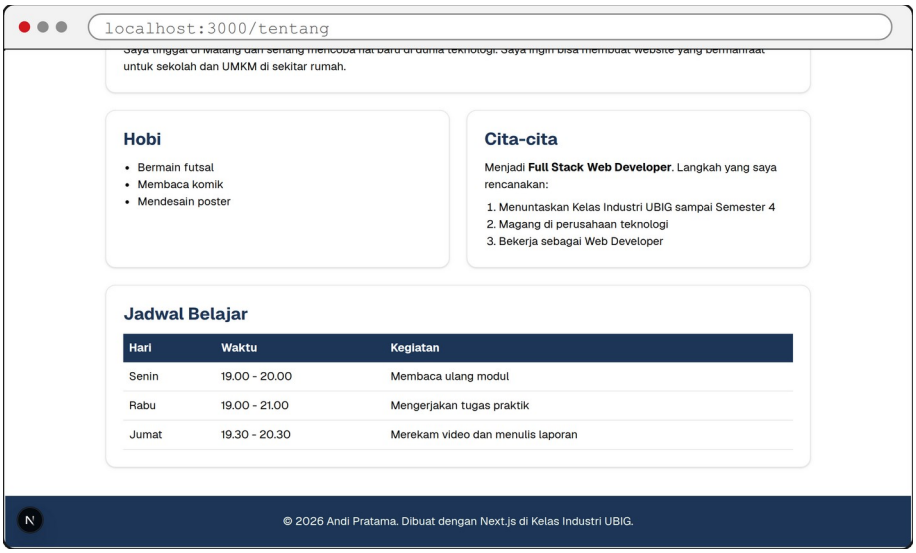
1. Jelaskan apa itu token tema, lalu bandingkan penulisan warna kartu di Minggu 12 (`bg-white dark:bg-gray-800`) dengan Minggu 14 (`bg-card`).
2. Jelaskan alur data dari tabel `projects` di Supabase sampai kartu proyek tampil di *browser*, sebutkan peran `src/lib/supabase.js` dan *server component*.
3. Jelaskan mengapa *build* Vercel gagal padahal di laptop berhasil, lalu tuliskan langkah memperbaikinya sampai versi *live* membaca Supabase.

### 7.3 Soal Praktik

Tampilkan jadwal belajar di halaman Tentang dari tabel `jadwal` Supabase (dibuat di soal praktik Minggu 13), dengan ketentuan:

1. jika tabel `jadwal` belum ada, buat dulu sesuai soal praktik Modul P1-W13 (kolom `hari`, `waktu`, `kegiatan`, tiga baris, dan *policy* baca publik);
2. di halaman Tentang (`src/app/tentang/page.js`), hapus konstanta `jadwal` dan impor `supabase`;
3. ubah fungsi `Tentang` menjadi `async`;
4. ambil data dengan `.from("jadwal").select("*").order("id");`
5. ganti `{jadwal.map(` menjadi `{jadwal && jadwal.map(;`
6. periksa dengan `npm run build`, lalu lakukan *commit* dan *push*.

Contoh hasilnya terlihat pada Gambar 7.1.




**Gambar 7.1** Contoh hasil soal praktik: jadwal belajar dari Supabase

## 7.4 Rubrik Penilaian Tugas

Rubrik ini menilai tugas rumah minggu ini (komponen praktik dan kode), bukan soal praktik di atas. Kriterianya tercantum pada Tabel 7.1. Tantangan bersifat opsional; tanpa tantangan, nilai tertinggi 90.

**Tabel 7.1** Rubrik penilaian tugas P1-W14

| No | Aspek | Kriteria Nilai Penuh | Skor |
|---|---|---|---|
| 1 | shadcn/ui dan token tema | shadcn/ui terpasang, huruf Geist normal, tidak ada pasangan `dark:` tersisa selain pesan berhasil, mode gelap berfungsi | 20 |
| 2 | Data proyek dari Supabase | Halaman Proyek, Beranda, dan detail membaca Supabase; pesan *error* dan data kosong ada; `src/data/proyek.js` dihapus | 25 |
| 3 | Keahlian dan komponen lain | Keahlian dari tabel `skills`; `SkillCard`, `Kartu`, Kontak, Hero, tombol tema, dan tombol Kirim memakai shadcn/ui | 20 |
| 4 | Versi *live* | *Environment Variables* bertipe Config, Vercel Ready, website *live* menampilkan data Supabase | 15 |
| 5 | GitHub | *Commit* praktik dan tugas ada di `portofolio-next`; `.env.local` tidak ter-*commit* | 10 |
| 6 | Tantangan | `revalidate` atau *variant* `Badge` per kategori berjalan (nilai tambah) | 10 |

# BAB VIII
# PENUTUP

Selamat, portofoliomu kini benar-benar *full stack*! Minggu ini kamu memasang shadcn/ui, merapikan warna dengan token tema, menyusun kartu dengan `Card` dan `Badge`, menghubungkan Next.js ke Supabase dengan `supabase-js`, menangani *error* dan data kosong, membuat halaman detail dari database, lalu menerbitkannya ke Vercel dengan *Environment Variables*. Data portofolio tidak lagi ditulis di kode.

Sejauh ini website hanya **membaca** data. Di Minggu 15 kamu membuat website **menulis** data: form Buku Tamu menyimpan pesan ke tabel baru `buku_tamu` dengan *Server Actions*, dan halaman kelola yang dijaga PIN menyematkan, menghapus, serta mengubah data. Form-nya disusun dari komponen shadcn/ui `Input`, `Textarea`, dan `Label`, dan penghapusan dikonfirmasi dengan `AlertDialog`. Sampai jumpa di pertemuan berikutnya!

# GLOSARIUM

| Istilah | Pengertian |
|---|---|
| **Badge** | Komponen label kecil untuk menandai kategori atau status, misalnya kategori proyek. |
| **Base UI** | Pustaka yang mengurus perilaku komponen shadcn/ui, seperti fokus *keyboard* dan aksesibilitas. |
| **Card** | Komponen wadah berbingkai yang tersusun dari `CardHeader`, `CardTitle`, `CardDescription`, `CardContent`, dan `CardFooter`. |
| **Client Supabase** | Objek hasil `createClient` yang menyimpan alamat dan kunci project untuk mengirim permintaan ke Supabase. |
| **components.json** | Berkas pengaturan shadcn/ui yang menyimpan gaya, letak `globals.css`, dan alias folder komponen. |
| **Config** | Tipe *Environment Variable* Vercel yang nilainya boleh dibaca anggota project, untuk nilai yang aman diketahui umum. |
| **Environment Variables** | Pengaturan nilai rahasia atau konfigurasi di Vercel yang dibaca saat *build*, pengganti `.env.local` di laptop. |
| **Halaman dinamis** | Halaman yang dibuat ulang setiap ada pengunjung, ditandai *ƒ* pada hasil *build*. |
| **Halaman statis** | Halaman yang dibuat sekali saat *build*, ditandai ○ pada hasil *build*. |
| **Pustaka komponen** | Kumpulan komponen antarmuka siap pakai, seperti tombol, kartu, dan form. |
| **Redeploy** | Menjalankan ulang *build* dan *deploy* sebuah *deployment* di Vercel, misalnya setelah *Environment Variables* berubah. |
| **revalidate** | Pengaturan Next.js agar halaman statis diambil ulang datanya setelah jangka waktu tertentu. |
| **Separator** | Komponen garis pemisah tipis di antara dua bagian halaman. |
| **shadcn/ui** | Kumpulan komponen antarmuka siap pakai yang kodenya disalin ke project sehingga bebas diubah. |
| **single** | *Method* `supabase-js` untuk mengambil tepat satu baris sebagai object. |
| **supabase-js** | Pustaka resmi JavaScript untuk membaca dan menulis data Supabase. |
| **Token tema** | Nama warna di `globals.css` yang nilainya berbeda di mode terang dan gelap, misalnya `--card` untuk kelas `bg-card`. |
| **Variant** | Pilihan gaya sebuah komponen, misalnya `default`, `outline`, dan `ghost` pada `Button`. |

# DAFTAR PUSTAKA


Next.js. (2026). *Environment variables*. Next.js Docs. https://nextjs.org/docs/app/guides/environment-variables

Next.js. (2026). *Fetching data*. Next.js Docs. https://nextjs.org/docs/app/getting-started/fetching-data

shadcn. (2026). *Card*. shadcn/ui. https://ui.shadcn.com/docs/components/card

shadcn. (2026). *Installation: Next.js*. shadcn/ui. https://ui.shadcn.com/docs/installation/next

shadcn. (2026). *Theming*. shadcn/ui. https://ui.shadcn.com/docs/theming

Supabase. (2026). *JavaScript client library: Fetch data*. Supabase Docs. https://supabase.com/docs/reference/javascript/select

Supabase. (2026). *Use Supabase with Next.js*. Supabase Docs. https://supabase.com/docs/guides/getting-started/quickstarts/nextjs

Vercel. (2026). *Environment variables*. Vercel Docs. https://vercel.com/docs/environment-variables